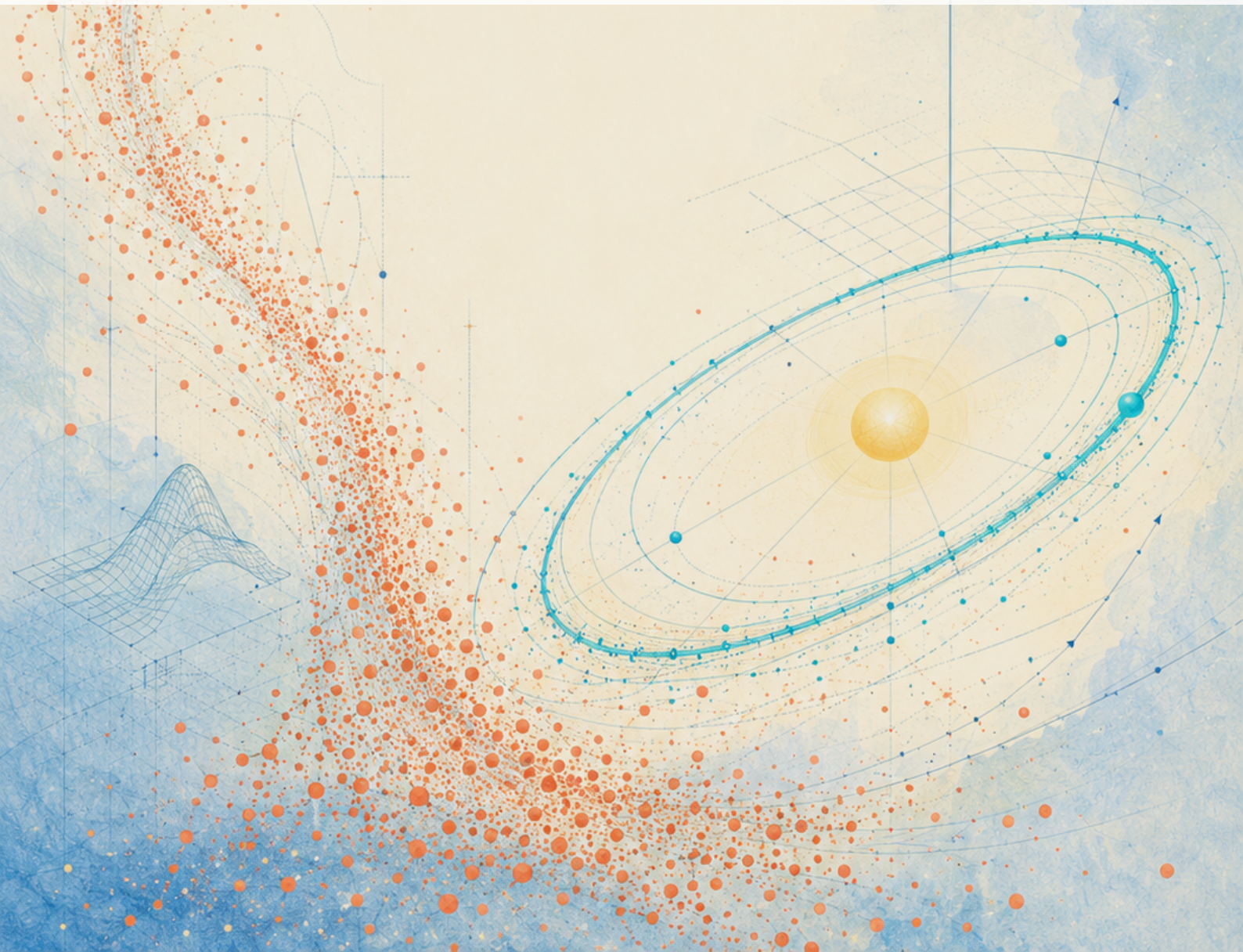
برمجة الفيزياء

الحوسبة العلمية باستخدام Julia و Python

77641



المحتويات

-1 عن المقرر وهذه المواد 1
-1.1 خريطة العام 1
-1.2 المواد التي ترافقنا كل أسبوع 1
-1.3 كيف نقرأ الكتاب 2
-1.4 لماذا رقم هذا الفصل -1؟ 2

0 مقدمة: الفيزياء في عصر الحوسبة 3
0.1 لماذا نستخدم الحواسيب في الفيزياء؟ 3
0.1.1 نبدأ بالقياسات والحركة 3
0.1.2 نجم يمزّقه ثقب أسود 3
0.1.3 متابعة مرشّح لبوزون هيغز في CERN 4
0.2 ما هو البرنامج؟ وما هو الحاسوب؟ 5
0.3 لماذا نبدأ بـJulia؟ 5
0.4 لماذا Python؟ 7
0.4.1 مثال كامل: الإشارة في موقعي LIGO 7
0.5 لماذا نتعلم اللغتين معًا؟ 9
0.6 كيف يُنظَّم العام الدراسي؟ 9
0.7 ما الذي سنتعلمه؟ 9
0.8 تشويق: إعادة تحليل كبلر للمريخ 10
0.8.1 الحصول على مستودع المقرر 11
0.9 بيئة المقرر: نُثبّت مرة ونشغّل بتكرار قابل للإنتاج 11

1 حسابات Julia الأولى وملاءمة أوهلية موجَّهة 13
1.1 أين نرى النتيجة؟ 13
1.2 Julia كآلة حاسبة 13
1.3 أسماء للقيم قبل الفيزياء 14
1.4 سير عمل موجَّه من القياس إلى النموذج 14
1.4.1 نبني التحليل خطوة مرئية في كل مرة 16
1.5 التمرين: قياس ثابت النابض 21
1.6 الواجب الأساسي: استنتاج الجاذبية من قياسات السقوط الحر 22
1.7 مكافأة اختيارية: اجعل رسمًا علميًا يوصّل الفكرة 22
1.8 استكشاف إضافي: منحنيات ليساجو 22

2 الأسبوع 2: القيم والقرارات وجسم على منحدر 25
2.1 محاضرة: من المتغيرات إلى الخوارزمية 25
2.1.1 المتغير يمنح قيمةً اسمًا 25
2.1.2 تتابع Julia البرنامج بالترتيب 26
2.1.3 يمكن إسناد قيمة جديدة لمتغير 26
2.1.4 للقيم أنواع 26
2.1.5 أربعة أنواع قيم بسيطة 27
2.1.6 تنتج المقارنات قيمًا منطقية 27
2.1.7 قرار أول باستخدام if 28
2.2 الحركية متعددة القطع كقرار 28
2.2.1 عدة فروع 29
2.3 للقيم أنواع 30
2.4 القيم المنطقية والقرارات التدريجية 31
2.5 الزوايا والمنطق والتحقق قبل الحساب 32
2.6 نموذج احتكاك متعدد القطع على منحدر 33
2.7 التمرين: قوة سحب قد تعكس حركة وشيكة 34
2.8 الواجب الأساسي: هل يعبر المقذوف جدارًا؟ 34
2.9 مكافأة اختيارية: الوحدات وأطوار التيار المتردد 34
2.10 استكشاف إضافي: الانعكاس الداخلي الكلي 35

3 الحلقات والمتجهات والحركة المرصودة 37
3.1 المتجهات تخزّن تواريخًا مرتبة 37
3.1.1 إنشاء المتجهات 37
3.1.2 الفهرسة والتقطيع 38
3.1.3 تغيير متجه 39
3.1.4 حساب المتجهات والدوال المفيدة 39
3.2 تكرار مخطَّط: حلقة for 40
3.3 الملخصات: المتوسط والانحراف المعياري للعينة 43
3.4 تكرار متحكم فيه بشرط: while 43
3.5 من المشتقة إلى تحديث أويلر 44
3.6 التنقيح: المقارنة مع جواب معلوم 46
3.7 تمرين: أول زمن تبريد مرصود 47
3.8 الواجب الرئيسي: رمي مطرقة مع مقاومة هواء 47
3.9 مكافأة اختيارية: ثلاث صيغ تكامل 48
3.10 استكشاف إضافي: من مشية واحدة إلى انتشار 48

4 المصفوفات والحلقات والدورانات 49
4.1 كتابة المصفوفات وأبعادها 49
4.1.1 فهرسة المصفوفة 49

4.2 حلقات while: نكرّر ما دام الشرط صحيحًا 50
4.2.1 إدخال قيمة حارسة: متوسط الصف 50
4.2.2 التحقق من صحة المدخلات 51
4.2.3 لعبة تخمين 51
4.2.4 توقيت حلقة while: المتسلسلة التوافقية 52
4.3 جداء المصفوفة بالمتجه وبالمصفوفة 53
4.4 الحلقات المتداخلة: جدول الضرب 53
4.5 الحلقات المتداخلة: ضرب المصفوفات 54
4.6 التحقق من تماثل المصفوفة 55
4.7 مصفوفات الدوران في بعدين 56
4.7.1 التحقق من أن المصفوفة تمثل دورانًا فعلًا 57
4.8 دوران مستطيل في المستوى 57

5 دوال تُجري حسابات الفيزياء 61
5.1 من حساب إلى دالة 61
5.2 النطاق: أين يوجد الاسم؟ 62
5.3 الوسائط الموضعية والمسمّاة وقيمها الافتراضية 63
5.4 إعادة قيمة 64
5.5 التعديل في المكان واصطلاح علامة ! 65
5.6 البث: التطبيق عنصرًا عنصرًا 65
5.7 صيغ بناء المصفوفات بالاستيعاب 66
5.8 القواسم والأعداد الأولية 66
5.9 تكامل حركة بندول بطريقة أويلر 67
5.10 الزمن الدوري بدلالة زاوية الإطلاق 68
5.11 نظرة إلى الخطوة التالية 69



1-. عن المقرر وهذه المواد

لماذا نتعلّم البرمجة ونستخدم الحواسيب في الفيزياء؟ إنها تتيح لنا تحويل النموذج إلى حساب يمكن تكراره وتغييره ومقارنة نتائجه بالقياسات. طوال المقرر سنستخدم الحوسبة لربط أفكار الفيزياء والرياضيات بالبيانات والطرائق العددية والأسئلة العلمية التي نريد استكشافها.

1.1- خريطة العام

سنبدأ بلغة Julia وبحسابات صغيرة وواضحة تساعدنا على اكتساب الثقة: قراءة جدول مُعطى، ورسم القياسات، وملاءمة خط مستقيم، وتتبع حركة بسيطة. بعد ذلك سنكتب خوارزمياتنا للطرائق العددية، ونقارنها بالحزم البرمجية المعروفة، ونجمعها في استقصاءات أوسع. وفي مرحلة لاحقة سنستخدم Python لننقل الأفكار العلمية التي تعلّمناها إلى لغة أخرى وبيئتها البحثية الواسعة. يقدّم الفصل 0 موضوع البرمجة في الفيزياء، ومن الفصل 1 فصاعدًا يطابق رقم الفصل رقم أسبوع المقرر.

1.2- المواد التي ترافقنا كل أسبوع

لكل مادة أسبوعية دورها الخاص:

- **كتاب المحاضرات والدفتر:** يطوّر الكتاب الأفكار خطوةً خطوة، مع أمثلة محلولة ومخرجاتها. ويتبع ذلك دفتر محاضرة كامل قابل للتشغيل، فنستطيع تنفيذ الأمثلة واستكشافها. وفتح الدفتر للتشغيل اختياري، ويمكن في VS Code مع إضافة Jupyter أو في JupyterLab؛ كما تتضمّن الدفاتر مخرجات محفوظة. أما شرائح Reveal فتساند المحاضرة داخل الصف.
- **دليل التمرين والدفتر:** نبدأ بمراجعة موجزة ومثال محلول قصير، ثم نخصّص معظم وقت التمرين للعمل على الواجب الأساسي بمساعدة طاقم التدريس. وتبقى ورقة الواجب نفسها وثيقة مستقلة.
- **الواجب الأساسي:** يتيح لنا الواجب الأسبوعي ممارسة ما تعلّمناه، ويرافقه حلّ يدعم التدريس والمراجعة. ونقدّمها في صفحة المقرر على Moodle، حيث يوجد الواجب ورابط التسليم. ويُصحَّح الواجب الأول يدويًا في الأسبوع الأول: نرفع سكربتًا وشكلًا. ومن الأسبوع الثاني، تُصحّح أسئلة البرمجة المناسبة في CodeRunner آليًا بتشغيل اختبارات معلنة على الإجابة المقدّمة، ويوضح السؤال ما ينبغي تقديمه وما الذي تفحصه الاختبارات.
- **التعلّم الاختياري:** يضيف كل تمرين مكافأة نجزه ثلث نقطة إلى العلامة النهائية. والمكافأة اختيارية وغير ممتحنة ولا تشترط معرفةً نحتاجها في أسبوع لاحق. ويقترح قسم مستقل بعنوان استكشاف إضافي مواد للتعلّم الذاتي خارج صلب المقرر، بينما

تساعدنا أسئلة المراجعة الأسبوعية على التحقق من فهمنا استعدادًا للامتحان.

1.3- كيف نقرأ الكتاب

تمنحنا الصناديق الملوّنة فرصةً للتوقف عند فكرة تحتاج إلى شرح مستقل:

- **صناديق الحوسبة الرمادية المائلة إلى الأزرق** تشرح أدوات البرمجة أو اصطلاحاتها أو سير العمل.
- **صناديق الفيزياء البنفسجية** تعرض النموذج الفيزيائي وكمياته وحدوده.
- **صناديق الرياضيات الخضراء** تطوّر علاقة رياضية أو اشتقاقًا.
- **الصناديق البرتقالية** تقدّم تلميحًا أو تنبيهًا أو فكرة يجدر تذكّرها.

تظهر الشيفرة في صناديق زرقاء فاتحة. ويعرض الصندوق الأخضر الفاتح أسفل المثال البرمجي مخرجاته. نقارن المخرجات بالشرح الذي يليها، ثم نتبع الطريقة نفسها عندما نشغّل دفترًا بأنفسنا.

1.4- لماذا رقم هذا الفصل 1−؟

يأتي هذا الدليل قبل أول فصل مرقّم في المقرر، لذلك يحمل الرقم 1−. الفصل 0 هو مقدمتنا إلى البرمجة في الفيزياء. والأسبوع 1 هو الفصل 1، والأسبوع 2 هو الفصل 2، ويستمر هذا التوافق طوال العام. وهكذا يشير الحديث عن الأسبوع 6 أو الفصل 6 إلى الجزء نفسه من المقرر.



0. مقدمة: الفيزياء في عصر الحوسبة

0.1 لماذا نستخدم الحواسيب في الفيزياء؟

لماذا نحتاج البرمجة والحواسيب في الفيزياء؟ إنها تساعدنا على تتبّع فكرة من النموذج إلى الحساب، ومن الحساب إلى قياسات نستطيع فحصها. ويمكننا أن نبدأ بأسئلة مألوفة ثم نرى كيف تصل المقاربة نفسها إلى مسائل على حدود البحث.

0.1.1 نبدأ بالقياسات والحركة

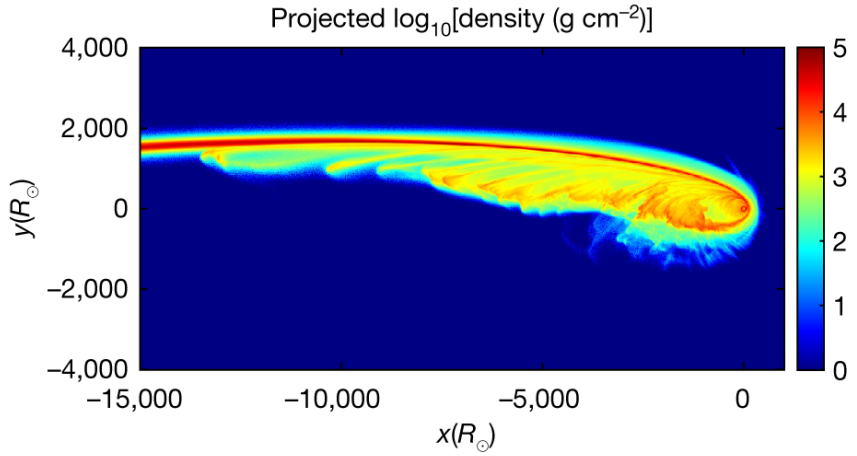
لنفترض أننا نقيس الجهد والتيار في موصل. يتيح لنا الرسم أن نرى كيف تتغيّر الكميتان معًا، ويعطينا ملاءمة خط مستقيم تقديرًا للمقاومة عندما يسلك الموصل سلوكًا أوميًا. في الأسبوع الأول سنتابع مثالًا مُعطى ينتقل من القياس إلى الرسم إلى الملاءمة. وبعد ذلك سنبني طريقة الملاءمة بأنفسنا ونفحص كيف تؤثر البواقي وعدم اليقين في القياس على الاستنتاج.

في الميكانيكا، يقودنا نموذج مقذوف بسيط إلى مسار قطع مكافئ يمكننا حسابه بمعادلات مألوفة. ماذا يتغيّر عندما تعتمد مقاومة الهواء على السرعة، أو عندما تتغيّر كثافة الهواء مع الارتفاع؟ إن إضافة هذين التأثيرين تربط الموضع بالسرعة في معادلات الحركة. ويدفع التكامل العددي هذه المعادلات إلى الأمام بخطوات زمنية صغيرة، ما يتيح لنا مقارنة المسار المثالي بتنبوء أكثر تفصيلًا.

0.1.2 نجم يمزّقه ثقب أسود

ماذا يحدث عندما يمرّ نجم قريبًا جدًا من ثقب أسود هائل؟ يجذب الثقب الأسود الجانب القريب من النجم بقوة أكبر من الجانب البعيد. وعندما يتغلّب هذا الفرق المديّ على جاذبية النجم الذاتية، يتمزّق النجم. وتمتدّ غازاته إلى تيارات تعود وتصطدم وتسخن وتبدأ في تكوين تدفق تراكمي.

تخبرنا التقديرات التحليلية من أين يبدأ التمزّق وتحدّد المقاييس الزمنية المدارية المميّزة. ولمتابعة التيارات العائدة وهي تصطدم وتشكّل صدمات وتشعّ، يحلّ الباحثون معادلات هيدروديناميكية وإشعاعية مترابطة عدديًا عبر الفضاء والزمن الثلاثي الأبعاد. ويعرض الشكل أدناه كثافة العمود المسقطة الناتجة عن حساب من هذا القبيل أجراه عضوان من قسمنا؛ ويبيّن سلّمه اللوني أين تتركّز الغازات المتمزّقة. وتتنبأ هذه المحاكاة أيضًا بكيفية تغيّر ضوء الحدث مع تطور التدفق، وتتيح لنا تقديم تنبؤات يستحيل تمامًا الحصول عليها بالحسابات التحليلية.

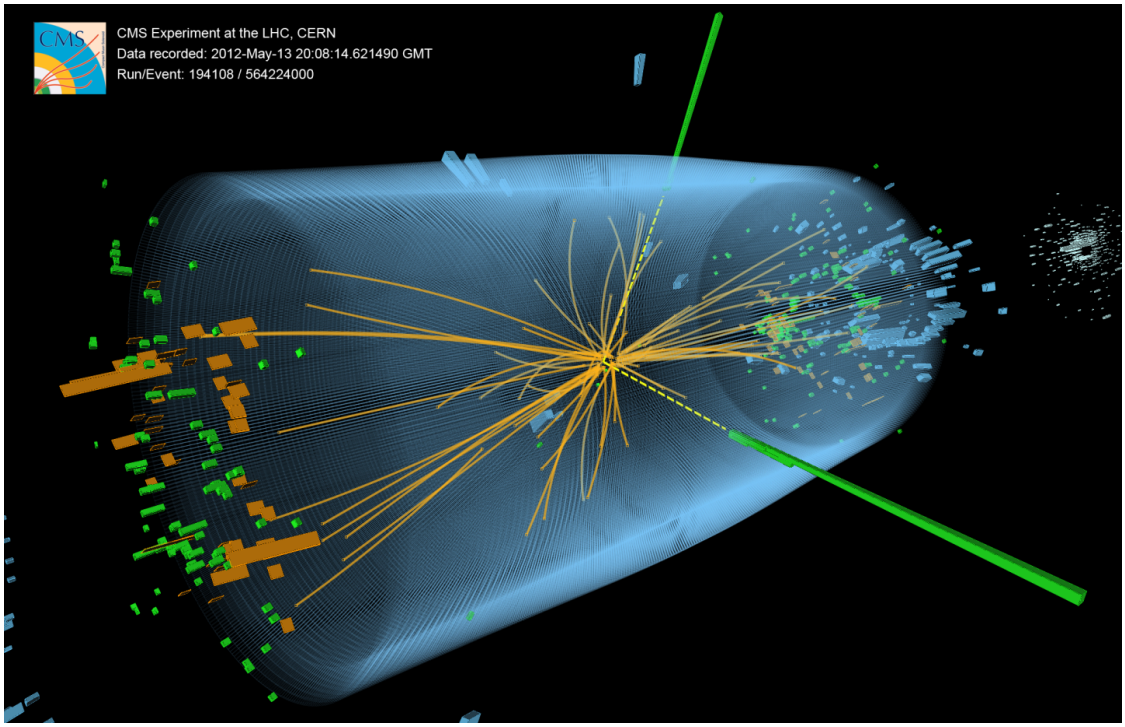


لوغاريتم كثافة العمود المسقطة خلال حدث تمزّق مديّ محاكى؛ وتُقاس المواضع بأنصاف أقطار الشمس. من *E. Steinberg and N. C. Stone, "Stream–disk shocks as the origins of peak light in tidal disruption events," Nature* 625, 463–467 (2024)، الشكل 1، *doi:10.1038/s41586-023-06875-y*.

0.1.3 متابعة مرشّح لبوزون هيغز في CERN

في CERN، يتكوّن بوزون هيغز في تصادمات جسيمات نادرة جدًا ويتحلّل فورًا تقريبًا. وتسجّل كواشف CERN ملايين أحداث التصادم. ولكل حدث منها يجب حساب ديناميكيات جميع الجسيمات المشاركة انطلاقًا من الطاقات والاتجاهات المقيسة للجسيمات الناتجة. وهذه مهمة مستحيلة تمامًا على البشر العاديين، ولهذا يصبح التحليل المحوسب ضروريًا.

يعرض الشكل أدناه حدثًا معينًا أنتج فوتونين، وهو مرشّح مرجّح جدًا لتكوّن جسيم هيغز وتحلّله. وقد افتُرض وجود هذا الجسيم لتفسير كيف اكتسبت بعض الجسيمات كتلتها.



مرشّح فوتونين من *CMS* في البحث عن بوزون هيغز، مرسوم من إشارات الكاشف المعاد بناؤها. الصورة: *CMS/CERN*، نظرة عامة من *CERN* على بوزون هيغز.

يوضّح لنا هذان المثالان كيف يمكن استخدام الحواسيب لتحليل البيانات ونمذجة الأنظمة بطرق تختلف نوعيًا وكميًا، فتقدّم لنا نحن الفيزيائيين أدوات إضافية.

0.2 ما هو البرنامج؟ وما هو الحاسوب؟

البرنامج تسلسل من التعليمات يخبر الحاسوب كيف ينفّذ مهمة. ففي مثال المقذوف، يمكن أن يبدأ بموضع وسرعة، ويحسب التسارع، ويحدّث الحركة عند أزمنة متتالية. ولكتابة تلك التعليمات، نحتاج إلى تحديد البيانات التي نخزّنها، وكيف نحسب بها، ومتى نكرّر أو نتوقّف. وغالبًا ما يستطيع القارئ البشري أن يكمل خطوة ناقصة في اشتقاق؛ أما الحاسوب فيحتاج إلى تلك الخطوة مكتوبة صراحةً.

الحاسوب آلة فيزيائية تنفّذ هذه التعليمات مرارًا. فمعالجه المركزي (CPU) ينفّذ العمليات الحسابية والمنطقية؛ والذاكرة (RAM) تحمل البيانات والتعليمات قيد الاستخدام؛ والتخزين الدائم يحفظ الملفات بعد انتهاء البرنامج؛ وأجهزة الإدخال/الإخراج تصل الآلة بالملفات والشاشات والأجهزة والشبكات. وعندما نشغّل سكربت Julia، يترجم نظام لغة Julia تعليماتنا إلى عمليات يستطيع المعالج تنفيذها، بينما يدير نظام التشغيل الملفات والذاكرة والوصول إلى الشاشة.

بينما نقرأ برنامجًا، يمكننا تتبّع البيانات من القيم الابتدائية عبر الحسابات وصولًا إلى النتيجة. وتصف الخوارزمية خطوات ذلك الحساب، ويساعدنا النموذج الفيزيائي على تفسير ما ينتج. وسنبني طريقة العمل هذه تدريجيًا. فيقدّم الأسبوع 2 الأنواع وتحويل المدخلات والقرارات، ويقدّم الأسبوع 3 الحلقات وقواعد التوقّف، ويقدّم الأسبوع 5 تعريفات الدوال.

0.3 لماذا نبدأ بـJulia؟

صُمّمت Julia للعمل العددي والعلمي. لديها تدوين مباشر للمتجهات والمصفوفات والجبر الخطي وتعددية الإرسال وحلّالات المعادلات التفاضلية، مع بقائها مناسبة للسكربتات والدفاتر الاستكشافية. وهي سريعة بما يكفي بحيث تكون النسخة الأولى القابلة للقراءة من نموذج عددي مفيدة غالبًا بالفعل.

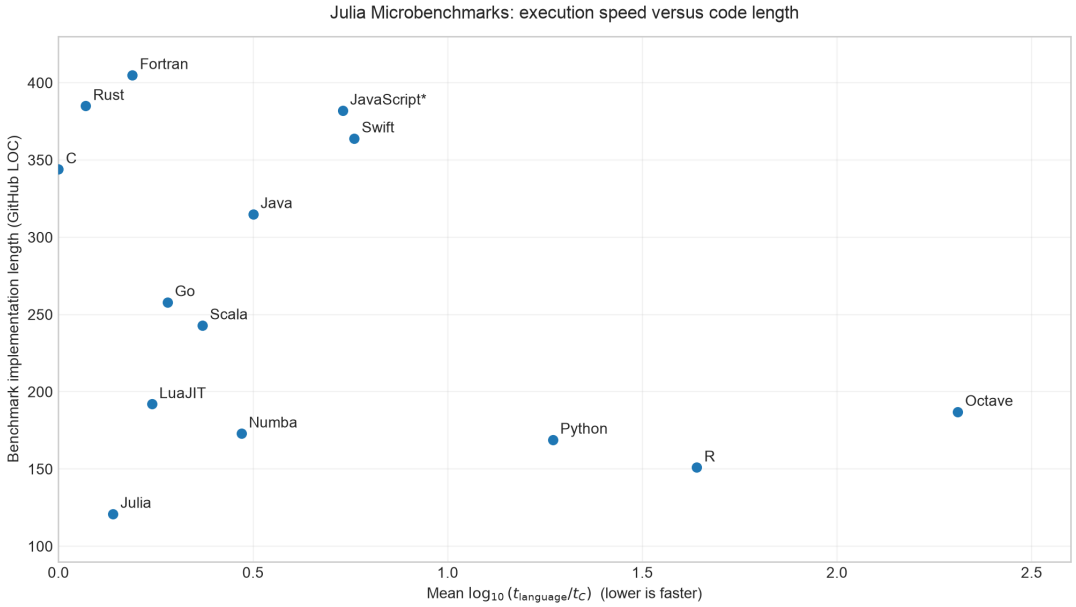
بالنسبة لنا كفيزيائيين، الفائدة الأولى المهمة هي أن الكود يمكن أن يبدو مثل النموذج. فمتجه القياسات، والمصفوفة التي تمثّل تحويلًا خطيًا، ومصفوفة قيمٍ مُعيّنة زمنيًا، كلها كائنات عادية في Julia. والتمييز بين قيمة مفردة ومتجه وحسابٍ عنصرًا بعنصر واضح في التدوين. وهذا يجعل الأبعاد والفهارس والوحدات والعملية الفيزيائية المقصودة أسهل في الفحص قبل أن يُدفن خطأ عددي داخل برنامج طويل.

تتّصف Julia بأنها معبّرة لأن تدوينها يتيح لنا كتابة العملية قريبة من الرياضيات، وقابلة للتركيب لأن العمليات المحدّدة جيدًا وأدوات الحزم يمكن دمجها في حساب أكبر. فمثلًا يمكن أن تدخل عملية على مصفوفة في حساب آخر، بينما يمكن تعريف الفكرة العددية نفسها للأعداد والمتجهات والمصفوفات، أو لاحقًا للقيم التي تحمل وحدات. وتختار Julia الطريقة المناسبة وفق أنواع المدخلات، وتُسمّى هذه الفكرة تعددية الإرسال. سنستخدم الأدوات العلمية أولًا، ثم نعود إلى هذه الفكرة التصميمية عندما تساعدنا على شرح كيف تعمل عملية واحدة في عدة سياقات فيزيائية.

يمكن لمشاريع Julia أن تسجّل الحزم والنسخ المستخدمة في تحليلٍ ما في ملفات المشروع. وهذا يجعل من العملي العودة إلى حسابٍ بعد أشهر أو مشاركته مع زميل دون تغيير صامت لأدوات الحساب التي يقوم عليها. وللغة أيضًا مكتبة قياسية قوية للجبر الخطي العددي ومنظوم بيئي متنامٍ للمعادلات التفاضلية والاستمثال والإحصاء والوحدات والرسم وتعلّم الآلة والبيانات العلمية. سنبني الطرق الأساسية بأنفسنا أولًا، ثم نستخدم تلك الحزم كتطبيقات مُفحَصة بعناية لأفكار فهمناها بالفعل.

السرعة مهمة في الفيزياء لأنها تتيح لنا اختبار نموذج، وفحص نتيجة، وتغيير افتراض، والتكرار. ويلخّص الشكل أدناه مجموعة الاختبارات المصغّرة لمشروع Julia. يقارن محوره الأفقي زمن التنفيذ بلغة C على مقياس لوغاريتمي، فالنقاط الأبعد إلى اليسار أسرع؛ أما محوره الرأسي فهو طول تطبيق الاختبار. وتقع Julia قرب اللغات المترجَمة في هذه الأنوية العددية الصغيرة بينما

يبقى كود مثالها مدمجًا. وتصف هذه التوقيتات الأنوية المختارة؛ أما سرعة برنامج كامل فتعتمد أيضًا على خوارزميته وتوزيع بياناته وتخصيصاته وطريقة قياسنا له. والبيانات والبرامج متاحة في مستودع اختبارات Julia المصغّرة.



والأهم في هذا المقرر أن Julia تتيح للكود أن يتبع البنية الرياضية عن قرب. لنفترض أن نموذجًا ديناميكيًا خطيًا كُتب هكذا

$$\mathbf{x}_{n+1} = A\mathbf{x}_n + B\mathbf{u}_n, \qquad \mathbf{y}_n = C\mathbf{x}_n.$$

فإن تطبيق Julia شبه مطابق:

```
x_next = A * x + B * u
y = C * x
```

هنا x وu وy متجهات، بينما A وB وC مصفوفات. يمكننا مقارنة كل سطر من الكود بمعادلته مباشرة. وبينما نفعل ذلك، يساعدنا فحص أبعاد المصفوفات والوحدات الفيزيائية على رؤية ما إذا كان الحساب يعبّر عن النموذج الذي قصدناه.

وهناك تفاصيل عملية نعتاد عليها. فقد يستغرق التشغيل الأول لكود Julia الجديد وقتًا أطول بينما تترجم Julia كود آلة متخصصًا؛ وقد تكون التشغيلات التالية أسرع بكثير. ومجتمعها أصغر من مجتمع Python، لذا قد تؤثر الحزم المتاحة في اختيارنا للأداة. وسنتعلّم قياس الأداء، وسنستخدم Python عندما يناسب منظومها البيئي المهمة.

تدعم Julia أيضًا إدخال رموز يونيكود بأسلوب LaTeX في حلقة القراءة–التقييم–الطباعة (REPL) وفي VS Code. ولكتابة تعبير قياسي مثل

$$v_1' \cdot (A v_2),$$

نكتب أمرًا بشرطة مائلة للخلف ونضغط Tab بعد كل رمز:

- `\alpha<Tab>` ينتج α.
- `\cdot<Tab>` ينتج $\cdot$.
- `v\_1<Tab>\prime<Tab> \cdot<Tab> (A * v\_2<Tab>)` ينتج $\mathrm{v}_1' \cdot (\mathrm{A} * \mathrm{v}_2)$.

وهكذا نحصل على الشكل الرياضي الموضّح أعلاه: يمكن لأسماء المتغيرات أن تحمل الحروف اليونانية نفسها، والفهارس السفلية، والفواصل العليا، والمؤثرات التي تظهر في الحساب.

قراءة المعادلات إلى جانب الكود

جرّب مطابقة سطر من الكود بالخطوة المقابلة في النموذج. وغالبًا ما تكون هذه طريقة مفيدة لاكتشاف خطأ في إشارة، أو فهرس في غير موضعه، أو أبعاد غير متوافقة، وللتحقق من الافتراضات التي يستخدمها الحساب.

0.4 لماذا Python؟

Python هي اللغة العامة المهيمنة في المنظوم البيئي للبيانات العلمية. وتُستخدم مكتباتها على نطاق واسع للمصفوفات العددية والرسم والاستمثال والفلك وتعلّم الآلة والجبر الرمزي وتحليل الصور والتحكم التجريبي. سنقدّم Python بعد أن ترسخ الأفكار الحاسوبية الأساسية، حتى نتمكّن من التعرّف على سير العمل العلمي نفسه بلغة ثانية.

وتكتسب Python قيمة خاصة لأنها تصل التحليل ببيئة البحث الأوسع. فقد يوفّر مختبر سكربت Python لجهاز، أو يشارك زميل دفتر Jupyter، أو ينشر أرشيف عام مثالًا يستخدم مكتبة Python متخصصة. ويتيح لنا تعلّم Python قراءة سير العمل هذه وتكييفها، وفهم ما يسهم به كل خطوة، وفحص النتيجة. واللغة مفيدة أيضًا خارج الفيزياء العددية: فمعالجة الملفات وخدمات الويب والأتمتة وقواعد البيانات وتصوير البيانات تستخدم الأدوات الأساسية نفسها.

يوجّه كثير من كود Python العددي الحسابات الثقيلة إلى مكتبات مترجَمة مثل NumPy وSciPy. وهذا يتيح لتعبير Python قصير أن ينفّذ عملًا عدديًا كبيرًا بكفاءة. ولذلك قد يكون زمن تشغيل حلقة مكتوبة بـPython العادي مختلفًا تمامًا عن زمن حسابٍ تتولّاه مكتبة عددية. وعندما نفحص الأداء، سننظر في أين يحدث العمل ونقيس الزمن الذي يستغرقه.

كما أن Python مقروءة بما يكفي بحيث يبدو تحليل قصير مثل تعليمات لزميل. والقائمة أدناه معاينة مُعطاة، وليست درسًا: فلم نعلّم بعد صياغة Python، وهذا الكود غير مُقيَّم. اقرأه لترى شكل تحليل يمكننا العودة إليه لاحقًا. هنا يزيل `dropna` الصفوف ذات المدخلات الناقصة في الأعمدة المسمّاة، ويربّع `2 **` عمودًا عنصرًا بعنصر، ويلائم `np.polyfit(..., 1)` كثير حدود من الدرجة الأولى ويعيد معاملاته، لذا `fit[0]` هو الميل:

```
clean = data.dropna(subset=["time_s", "position_m"])
clean["time_squared"] = clean["time_s"] ** 2
fit = np.polyfit(clean["time_squared"], clean["position_m"], 1)
gravity = 2 * fit[0]
```

تقوم الأسماء هنا بعمل علمي: يستطيع القارئ أن يرى أي الأعمدة استُخدمت، وأي تحويل أُجري، ولماذا يرتبط الميل المُلائَم بالجاذبية.

ومع توفّر هذا العدد الكبير من حزم Python، سنجد أحيانًا عدة أدوات للمهمة نفسها. سنختار الموثّقة جيدًا منها، ونفهم كيف تنطبق على نموذجنا، ونسجّل بيئة الحزم حتى نتمكّن من العودة إلى التحليل لاحقًا. ويصبح تكييف مثال من مكتبة أسهل بكثير عندما نستطيع شرح ما تفعله كل خطوة وفحص نتيجتها.

0.4.1 مثال كامل: الإشارة في موقعي LIGO

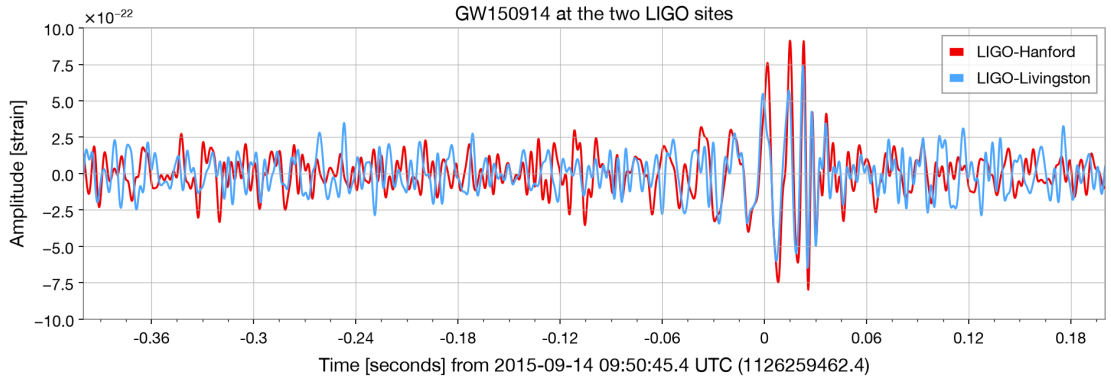
كان GW150914 أول كشف مباشر لموجات الجاذبية وأول رصد لاندماج ثقبين أسودين. ويمكننا مقارنة الانفعال العام من هانفورد وليفينغستون (أي مقدار تشوّه الفضاء نفسه عند موقعين في الولايات المتحدة) حول الحدث بالمثال التالي بلغة Python، المقتبس من مثال ترشيح GWpy لحدث GW150914. وهو يجلب 32 ثانية من كل موقع، ويطبّق مرشح النطاق 50–250 هرتز نفسه ومرشحات الثلم عند 60 و120 و180 هرتز، ويعرض 0.6 ثانية المركزية.

الكود أدناه معاينة «كما هو»: لم نعلّم بعد صياغة Python ولا مكتبة GWpy، ولا يُقيَّم أيٌّ منه هذا الفصل. وهو هنا لنأخذ فكرة عن شكل سير العمل في Python، ولنبيّن مدى اتساع منظوم Python البيئي. فبإمكان المرء أن يجد حزمًا لكل شيء تقريبًا.

استيعاب القوائم مثل `[filter_design.notch(f, hdata.sample_rate) for f in (60, 120, 180)]` يبني مدخل مرشح واحدًا لكل تردد؛ وتفكّك النجمة في `concatenate_zpks(bp, *notches)` تلك القائمة إلى أرجومنتات منفصلة؛ ويضرب `lfilt *= -1` السلسلة في مكانها بسالٍ واحد.

والمرشحات والإزاحة الزمنية وقلب الإشارة هي العمليات التي تجعل مقارنة الأثرين ممكنة.

```
from gwosc.datasets import event_gps
from gwpy.timeseries import TimeSeries
from gwpy.signal import filter_design
from gwpy.plot import Plot
t0 = event_gps("GW150914")
hdata = TimeSeries.fetch_open_data("H1", t0 - 16, t0 + 16)
ldata = TimeSeries.fetch_open_data("L1", t0 - 16, t0 + 16)
bp = filter_design.bandpass(50, 250, hdata.sample_rate)
notches = [filter_design.notch(f, hdata.sample_rate) for f in (60, 120,
180)]
zpk = filter_design.concatenate_zpks(bp, *notches)
hfilt = hdata.filter(zpk, filtfilt=True)
lfilt = ldata.filter(zpk, filtfilt=True)
lfilt.shift("6.9ms")
lfilt *= -1
plot = Plot(figsize=(12, 4))
ax = plot.gca()
ax.plot(hfilt, label="LIGO-Hanford", color="gwpy:ligo-hanford")
ax.plot(lfilt, label="LIGO-Livingston", color="gwpy:ligo-livingston")
ax.set(xlim=(t0 - 0.4, t0 + 0.2), ylim=(-1e-21, 1e-21),
       title="GW150914 at the two LIGO sites", ylabel="Amplitude [strain
]")
ax.set_xscale("seconds", epoch=t0)
ax.legend()
plot;
```



هذا هو التوليف الفعلي الذي تعيده هذه الأوامر. تحتوي كل سلسلة مُنزَّلة على 131,072 عيّنة بمعدل 4,096 هرتز خلال 32 ثانية؛ وتُبقي الفترة الأوسع آثار حواف المرشح بعيدة عن نافذة الحدث المعروضة. وتُزاح سلسلة ليفينغستون إلى الأمام بمقدار 6.9 ms، وهو فرق زمن الوصول بين الموقعين المقيس لهذا الحدث، وتُضرب في -1 لمراعاة الاتجاهين النسبيين للكاشفين. وتجعل هذه العمليات أشكال الموجة قابلة للمقارنة المباشرة؛ وهي مذكورة صراحةً لا ملائمَةً هنا. وتكتم الفاصلة المنقوطة الختامية تمثيلًا مكررًا في الدفتر؛ وفي سكربت مستقل يمكننا استدعاء `plot.show()` أو حفظ الرسم. ويوجد سكربت إعادة الإنتاج والنسخ المسجّلة في `graphics/generate_gwpy_example.py` و`graphics/gw150914-gwpy-output.json`.

GW150914: ما الذي يقيسه هذا الرسم؟

اندمج ثقبان أسودان وأصدرا موجات جاذبية. وتغيّر موجة عابرة الطول النسبي لذراعَي مقياس التداخل: فالانفعال $h = \Delta L/L$ كمية بلا أبعاد. وتحتوي مخرجات الكاشف على ضوضاء إلى جانب أي إشارة. ويكتم مرشح النطاق الترددات خارج 50–250 هرتز، وتكتم مرشحات الثلم توافقات ضيقة لشبكة الكهرباء، ويتجنّب الترشيح إلى الأمام والخلف تأخيرًا طوريًا مصطنعًا. والترشيح وحده لا يثبت وجود كشف. أما توافق كاشفين مستقلين بعد التأخير والتصحيح الاتجاهي الموثّقين فهو دليل أقوى بكثير من أثر واحد، لكن التحليل الحقيقي يقدّم أيضًا دلالة الضوضاء إحصائيًا ويقارن نماذج شكل الموجة.

رُصد GW150914 في 14 سبتمبر 2015 وأُعلن في 2016. ورصده كاشفا LIGO هانفورد وليفينغستون؛ ولم يكن Virgo يرصد حينها. ومُنحت جائزة نوبل في الفيزياء لعام 2017 إلى راينر فايس وباري باريش وكيب ثورن تقديرًا لمساهماتهم الحاسمة في كاشف LIGO ورصد موجات الجاذبية. وتقيّد المعايرة والضوضاء والترشيح وافتراضات شكل الموجة التفسير. وبيانات GWOSC العامة موزّعة برخصة CC BY 4.0؛ انظر https://gwosc.org/GW150914/.

0.5 لماذا نتعلم اللغتين معًا؟

بعد أن ننجز حسابًا فيزيائيًا في Julia، سيبدو كثير منه مألوفًا عندما نصادفه في Python. ويبقى النموذج ووحداته كما هما، حتى مع تغيّر تدوين تخزين البيانات أو استدعاء مكتبة. وتمنحنا حالة ذات جواب معروف فحصًا أوليًا في أي من اللغتين، وتساعدنا الرسوم والبواقي على استكشاف النتيجة. ويمنحنا تعلّم لغة ثانية فرصة للتعرّف على هذه الأفكار في سياق آخر.

نبدأ بـJulia لأن تدوينها العددي يتيح لنا متابعة بنية الحساب الفيزيائي عن قرب. ولاحقًا ستفتح Python المنظوم البيئي الأكبر الذي نصادفه في المختبرات ومجموعات البحث والبرمجيات العلمية العامة. وبنهاية المقرر ينبغي أن نكون مرتاحين في قراءة الكود باللغتين، واختيار أداة لمهمة معيّنة، وشرح الفيزياء وراء الحساب.

0.6 كيف يُنظَّم العام الدراسي؟

الفصل الأول هو فصل Julia في المقام الأول. نبدأ فورًا بسير عمل مختبري مُعطى لفحص القياسات وملاءمة خط مستقيم ورسم البواقي وتفسير المعاملات بوحداتها؛ ثم يأتي العمل المستقل بالملفات والجداول في الأسبوع 7. وبعد ذلك نبني أسس البرمجة والطرق العددية اللازمة لإجراء مثل هذه التحليلات باستقلالية. وفي الأسبوع 12 يجمع تحقيق تيكو براهي–كبلر هذه المهارات في تحليل واحد قابل لإعادة الإنتاج.

يقدّم الفصل الثاني Python بنقل طرق فهمناها أصلًا، ثم يوسّعها إلى مسائل القيم الذاتية، ومحاكاة مونت كارلو، والملاءمة غير الخطية والملاءمة الواعية بعدم اليقين، وتحليل فوريه، والمعادلات التفاضلية الجزئية، والأداء، وتعلّم الآلة، وسير العمل الرمزية. وتوفّر واجبات الأسبوع تدريبًا لازمًا وفحوص جاهزية، ويعطي التقييم المحوسب وزنًا أكبر لحلّ مسألة غير مألوفة باستقلالية وشرح المعنى الفيزيائي للنتيجة.

0.7 ما الذي سنتعلمه؟

لتحويل مسألة فيزيائية إلى برنامج، نحتاج أولًا إلى تحديد كيف سيسير الحساب. بأي قيم نبدأ؟ وما الذي يتغيّر في كل خطوة، ومتى ينبغي أن نتوقّف؟ سنتعلّم التعبير عن هذه الخيارات من خلال التفكير الخوارزمي، باختيار تمثيل للبيانات وتقسيم الحساب إلى خطوات نستطيع متابعتها واختبارها. وستساعدنا الحالات البسيطة والتوقعات الفيزيائية على فهم المخرجات.

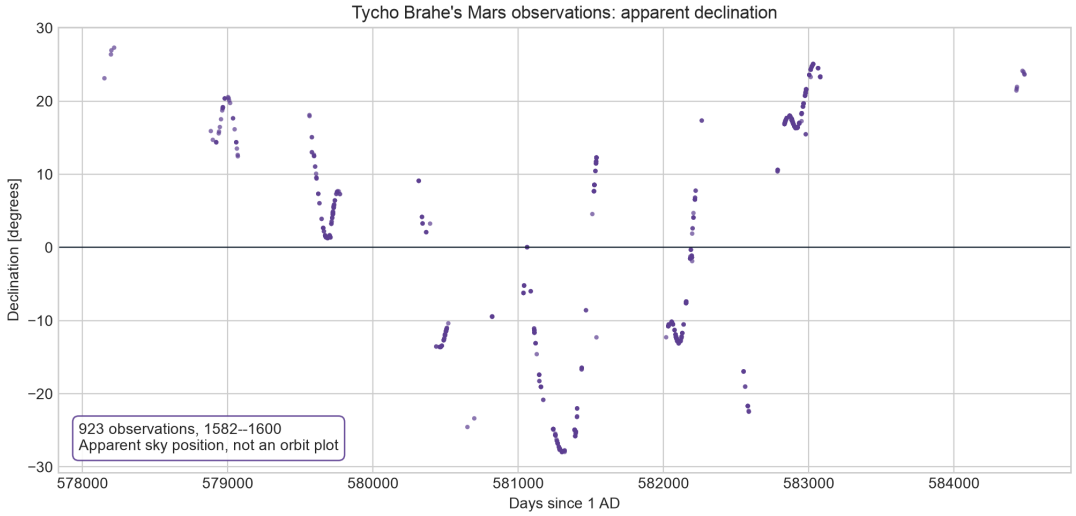
ثم سنستخدم تلك العادات من أجل:

- قراءة بيانات تجريبية ورصدية من الملفات؛
- تصوير البيانات والنماذج والبواقي وعدم اليقين؛
- ملاءمة نماذج خطية وغير خطية وتفسير معاملاتها؛
- حلّ المعادلات وتكامل الدوال وحلّ المعادلات التفاضلية العادية عدديًا؛
- العمل مع المتجهات والمصفوفات وتحويلات فوريه والعمليات العشوائية والمعادلات التفاضلية الجزئية البسيطة؛
- كتابة كود علمي مقروء وقابل للاختبار وقابل لإعادة الإنتاج بـJulia وPython.

0.8 تشويق: إعادة تحليل كبلر للمريخ

في نهاية الفصل الأول سنستخدم أرصادًا مرتبطة ببرنامج تيكو براهي لرصد المريخ لإعادة بناء نسخة من حجة كبلر. وستصبح مجموعة البيانات هذه دليلًا لاختبار قانون كبلر الأول: مدارات الكواكب إهليلجات والشمس في إحدى بؤرتيها. سنأخذ أرصادًا جُمعت في تواريخ مختلفة، ونحوّل المواضع الزاوية المجدولة إلى وصف هندسي مشترك، ونلائم نماذج مدارية متنافسة، ونبيّن كميًا لماذا ينجح الإهليلج حيث تفشل الدائرة.

يعرض الرسم أدناه 923 قياسًا للانحراف الظاهري للمريخ، أي زاويته شمالًا–جنوبًا في السماء، خلال الأعوام 1582–1600. ويمكننا بالفعل رؤية نمط في كيفية تغيّر تلك الزاوية مع الزمن. ولربطه بشكل المدار، سنحتاج إلى مراعاة هندسة الرؤية ومقارنة تنبؤات النموذجين الدائري والإهليلجي. ويترك رسم الانحراف وحده ذلك السؤال مفتوحًا.



سجّل تيكو التواريخ والمواضع المقيسة في جداول. وتحويل تلك المدخلات إلى رسم يمنحنا نظرة أولى على الأرصاد، وسيساعدنا بناء نموذج مداري على فهم ما ينتج النمط. وعلى الطريق سنستعين بأفكار من المقررات التي تدرسونها إلى جانب هذا المقرر:

- **الميكانيكا:** القوى المركزية، والحركة المدارية، والتفسير الفيزيائي لإهليلج والشمس في إحدى بؤرتيه؛
- **الرياضيات:** المتجهات، وتحويلات الإحداثيات، والقطوع المخروطية، والمصفوفات، وملاءمة المربعات الصغرى؛
- **المختبر:** عدم اليقين، والبواقي، ومصدر البيانات، والتمييز بين منحنى يبدو جيدًا واستنتاج مدعوم بالأدلة.

بحلول الوقت الذي نصل فيه إلى هذا التحقيق، ستكون لدينا الأدوات لكتابة التحليل بأنفسنا واستخدام أرصاد تاريخية لفحص الادعاء التجريبي المركزي لقانون كبلر الأول. وفي الوقت الحالي، يمنحنا الرسم سؤالًا نحمل معنا خلال الفصل. وسنطوّر الهندسة وطرق الملاءمة عندما نكون جاهزين لاستخدامها.

0.8.1 الحصول على مستودع المقرر

المستودع الطلابي هو nshaviv/physprog. نفتح طرفية في دليل أبٍ نختاره ونشغّل:

```
git clone https://github.com/nshaviv/physprog.git
cd physprog
```

وإذا لم يكن Git مثبّتًا، نفتح صفحة المستودع ونستخدم Code → Download ZIP، ثم نفكّ الأرشيف ونفتح المجلد؛ وبقية الخطوات مماثلة.

ينشئ Git مجلد المقرر المحلي. نشغّل أوامر البيئة من جذر هذا المستودع الجديد ونفتحه في VS Code. ويسجّل ملف README الطلابي المحاضرات والواجبات والحلول التي صدرت. وتوجد تعليمات التثبيت والتحديث المفصّلة في STUDENT_SETUP.md في جذر المستودع. ويمكن قراءة الدفاتر من مخرجاتها المحفوظة؛ وفتحها للتشغيل في VS Code مع إضافة Jupyter أو في JupyterLab اختياري. ولا يلزم أن يطابق موضع عملنا الشخصي حاسوب المدرّس.

0.9 بيئة المقرر: نُثبّت مرة ونشغّل بتكرار قابل للإنتاج

نثبّت Julia 1.12 من https://julialang.org/install/؛ ونثبّت Visual Studio Code وملحقه الرسمي لـJulia من https://www.julia-vscode.org/docs/stable/gettingstarted/. وفي طرفية جديدة نتأكد أن الأمر الموجود في مسارنا هو Julia المقصود:

```
julia --version
```

```
julia version 1.12.6
```

نتأكد أن المخرجات تبدأ بـjulia version 1.12. وإذا ظهر إصدار آخر، نعيد تشغيل الطرفية بعد التثبيت ونفحص أي تثبيت لـJulia تجده.

نفتح جذر المستودع في VS Code. وهو المجلد الذي يحتوي على Project.toml بعد النسخ. ونشغّل الأوامر أدناه من ذلك الجذر. ويُنشّط الخيار --project=. بيئة المقرر المسجّلة؛ ويخبر الخيار التالي -e لغة Julia بتقييم أرجومنتها.

```
julia --project=. -e 'using Pkg; Pkg.instantiate()'
```

تقرأ Julia ملف Project.toml والبيان المسجّل، وتنزّل أي حزم مسجّلة ناقصة، وتبلّغ عن تثبيت الحزم أو ترجمتها المسبقة. ونشغّل هذا مرة واحدة أثناء الاتصال بالشبكة. ولا نضع Pkg.add أو Pkg.update في عمل المقرر: فتلك الأوامر ستغيّر بيئة بدل استخدام البيئة المسجّلة.

ثم نشغّل أمر التحقق من المقرر مع اختيار المشروع نفسه:

```
julia --project=. verify_environment.jl
```

```
Course Julia environment verified.
```

يفحص السكربت البيئة المهيّأة وينتهي برسالة نجاحه. وقد ينشئ مخرجات رسم مؤقتة خارج المستودع. وإذا أبلغ عن حزمة ناقصة، نغلق جلسات Julia REPL القديمة، ونتأكد أن VS Code اختار بيئة المقرر، ونعيد أمر التثبيت.

وفي VS Code نختار هذه البيئة نفسها للملحق Julia. والملف الذي يُشغَّل من المحرّر يرسل القيم ونصوص println إلى Julia REPL المدمج. فالمحرّر هو حيث نكتب الكود، والـREPL هو جلسة Julia الجارية التي تقيّمه. ويصبح هذا التمييز مفيدًا في الفصل الأول.

اختياري: تشغيل الدفاتر في VS Code

يحتوي كل دفتر مسبقًا على مخرجاته المحفوظة، فنستطيع قراءته دون تشغيل أي شيء. ولتشغيل دفتر تفاعليًا:

1. في VS Code نثبّت إضافة Jupyter (وتبقى إضافة Julia لملفات jl.).
2. نفتح جذر المستودع، ثم نفتح ملف ipynb. ونستخدم Select Kernel لاختيار `Julia 1.12` لدفاتر المحاضرة والتمرين، أو بيئة Python الخاصة بالمقرر لدفاتر Python في الفصل 0.
3. نشغّل الخلايا خليةً خلية، أو نستخدم Restart Kernel and Run All لإعادة إنتاج الفصل من بداية نظيفة.

وتُفتح الدفاتر نفسها أيضًا في JupyterLab. وتشغيل الدفاتر اختيار وليس متطلبًا.



1. حسابات Julia الأولى وملاءمة أوهلية موجَّهة

نبدأ الحساب العلمي بالتوقّع لما ينبغي أن يحدث، وجعل الحساب مرئيًا، والسؤال عمّا إذا كانت للنتيجة الوحدات والمعنى الصحيحان. اليوم نستخدم Julia كآلة حاسبة، ثم نتبع وصفة مُعطاة تمضي من البيانات إلى الملاءمة إلى الرسم. وتتضمن الوصفة متجهات وأوامر رسم سنتعلّمها باستقلالية لاحقًا.

1.1 أين نرى النتيجة؟

في REPL الخاص بـJulia، تؤدي كتابة تعبير إلى تقييمه وعرض قيمته. وفي ملف VS Code يمكن للتقييم أيضًا أن يعرض قيمة في نافذة النتيجة. ذلك العرض مريح، لكنه ليس جزءًا دائمًا من تقرير السكربت. أما `println` فيرسل نصًا صراحةً إلى الـREPL أو الطرفية أو مخرجات السكربت، ونستخدمه عندما يجب أن يرى القارئ نتيجة.

```
1 + 1
println(1 + 1)
```

```
2
```

تقييم السطر الأول في الـREPL أو في عرض المحرّر يُظهر 2. والسطر الثاني يطبع 2 إلى مخرجات الـREPL. وقد يقيّم السكربت تعبيرات كثيرة دون عرضها، لذا نستخدم `println` لتقرير نصي مقصود.

1.2 Julia كآلة حاسبة

في Julia حساب وتجميع وأُسّ ودوال رياضية مألوفة. ومعظم التعبيرات أدناه استدعاءات دوال: اسم دالة يتبعه قوسان، مثل `exp(1)`. ويطلب الاستدعاء من الدالة المسمّاة أن تعمل على القيم بين القوسين وتعيد نتيجة. وأرجومنتات `exp` و`sin` بالراديان عندما تمثّل زوايا. و`mod(a,b)` باقي القسمة على `b`؛ و`div(a,b)` قسمة خارج صحيحة تهمل الباقي بالنسبة للأعداد الصحيحة الموجبة.

```
println(exp(1))
println(sin(pi / 2))
println(mod(17, 5))
println(div(17, 5))
```

```
2.718281828459045
1.0
2
3
```

هنا `2 + 5*3 == 17`؛ فالخارج 3 والباقي 2.

يمكننا إدخال رموز رياضية يونيكود في الـREPL وفي VS Code. اكتب `\pi<Tab>` للحصول على π. وفي `2pi` أو 2π تفسّر Julia المعامل العددي كضرب في `pi`، وهو ليس اسم متغير واحد. وبين اسمَي متغيرين علينا كتابة إشارة الضرب صراحةً، كما في `mass * speed`.

```
println(2pi)
println((2 + 3) * 4)
println(2^3)
```

```
6.283185307179586
20
8
```

1.3 أسماء للقيم قبل الفيزياء

إسناد مثل `x = 2` يربط الاسم على اليسار بالقيمة المحسوبة على اليمين. وهو تعليمة، لا العبارة الجبرية $x = 2$ لكل الأزمنة. ويمكننا بعدها استخدام الأسماء لإظهار حساب.

```
x = 2
y = 3
println(x + y)
println(x^2 + y^2)
```

```
5
13
```

نبقي الأسماء خالية من اللواحق الوحدية ونسجّل كل وحدة في تعليق قصير في نهاية السطر حيث تُعرَّف القيمة، مثل `voltage = 2.0 # V`. ولا تستنتج Julia الوحدات من التعليق ولا تفرضها، فيحمل التعليق وتوثيق الدوال الوحدات.

1.4 سير عمل موجَّه من القياس إلى النموذج

تستعيد الوصفة التالية الشرح الكامل لتحليلنا الأول. ونوفّر خطواتها لقراءة الملف والمتجهات والرسم. ويُتوقّع منا تشغيل هذه الخطوات وتفسيرها الآن؛ أما العمل المستقل على DataFrames/CSV فيعود إلى الأسبوع السابع، وإتقان الملاءمة إلى الأسبوع الثامن. والاشتقاق الرياضي خلفية اختيارية منفصلة عن المكافأة المُقيَّمة بثلث نقطة. ولا نعرّف دوالًا قبل الأسبوع الخامس.

نفتح دفتر الأسبوع الأول المُعطى أو نشغّل البرنامج المحفوظ من الدليل الذي يحتوي `ohm_law.csv`. وبيئة المقرر المشتركة تعلن مسبقًا CSV وDataFrames وPlots، والتثبيت موصوف في الفصل 0.

ما هو ملف CSV؟

يخزّن ملف القيم المفصولة بفواصل (CSV) جدولًا كنص عادي. ويعطي السطر الأول عادةً أسماء الأعمدة، وكل سطر لاحق صف واحد، وتفصل الفواصل بين العناصر. فمثلًا،

```
current_A,voltage_V
0.10,0.52
0.20,1.01
```

جدول من عمودين: التيار بالأمبير والجهد بالفولت. وتستطيع برامج الجدولة والمختبر عادةً تصدير جدول بصيغة CSV. والنص العادي مفيد لأن الإنسان يستطيع فحص الملف، وتستطيع برامج كثيرة قراءته. وسنستخدم حزمة `CSV.jl` لقراءة الملف وحزمة `DataFrames.jl` لتنظيم الجدول.

الموصل الأومي

بالنسبة لموصل أومي في حالة ثابتة تقريبًا وفي نظام المجال الضعيف/الاستجابة الخطية، $V = RI$. نأخذ الجهد المطبَّق V بالفولت والتيار الموجب I بالأمبير في الاتجاه الموجب المختار، ووحدات $R = V/I$ هي Ω، ووحدات الموصلية $G = 1/R$ هي S. ويرسي التشتت الانسياق المستقر ويساعد في تحديد المقاومة، لكن التصادمات السريعة وحدها لا تضمن الخطية. ويمكن للتسخين والملامسات والمجالات القوية والأجهزة غير الأومية أن تبطل النموذج. وفحصه المثالي هو $I = 0 \Rightarrow V = 0$.

خلفية رياضية: نموذج الخط المستقيم

للنموذج $y = ax + b$ ميل a وحد ثابت b. ويخبرنا الميل بمقدار تغيّر y عند ازدياد x بوحدة واحدة؛ والحد الثابت هو قيمة النموذج عند $x = 0$.

كيف نقدّم فكرة برمجية جديدة

قبل استخدام أي بناء جديد، نسأل لماذا نحتاجه، ونعرّفه بلغة واضحة، ونعرض صيغته العامة في Julia، ونجرّب أصغر مثال مفيد. ثم نستخدمه في حساب أغنى، ونربطه أخيرًا بمهمة علمية. وعند قراءة الكود نتبع الترتيب نفسه: الغاية، فالصيغة، فالحالة البسيطة، فالتطبيق.

طرق تشغيل Julia في الأسبوع الأول

- **طرفية النظام:** نفتح Terminal أو PowerShell أو صدفة Linux ونكتب `julia`. يبدأ هذا Julia REPL حيث يُقيَّم `1+1` فورًا.
- **طرفية VS Code:** نفتح **Terminal > New Terminal** داخل VS Code ونكتب `julia`. وهو الـREPL نفسه، لكن بجانب ملفات المقرر.
- **REPL جوليا في VS Code:** نستخدم لوحة الأوامر ونختار *Julia: Start REPL*. ومع فتح ملف `.jl`، يرسل `Shift+Enter` السطر الحالي أو الشيفرة المحدّدة إلى ذلك الـREPL ويبدأه عند الحاجة.
- **برنامج كامل محفوظ:** نستخدم **Run > Run Without Debugging** لتنفيذ ملف `.jl` النشط من أعلى إلى أسفل. وهذا هو الخيار المعتاد للعمل المُسلَّم لأن التحليل محفوظ وقابل للتكرار.

ولاحقًا سنقدّم الدفاتر، التي تكون خلاياها أكثر تفاعلًا. ولا نخلط بين خلية دفتر وتحليل محفوظ: فكلاهما يحتاج إلى مدخلات واضحة وكود ورسوم واستنتاجات.

1.4.1 نبني التحليل خطوة مرئية في كل مرة

الخطوة 1: تحميل الأدوات. يذكر السطر الأول من السكربت الحزم المثبَّتة التي سيستخدمها البرنامج.

```
using CSV, DataFrames, Plots
```

تجعل الكلمة المفتاحية `using` الأدوات العامة من هذه الحزم متاحة في هذا الملف. فCSV يقرأ الملف النصي، وDataFrames يوفّر بنية جدول، وPlots ينشئ رسومًا. وهذا ليس حسابًا بعد: إنه إعلان البرنامج عن الأدوات التي يحتاجها.

الخطوة 2: تحميل الجدول وفحصه. تفتح السطور التالية الملف وتعرض صفوفه الخمسة الأولى.

نحتاج إلى نمطين برمجيين أساسيين قبل ذلك. استدعاء الدالة، الذي رأيناه مع `exp` و`sin`، يطلب من أداة مسمّاة إجراء عملية. وصيغته العامة `function_name(input1, input2)`، فمثلًا `println("ready")` يستدعي `println` بمدخل نصي واحد. والإسناد يمنح نتيجة اسمًا قابلةً لإعادة الاستخدام. وصيغته العامة `name = expression`، وتقيّم Julia الطرف الأيمن أولًا ثم تخزّن النتيجة تحت الاسم على اليسار. وأصغر مثال هو `x = 2`. وتجمع الكتلة التالية هذه الأنماط مع ملف بيانات حقيقي.

المدخل الثاني لـ`CSV.read`، أي `DataFrame`، يسمّي نوع الجدول الذي نريد استعادته: جدولًا بأعمدة مسمّاة. وتعيد الأداة المساعدة `first(data, 5)` الصفوف الخمسة الأولى من ذلك الجدول لنفحصها.

```
root = get(ENV, "PHYSPROG_HOME", pwd())    # the course folder
candidates = [joinpath(root, "assets", "data", "ohm_law.csv"),
              joinpath(root, "reveal", "week01", "ohm_law.csv")]
datafile = candidates[findfirst(isfile, candidates)]
data = CSV.read(datafile, DataFrame)
println(first(data, 5))
```

مع الملف المُعطى تكون صفوف البيانات الخمسة المطبوعة:

صف	current_A	voltage_V
1	0.1	0.52
2	0.2	1.01
3	0.3	1.52
4	0.4	2.0
5	0.5	2.49

نقرأ `CSV.read(datafile, DataFrame)` من اليسار إلى اليمين. فـ`CSV.read` تعني «استخدم الدالة read التي توفّرها حزمة CSV». ومدخلها الأول اسم الملف، ومدخلها الثاني يطلب `DataFrame`، أي جدولًا بأعمدة مسمّاة. وتخزّن علامة المساواة الجدول الناتج تحت الاسم data. ويأخذ السطر التالي الصفوف الخمسة الأولى ويطبعها في الـREPL أو الطرفية. وهذا فحص عقلي أول: هل أسماء الأعمدة صحيحة، وهل الأعداد معقولة، وهل فتحنا الملف المقصود؟

الخطوة 3: تسمية الكميات الفيزيائية. نستخرج العمودين إلى أسماء متغيرات خالية من الوحدات تسجّل تعريفاتها الوحدات في تعليق قصير.

يمنح `DataFrame` كل عمود اسمًا. وتختار صيغة الوصول `table.column_name` عمودًا كاملًا، فمثلًا `data.current_A` تختار القياسات المخزّنة تحت العنوان `current_A`. ونسند العمودين المختارين إلى اسمين ذوي معنًى:

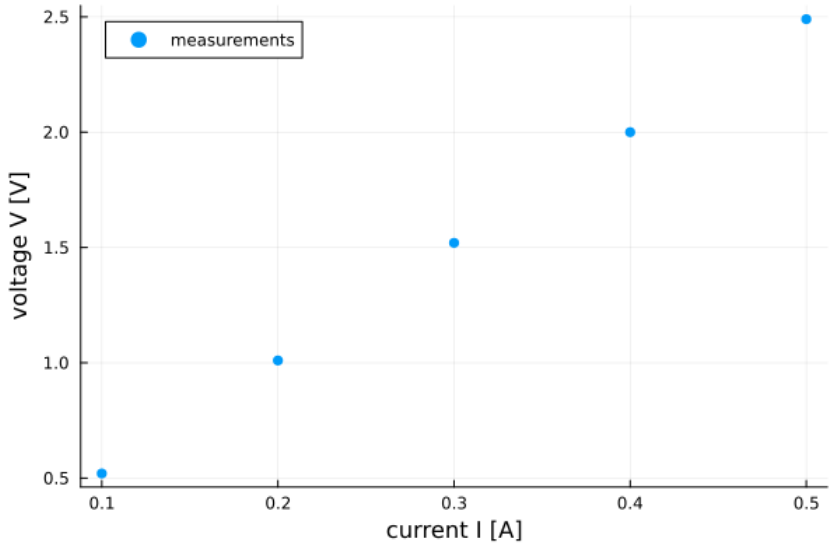
```
current = data.current_A   # A
voltage = data.voltage_V   # V
```

الاسمان current وvoltage متغيران: تسميتان تتيحان للتعليمات اللاحقة أن تشير إلى قيم يخزّنها البرنامج. ويسند علامة المساواة القيمة على يمينه إلى المتغير على يساره؛ وهو لا يعبّر عن هوية رياضية دائمة. والنقطة في `data.current_A` تختار العمود المسمّى `current_A`. ونبقي المعرّفات خالية من الوحدات ونقرأ الوحدة من التعليق في نهاية السطر، وهو تعليق تتجاهله Julia. ونوثّق الوحدات حيث تُعرَّف القيم، وفي توثيق الدوال أو توثيق الواجهة، وعند مواقع الاستدعاء التي قد تكون الوحدات فيها ملتبسة. ونستخدم هنا أعدادًا صحيحة للعدّ وقيم قياس عشرية دون تطوير نظام الأنواع في Julia؛ وسيسمّي الأسبوع الثاني هذه الأنواع البسيطة ويفحصها صراحةً.

الخطوة 4: جعل البيانات مرئية. قبل ملاءمة أي شيء، نرسم القياسات.

المتجه قائمة مرتّبة تُكتب داخل قوسين مربعين، مثل `[0, 1]`. ومخطط التشتت يمثّل كل زوج من القيم من متجهين بنقطة واحدة، فيتيح لنا رؤية الاتجاهات والقياسات غير المعتادة قبل الوثوق بنموذج. والصيغة الأساسية للاستدعاء `scatter(x_values, y_values)`. فمثلًا، `scatter([0, 1], [0, 1])` يرسم نقطتين. وللإعدادات المسمّاة الاختيارية الصيغة `keyword=value`؛ وهي تضيف تسميات أو تغيّر المظهر دون تغيير البيانات المقيسة. وعلى سبيل المثال، `markerstrokewidth=0` يزيل الإطار المرسوم حول كل علامة فتبقى النقاط المتداخلة مقروءة. ونطبّق الآن هذه الصيغة على التيار والجهد:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
```



شكل 1.1: المخرج الرسومي لاستدعاء scatter: الجهد المقيس يرتفع خطيًا تقريبًا مع التيار.

المدخلان الأولان لـscatter هما الإحداثيان الأفقي والرأسي. والمدخلات اللاحقة أزوجمنتات مسمّاة: لكلٍّ منها اسم، مثل `xlabel`، يتبعه علامة مساواة وقيمة. وهي تتحكم في مظهر الرسم. والتسميات ومفتاح الرسم ليست زينة؛ إنها تخبر القارئ بما قيس وبأي وحدات.

الرسم جزء من الحساب

لا تثق بملاءمة لم نرسمها. فقد يخفي الجواب العددي قيمة شاذة، أو اتجاهًا غير خطي، أو خطأ في الوحدات، أو عمودين

مبدَّلين.

الخطوة 5: ملاءمة خط من تعريفه. للخط التجريبي $V = RI + V_0$، الباقي للقياس i هو الفرق الرأسي

$$r_i = V_i - (RI_i + V_0).$$

ونختار الخط الذي يصغّر مجموع مربعات الفروق،

$$S(R, V_0) = \sum_{i=1}^{n} [V_i - (RI_i + V_0)]^2 .$$

فالتربيع يمنع إلغاء الفروق الموجبة والسلبية بعضها بعضًا، ويعاقب الفرق الكبير أكثر من الصغير. وهذا هو تعريف المربعات الصغرى الخطية غير الموزونة: فهو يعامل كل القياسات على أنها متساوية في الموثوقية. وسندرس التصغير لاحقًا في المقرر. وفي الأسبوع الأول نستخدم الصيغ المباشرة الناتجة بدل مُصغِّر عام أو طريقة مصفوفات.

ولأصغر مثال دقيق، تقع النقاط الثلاث $(0,1)$ و$(1,3)$ و$(2,5)$ كلها على $y = 2x + 1$. فيل أفضل ملاءمة 2، وحدّها الثابت 1، وكل البواقي صفر. والقياسات الحقيقية لا تقع بالضبط على خط واحد، لذا يكون حساب قانون أوم أدناه هو الحالة الأغنى.

صيغ الملاءمة الخطية المباشرة

نكتب $\overline{I}$ لمتوسط التيار و$\overline{V}$ لمتوسط الجهد. والميل والحد الثابت لأفضل ملاءمة هما

$$R = \frac{\sum_{i=1}^{n}(I_i - \overline{I})(V_i - \overline{V})}{\sum_{i=1}^{n}(I_i - \overline{I})^2}, \qquad V_0 = \overline{V} - R\overline{I}.$$

والبسط موجب عندما ينزع التيار والجهد إلى الزيادة معًا. والمقام يقيس مقدار تشتّت قيم التيار حول متوسطها. ولا تكون الملاءمة ممكنة إذا تساوت كل قيم التيار، لأنه لا يمكن عندئذ تحديد ميل. ويُختار الحد الثابت بحيث يمرّ الخط الملائم عبر $(\overline{I}, \overline{V})$.

خلفية رياضية اختيارية: من أين تأتي صيغ الملاءمة؟

هذا الصندوق لمن يريد أن يستشرف ما سيأتي. وهو غير مطلوب في الأسبوع الأول. نبحث عن قيمة صغرى لـ$S(R, V_0)$، وهي تعتمد على معاملين قابلين للضبط. وعند قيمة صغرى ناعمة، لا يمكن لتغيير أيٍّ من المعاملين بمقدار صغير أن يخفض S، لذا يجب أن يختفي كلا المشتقّين الجزئيين.

نشتق أولًا بالنسبة للحد الثابت:

$$0 = \frac{\partial S}{\partial V_0} = -2 \sum_{i=1}^{n} [V_i - (RI_i + V_0)] .$$

وبالقسمة على $-2n$ وإعادة الترتيب نحصل على $V_0 = \overline{V} - R\overline{I}$. لذا يمرّ الخط الملائم عبر نقطة البيانات المتوسطة. ثم نشتق بالنسبة للميل R:

$$0 = \frac{\partial S}{\partial R} = -2 \sum_{i=1}^{n} I_i \, [V_i - (RI_i + V_0)] .$$

نعوّض عن عبارة V_0. فيصبح الباقي $(V_i-\overline{V})-R(I_i-\overline{I})$. وباستخدام $\sum_i(I_i-\overline{I})=0$ و$\sum_i(V_i-\overline{V})=0$، تصبح المعادلة المتبقية

$$\sum_{i=1}^{n}(I_i-\overline{I})(V_i-\overline{V})=R\sum_{i=1}^{n}(I_i-\overline{I})^2.$$

وبحلّ هذه المعادلة لإيجاد R نحصل على صيغة الميل المباشرة أعلاه، وبتعويض ذلك الميل في $V_0=\overline{V}-R\overline{I}$ نحصل على الحد الثابت. وفي الأسبوع الثامن، سيصبح إتقان المربعات الصغرى والتصغير لاحقًا أداةً منتظمة في المقرر.

يجب أن تطبّق الصيغ العملية نفسها على كل متجه قياسات. والنقطة على عامل حسابي تخبر Julia أن تطبّق تلك العملية عنصرًا بعنصر. والنمط العام هو `value .operator vector`. ويبدأ الرمز `#` تعليقًا: تتجاهل Julia بقية ذلك السطر، بينما يستطيع القارئ استخدامه شرحًا. فمثلًا،

```
values = [1, 2, 3]
squared = values .^ 2     # [1, 4, 9]
```

نطبّق الآن العمليات المنقوطة على كل قياسات التيار والجهد. ويأتي أولًا مُجمِّلان للمتجه: `length(current)` يعيد عدد القياسات، و`sum(current)` يجمع عناصر المتجه. ونستخدمهما معًا لحساب المتوسطين. وينفّذ الكود الصيغ مباشرة، فتبقى كل كمية مرئية:

```
n = length(current)
I_bar = sum(current) / n   # A
V_bar = sum(voltage) / n   # V
dI = current .- I_bar      # A
dV = voltage .- V_bar      # V
resistance = sum(dI .* dV) / sum(dI .^ 2)   # ohm
offset = V_bar - resistance * I_bar   # V
```

يعدّ `length(current)` القياسات ويجمع `sum(...)` عناصر متجه. والنقطة في `-.` و`*.` و`^.` تعني «طبّق هذه العملية على العناصر المتقابلة». وبهذا يحتوي `dI .* dV` على حاصل ضرب $(I_i-\overline{I})(V_i-\overline{V})$ لكل قياس، جاهزًا ليجمعها `sum`. وتفرّق Julia عمدًا بين العمليات العنصرية وعمليات المصفوفات؛ فضرب المصفوفات العادي غير معرَّف لمتجهين من قياسات متقابلة. وفي الأسبوع الأول نستخدم الصيغ المنقوطة كلما كان الحساب مقصودًا أن يعمل على كل قياس منفصلًا.

الخطوة 6: وضع النموذج على الرسم نفسه والإبلاغ عن النتيجة.

نبني رسم النموذج من نسخة جديدة من النقاط المقيسة، ثم نضيف الخط الملائم إلى الرسم نفسه. وفي الصيغة `plot!(x, y)` تدل علامة التعجب على أن رسمًا قائمًا يُعدَّل، ثم يحفظ `savefig("name.png")` الرسم الحالي. واستدعاء `plot()` بلا بيانات يبدأ رسمًا جديدًا فارغًا. وللإبلاغ بالأعداد، تُدرج `$(expression)` قيمة محسوبة في سلسلة نصية، ويحدّ `round(value, digits=3)` من خاناتها العشرية المعروضة. وفيما يلي مثال صغير لكل صيغة، على رسمه الخاص:

```
plot()
plot!([0, 1], [1, 0])
savefig("example.png")
rounded = round(1.2345, digits=2)
println("value = $(rounded)")
```

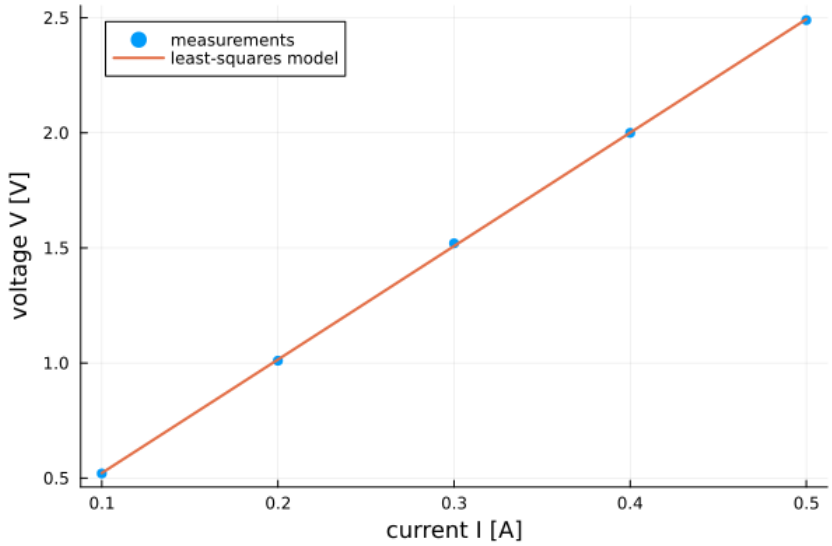
```
value = 1.23
```

وتعيد كتلة الإبلاغ العلمي الكاملة إنشاء مخطط القياسات وتضيف النموذج الملائم، فلا يبقى الخط اللعبة في الرسم. وعلى خط النموذج، يضبط `linewidth=2` عرض الخط:

```
scatter(current, voltage,
    xlabel="current I [A]", ylabel="voltage V [V]",
    label="measurements", markerstrokewidth=0)
plot!(current, resistance .* current .+ offset,
    label="least-squares model", linewidth=2)
savefig("ohm_law_fit.png")

println("R = $(round(resistance, digits=3)) ohm")
println("offset = $(round(offset, digits=3)) V")
```

```
R = 4.93 ohm
offset = 0.029 V
```



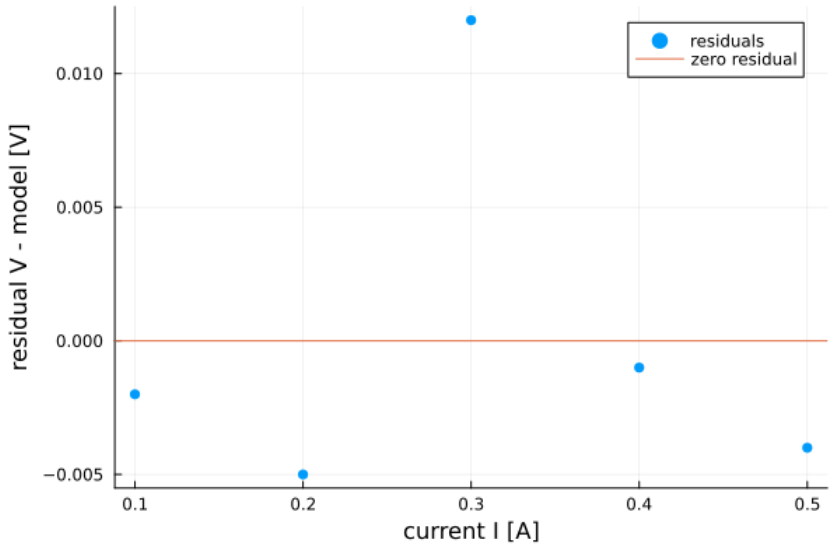
شكل 1.2: المخرج الرسومي بعد !plot: نموذج المربعات الصغرى مرسوم على القياسات.

تضرب النقطة في `resistance .* current` المقاومة العددية في كل قيمة تيار. ثم تضيف النقطة في `.+ offset` الحد الثابت إلى كل نتيجة. وتعبّر علامة التعجب في `plot!` عن «أضف إلى الرسم القائم» بدل إنشاء رسم جديد. ويحفظ استدعاء `savefig` التالي الرسم الحالي في ملف PNG داخل مجلد العمل.

الخطوة 7: فحص البواقي. الخط الملائم نموذج، لا برهانًا على جودة النموذج. نرسم البواقي لنتحقق هل تتقلّب حول الصفر دون نمط ظاهر. والصيغة المعدِّلة `!hline` تضيف خطًا أفقيًا إلى الرسم الحالي، ونستخدمها للمرجع الصفري:

```
residual = voltage .- (resistance .* current .+ offset)   # V
scatter(current, residual,
    xlabel="current I [A]", ylabel="residual V - model [V]",
    label="residuals", markerstrokewidth=0)
hline!([0], label="zero residual")
savefig("ohm_law_residuals.png")
```

يحفظ استدعاء `savefig` الثاني رسم البواقي الذي يكون حاليًا عند تلك النقطة. وإذا انحنت البواقي بانتظام فوق الصفر وتحته، فقد يكون الخط المستقيم هو النموذج الخطأ. وإذا كان باقٍ واحد أكبر بكثير من الآخرين، نفحص ذلك القياس بدل الوثوق الصامت بالملاءمة. وعند هذه النقطة لدينا برنامج علمي صغير كامل: يقرأ بيانات، ويفحصها، ويحفظ رسومًا، ويحسب معاملات نموذج، ويوصل معناها بالوحدات.



شكل 1.3: المخرج الرسومي لحساب البواقي: كل البواقي صغيرة وتقع على جانبي الصفر.

تشتّت القياس وخطأ النموذج والأرقام المعروضة

تشتّت القياس تباين في المشاهدات المتكررة ينتج عن دقة الجهاز وظروف التجربة. وخطأ النموذج عدم توافق منتظم بين النموذج المختار والنظام الفيزيائي؛ ونمط بواقٍ منحنٍ دليل على عدم توافق كهذا. وهذان يختلفان عن تدوير العرض. فطباعة ثلاث خانات عشرية تجعل المخرجات متسقة، لكنها لا تثبت أن الخانات الثلاث كلها ذات دلالة فيزيائية. ويجب أن يتبع عدد الأرقام المعنوية المبرَّر في النهاية من عدم يقين القياس، وهو ما سنعالجه بتفصيل أكبر لاحقًا.

الدليل الموجَّه لسير العمل هذا في فصل التمرين الأول، بيانات السقوط الحر.

خلفية فيزيائية: الوحدات في المعاملات الملائمة

إذا كانت لـy وحدات الفولت ولـx وحدات الأمبير، فلوحدات الميل فولت لكل أمبير، أي أوم. وللحد الثابت الوحدات نفسها التي لـy. وتحليل الوحدات طريقة بسيطة لالتقاط أخطاء الملاءمة. ولاحقًا يمكن لحزمة مثل `Unitful.jl` أن تحمل الوحدات مع القيم في الكود، ويبقى الاستدلال الفيزيائي نفسه.

1.5 التمرين: قياس ثابت النابض

ننقل التحليل نفسه إلى نابض. قياساتنا هي الاستطالة x بالمتر والقوة المؤثرة F بالنيوتن؛ والنموذج الملائم هو $F = kx + F_0$. وخلافًا لملاءمة تكون فيها الاستطالة على المحور الرأسي، يكون الميل هنا هو ثابت النابض k بوحدة N/m مباشرة. ونتوقّع أولًا ميلًا موجبًا ونتحقق من أن الحد الثابت له وحدة القوة.

قبل الجلسة نفتح دفتر تمرين الأسبوع الأول ونحدّد خلاياه المُعطاة وخلايا الملاءمة. وفي أول عشرين دقيقة نفحص المحاور، ونتابع ملاءمة واحدة، ونشرح الميل والحد الثابت وباقيًا واحدًا. ثم نبدأ واجب السقوط الحر المستقل بمساعدة طاقم التدريس. ومن الفحوص المفيدة أن نشرح لماذا يغيّر تبديل المحورين وحدات الميل الملائم ومعناه؛ وفي البيانات المشوّشة، لا تكافئ إعادة الملاءمة بمحورين مبدَّلين عمومًا مقلوب الميل.

1.6 الواجب الأساسي: استنتاج الجاذبية من قياسات السقوط الحر

نستقصي هل تتفق مسافات السقوط المقيسة مع الإطلاق من السكون تحت جاذبية منتظمة تقريبًا. نأخذ المسافة موجبة نحو الأسفل، فيكون $y(t) = \frac{1}{2}gt^2 + b$، مع $g > 0$ بوحدة $\mathrm{m/s}^2$ والحد الثابت b بالمتر. والتحويل المُعطى `t_s .^ 2` يجعل الزمن المُربّع هو الإحداثي الأفقي. نلائم $y = at^2 + b$ ونفسّر $g = 2a$؛ وعامل الاثنين يأتي من معادلة الحركة، لا من اصطلاح بحري.

نعمل على هذه المراحل:

1. نفحص الملف المُعطى `free_fall.csv`: نحدّد عمودَي الزمن والمسافة ووحدتيهما وعدد المشاهدات.
2. نتوقّع شكل y بدلالة t وشكل y بدلالة t^2 قبل تشغيل الملاءمة. نطبّق الوصفة المُعطاة ونحتفظ بالتنبؤات وبواقي المقيس ناقص المتوقّع.
3. ننشئ شكلًا واحدًا: النقاط المقيسة كعلامات مع النموذج الملائم فوقها، وكل محور مسمّى بكميته ووحدته.
4. نحفظ الشكل بصيغة PNG أو PDF ونذكر g والحد الثابت الملائمين. ونشرح هل تدعم البواقي النموذج على المجال المقيس، ونسمّي افتراضًا واحدًا لا تستطيع الملاءمة إثباته.
5. نقدّم السكربت والشكل على صفحة المقرر في Moodle، كما هو محدَّد في ورقة الواجب.

نقدّم سكربتًا كاملًا مع الشكل، وتعريف دالة Julia ليس جزءًا من عمل الأسبوع الأول. ويُصحَّح الواجب الأول يدويًا في Moodle: ناجح/راسب. ومن المفيد أن نُبقي ورقة الواجب بجانبنا، فهي المكان الذي تُذكر فيه الأعمدة المطلوبة والشكل والكميات التي يجب الإبلاغ عنها. ويجب ألّا يؤدي تقريب إجابة معروضة إلى تقريب القيم المستخدمة في الحساب.

1.7 مكافأة اختيارية: اجعل رسمًا علميًا يوصّل الفكرة

هذه المكافأة اختيارية، وليست شرطًا مسبقًا لعمل لاحق، وغير ممتحنة. وكل مكافأة تُنجزها تساوي **ثلث نقطة من العلامة النهائية.**

بعد قراءة مستقلة مدتها 10–15 دقيقة، نحسّن رسمًا أوهليًا مصغّرًا مُعطى. هنا الجهد هو الإحداثي الأفقي والتيار هو الإحداثي الرأسي، لذا يكون ميل النموذج هو الموصلية بوحدة S لا المقاومة بوحدة Ω. ونحدّد سلسلة التيار المقيس وسلسلة التيار الملائم قبل تغيير مظهرهما.

نضيف العنوان المطلوب، والتسميات مع الوحدات، ومدخلات مفتاح الرسم، وعلامات المشاهدات، وخط النموذج، والإطار، وحجم اللوحة، ثم نصدّر PNG وPDF. وتوفّر الشيفرة العددية السلسلتين المتوافقتين المتحقَّق منهما؛ وتُقيَّم الصور منفصلة وفق قائمة تحقّق صريحة. ونتأكد أن تغيير تسمية أو نمط خط لا يغيّر قياسًا. وتحدّد ورقة المكافأة السلاسل النصية وأسماء الملفات بدقة، فيكون الاكتمال، لا الذوق ولا مطابقة البكسل، هو معيار النجاح.

1.8 استكشاف إضافي: منحنيات ليساجو

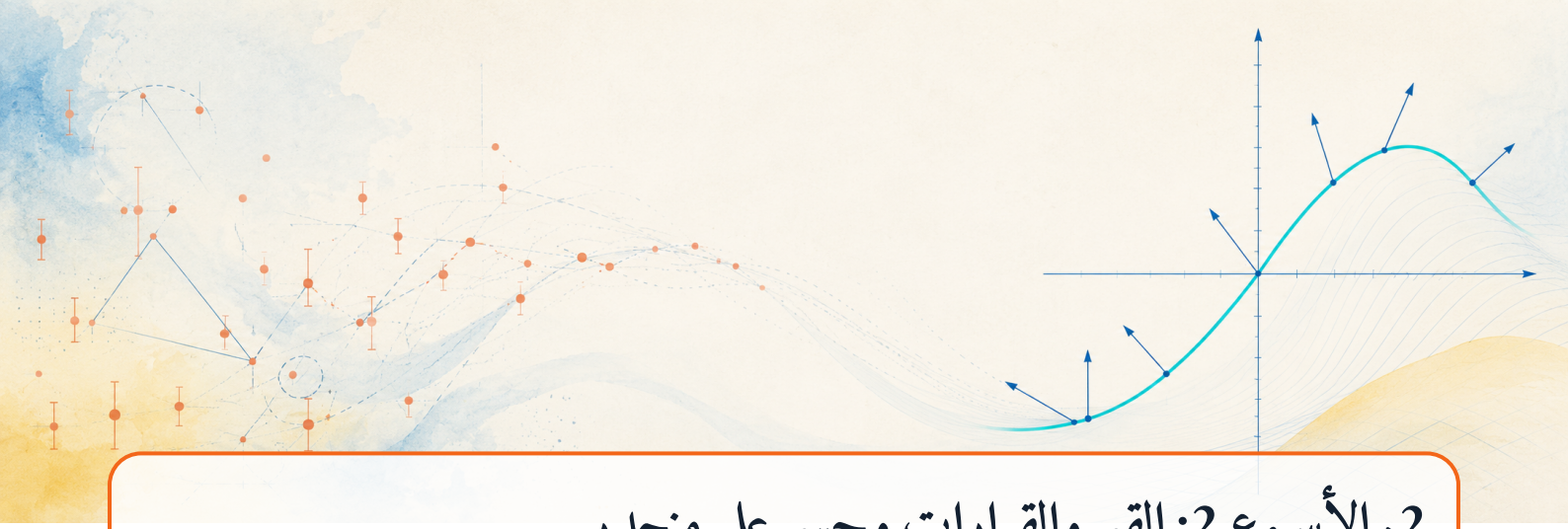
في هذا الاستقصاء الاختياري غير الممتحن نستخدم وصفة رسم مُعطاة لـ

$$x(t) = A\cos(at + \delta), \qquad y(t) = B\sin(bt).$$

هنا t بالثواني، وa, b ترددان زاويان بوحدة rad/s، وδ طور بالراديان، وA, B يحدّدان سعتي الإحداثيين. وتصف هذه المنحنيات حركتين جيبيتين مستقلتين ننظر إليهما معًا؛ وهي ليست مسارات تحت قوة جاذبية مركزية.

نُبقي السعتين ثابتتين ونقارن نسخ الترددات 1 : 1 و1 : 2 و2 : 3. ثم، لنسخة واحدة، نغيّر الطور من 0 إلى $\pi/2$. نحفظ رسمين متباينين ونكتب تخمينًا عن الإغلاق والتناظر. ونتحقق هل يغيّر مدى زمني أطول منحنى يبدو مفتوحًا: فنافذة الرسم المحدودة

ليست دليلًا على عدم الإغلاق. ولا يضيف هذا النشاط أي صياغة برمجية مطلوبة، وهو منفصل عن المكافأة المُقيَّمة.



2. الأسبوع 2: القيم والقرارات وجسم على منحدر

2.1 محاضرة: من المتغيرات إلى الخوارزمية

في الأسبوع الأول منحنا القياسات أسماء وحسبنا قيمًا جديدة منها. وقبل إضافة القرارات، سنتمهّل ونجعل تلك الفكرة الأساسية صريحة: ما هو المتغير، وماذا يفعل الإسناد، وكيف تتغير القيم بينما تنفّذ Julia سطرًا بعد سطر.

2.1.1 المتغير يمنح قيمةً اسمًا

المتغير اسم يشير إلى قيمة. والاسم المفيد يتيح لنا الحديث عن معنى العدد بدل تكرار كتابة العدد نفسه:
صيغة الإسناد العامة هي `name = expression`. تحسب Julia التعبير على اليمين وتجعل الاسم على اليسار يشير إلى النتيجة. وأصغر مثال هو `x = 2`. ويستخدم حساب علمي أغنى أسماءً ذات معنًى متعددة:

```
mass = 2.0          # kg
speed = 3.0         # m/s
momentum = mass * speed   # kg m/s

println(momentum)   # 6.0
```

```
6.0
```

علامة المساواة المفردة هي عامل الإسناد. تحسب Julia أولًا التعبير على يمين =؛ ثم تجعل الاسم على اليسار يشير إلى القيمة الناتجة. لذلك يضرب السطر الثالث أولًا `mass` في `speed`، ثم يسند النتيجة إلى `momentum`.

زخم كتلة متحركة

يقيس زخم الجسم مقدار الحركة التي يحملها: فالجسم الأثقل والأسرع يحمل زخمًا أكبر. ولنقطة كتلة تتحرك بسرعة أقل بكثير من سرعة الضوء، $\mathbf{p} = m\mathbf{v}$، ومقداره $p = mv$ على امتداد اتجاه الحركة. الكتلة m بالكيلوغرام، والسرعة v بالمتر لكل ثانية، والزخم بالكيلوغرام متر لكل ثانية $(\mathrm{kg\,m/s})$.

يمكننا فحص متغير بمجرد كتابة اسمه في خلية دفتر، أو طبعه ضمن رسالة:

```
momentum
println("momentum = $(momentum) kg m/s")
```

في الدفتر يعرض السطر الأول 6.0؛ أما التقرير المطبوع فهو:

```
momentum = 6.0 kg m/s
```

2.1.2 تتابع Julia البرنامج بالترتيب

ينفّذ الإسناد حسابًا في تلك اللحظة. وهو لا ينشئ علاقة جبرية دائمة بين المتغيرات:

```
x = 4
y = x + 1       # y is now 5
x = 10          # x changes, but y is still 5
```

عندما تنفّذ Julia `y = x + 1`، تستخدم القيمة الحالية لـx. وتغيير x لاحقًا لا يعيد حساب y تلقائيًا؛ بل نحتاج إلى تنفيذ `y = x + 1` من جديد. وهذا الترتيب سطرًا بسطر مهم بشكل خاص في الدفاتر، حيث يمكن تشغيل الخلايا أكثر من مرة أو خارج ترتيبها.

2.1.3 يمكن إسناد قيمة جديدة لمتغير

يمكن أن يشير الاسم نفسه إلى قيمة جديدة لاحقًا في البرنامج:

```
temperature = 20.0   # C
temperature = temperature + 0.5

println(temperature)     # 20.5
```

```
20.5
```

السطر الثاني ليس معادلة جبرية. تقرأ Julia القيمة القديمة على اليمين، وتضيف 0.5، وتسند النتيجة الجديدة إلى الاسم نفسه. وستستخدم الحلقات هذا النمط لتقديم الزمن، أو زيادة عدّاد، أو تحديث موضع محاكى.

يجب أن تنقل الأسماء المعنى

الأسماء: أسماء المتغيرات حسّاسة لحالة الأحرف، ولا يمكن أن تحتوي فراغات، ويجب استخدامها باتساق. ويمكن استخدام unicode، ولا سيّما الرموز اليونانية (مثل كتابة `\phi<Tab>` للحصول على ϕ). نفضّل الأسماء ذات المعنى على أسماء غامضة مثل a وb وc. **الوحدات**: تخزّن Julia عادة أعدادًا عادية دون وحداتها. لكن في الفيزياء يُستحسن تتبّعها. ويمكن إضافة لواحق وحدية إلى الأسماء، مثل `m_kg`، أو بدلًا من ذلك، كما نفعل هنا، تسجيل الوحدة في تعليق قصير في نهاية السطر حيث تُعرَّف القيمة، مثل `time = 0.35 # s` و`height = 1.20 # m` و`g = 9.81 # m/s^2`؛ وهذا ليس إلزاميًا لكنه مفيد جدًا لتجنّب أخطاء الوحدات. وفي واجب المكافأة 2 (أي أنه ليس جزءًا من مادة المقرر الرسمية) سنرى أننا نستطيع استخدام حزمة Unitful التي تضيف وحدات إلى المتغيرات وتمكّننا من تتبّعها. ويجب أيضًا ذكر الوحدات في توثيق الدوال أو توثيق الواجهة، وتوضيحها عند مواقع الاستدعاء عندما تكون ملتبسة.

2.1.4 للقيم أنواع

يمكن للمتغيرات أن تسمّي أنواعًا مختلفة من القيم. وقد استخدمنا في الأسبوع الأول عددًا صحيحًا للعدّ، وقياسات عشرية، وتسميات نصية. وهذا الأسبوع نضيف مقارنات منطقية ونجعل أربعة تمييزات صريحة، لأن برنامجًا يقرأ مدخلات ويختار فرعًا ويقرر متى يتوقف يجب أن يعرف نوع القيمة التي يتعامل معها.

النوع فئة من القيم تحدّد كيف تخزّنها Julia وأي العمليات تكون ذات معنى. فمثلًا، لـ5 و0.35 و"height" وtrue صيغ مختلفة، وهي تمثّل عددًا صحيحًا وعددًا عشريًا ونصًا وقيمة صواب/خطأ.

2.1.5 أربعة أنواع قيم بسيطة

الأنواع الأربعة التي نحتاجها الآن

نوع Julia	مثال	الدور العلمي المعتاد
Int	n = 5	العدّ وفهارس الحلقات
Float64	t = 0.35 # s	القياسات والكميات المحسوبة
String	label = "height [m]"	نص المدخلات والتسميات والرسائل
Bool	ready = true	أجوبة الصواب/الخطأ عن الشروط

تتيح لنا الدالة `typeof(value)` فحص نوع قيمة. ونستخدمها للتشخيص، لا بديلًا عن فهم معنى القيمة.

في حاسبات المقرر العادية ذات 64 بت، تطبع `typeof(5)` النص Int64. واسم Int في Julia يعني حجم العدد الصحيح المعتاد للحاسوب، وهو اسم بديل لـInt64 في هذه الحاسبات.

وترتبط هذه الأنواع مباشرةً بالأسبوع الأول:

```
current = [0.10, 0.20, 0.30, 0.40, 0.50]   # A
n = length(current)                    # Int
I_bar = sum(current) / n               # Float64, A
axis_label = "current I [A]"    # String
analysis_ready = true           # Bool

println(typeof(n))
println(typeof(I_bar))
println(typeof(axis_label))
println(typeof(analysis_ready))
```

يمكن لعدد صحيح وعدد عائم أن يشاركا في حساب واحد. وتحوّل Julia العدد الصحيح عند الحاجة:

```
2 * 0.35        # 0.7, a Float64
5 / 2           # 2.5, a Float64
Float64(5)      # 5.0
```

ولهذه المرحلة تكفي هذه القواعد العملية. لا نحتاج إلى تعليقات الأنواع، ولا إلى الأنواع المجردة، ولا إلى تفاصيل تخزين الأعداد العائمة.

النوع ليس وحدة فيزيائية

في `t = 0.35 # s`، النوع في Julia هو Float64، أما الوحدة الفيزيائية فهي الثانية. والوحدة تعيش في التعليق وفي توثيق الواجهة المعنيّ، لذا يمكن لـFloat64 آخر أن يمثّل المتر أو الكيلوغرام. ففحص النوع وتحليل الأبعاد يجيبان عن سؤالين مختلفين، ونحتاج إليهما معًا.

2.1.6 تنتج المقارنات قيمًا منطقية

تحتاج البرامج إلى أسئلة بنعم/لا للتحقق من البيانات واختيار ما تفعله بعد ذلك. وللمقارنة الصيغة `left_value operator right_value` وتنتج قيمة `Bool`: إما true أو false. نبدأ بأسئلة مباشرة أجوبتها واضحة:

```
2 < 3           # true
2 == 3          # false
5 != 4          # true
```

عوامل المقارنة الرئيسية هي > و=> و< و=< و== (يساوي) و=! (لا يساوي). وعلامة المساواة المفردة = تسند قيمة؛ أما المزدوجة == فتطرح سؤالًا.

يمكننا دمج القيم المنطقية عندما لا تكفي مقارنة واحدة. والعوامل && و|| و! تعني و، أو، وليس:

```
true && false      # false: both must be true
true || false      # true: at least one is true
!true              # false
```

ويمكن للصيغ نفسها أن تعبّر عن قاعدة صلاحية فيزيائية أغنى:

```
t = 0.35   # s
z0 = 1.20   # m
nonnegative_time = t >= 0
valid_height = z0 >= 0
valid_input = nonnegative_time && valid_height
```

2.1.7 قرار أول باستخدام if

لا يكون حساب جواب منطقي مفيدًا إلا إذا استطاع البرنامج أن يتصرف بناءً عليه. وعبارة if شرطية: تنفّذ كتلة واحدة عندما يكون شرطها true. وكتلة else الاختيارية تعمل عندما يكون الشرط false. والصيغة العامة هي:

```
if condition
    # commands when the condition is true
else
    # commands when the condition is false
end
```

كل شرطية تنتهي بـend. وإذا لم يكن ينبغي أن يحدث شيء في الحالة الخطأ، نحذف كتلة else. وهنا أصغر مثال ثنائي الاتجاه:

```
temperature = 25.0   # C
if temperature > 30
    println("hot")
else
    println("not hot")
end
```

```
not hot
```

قد يجعل المتابعة حسابًا بلا معنًى في بعض الحالات. وتوفّر Julia توقفًا مقصودًا بهذه الصيغة:

```
throw(ArgumentError("message"))
```

إنه يوقف التنفيذ ويبلّغ عن السبب. وسنستخدمه بعد تحويل فاشل وبعد مدخلات غير صالحة فيزيائيًا.

2.2 الحركية متعددة القطع كقرار

التسارع الموجَّب مقابل مقدار الجاذبية الموجب

الارتفاع موجب نحو الأعلى، لذا يكون تسارع الجاذبية

$$g = -9.81 \text{ m/s}^2.$$

ولارتفاع ابتدائي z_0 وسرعة v_0،

$$z_{\text{free}}(t) = z_0 + v_0 t + \frac{1}{2} g t^2.$$

وللحدود الثلاثة وحدات المتر. وهذا الفحص البُعدي مستقل عن كون قيمها في الكود من النوع Float64.

نموذجنا دون ارتداد لا يسمح للكرة بمواصلة الحركة تحت الأرض:

$$z(t) = \begin{cases} z_{\text{free}}(t), & z_{\text{free}}(t) > 0, \\ 0, & z_{\text{free}}(t) \le 0. \end{cases}$$

ويتحوّل التعريف متعدد القطع مباشرةً إلى شرطية:

```
t = 0.35   # s
z0 = 1.20   # m
v0 = 0.0    # m/s
g = -9.81   # m/s^2

if t < 0 || z0 < 0
    throw(ArgumentError("time and initial height must be non-negative"))
end

z_free = z0 + v0 * t + 0.5 * g * t^2   # m

if z_free > 0
    z = z_free
    state = "in air"
else
    z = 0.0
    state = "on the ground"
end

println("at t = $(t) s: z = $(round(z, digits=3)) m ($(state))")
```

```
at t = 0.35 s: z = 0.599 m (in air)
```

عند $t = 0.35$ s يعرض هذا البرنامج نحو $z = 0.599$ m. وتجتاز النتيجة ثلاثة فحوص: الزمن داخل مجال النموذج، والارتفاع أدنى من قيمته الابتدائية، ولكل حد في معادلة الموضع وحدات المتر.

2.2.1 عدة فروع

نستخدم `elseif` عندما تكون أكثر من حالتين متنافيتين وينبغي اختبارهما بالترتيب. والصيغة العامة هي `if`، يتبعها أي عدد من فروع `elseif`، ثم `else` اختياري، ثم `end` واحد. ولا يعمل إلا أول فرع صحيح. ويعرض مثال صغير لتصنيف الإشارة هذه البنية:

```
value = -2
if value < 0
    description = "negative"
elseif value == 0
    description = "zero"
else
    description = "positive"
end
```

وتعطي البنية نفسها تصنيفًا أكثر فائدة للارتفاع:

```
if z == 0
    state = "on the ground"
elseif z < 0.20
    state = "close to the ground"
else
    state = "in air"
end
```

تكون القرارات المتداخلة ضرورية أحيانًا، لكن سلسلة `else/elseif/if` المسطّحة أسهل في الفحص عندما تعبّر عن المنطق نفسه.

ثلاث فئات من الأخطاء

- **خطأ صياغي:** لا تستطيع Julia تحليل البرنامج، مثلًا بسبب `end` مفقود أو قوس إغلاق ناقص.
- **خطأ زمن التشغيل:** يبدأ البرنامج ثم يصل إلى عملية غير صالحة، مثلًا رفع String إلى قوة.
- **خطأ منطقي:** يعمل البرنامج لكنه ينفّذ الخوارزمية أو الفيزياء الخطأ، مثلًا استخدام g موجب بينما الارتفاع موجب نحو الأعلى، أو عدم تقديم الزمن في حلقة `while`.

نقرأ الخطأ الدقيق ورقم السطر، ونعزل أصغر كتلة فاشلة، ونفحص القيم والأنواع، ونتحقق من الأبعاد والإشارات، ونختبر حالات أجوبتها معروفة سلفًا.

نجعل البرامج الآن تختار بين حالات فيزيائية مختلفة. وقبل الوثوق بأي حساب، علينا أن نعرف نوع القيمة التي بين أيدينا، وأن نحوّل عن قصد، وأن نذكر المجال الذي ينطبق فيه النموذج.

2.3 للقيم أنواع

تحمل قيم Julia أنواعًا. فالأعداد الصحيحة والأعداد العشرية والنصوص وقيم الصواب تخدم أغراضًا مختلفة.

```
println(3, "  ->  ", typeof(3))
println(3.0, "  ->  ", typeof(3.0))
println("3", "  ->  ", typeof("3"))
println(3 < 4, "  ->  ", typeof(3 < 4))
```

```
3  ->  Int64
3.0  ->  Float64
3  ->  String
true  ->  Bool
```

قد يعتمد نوع العدد الصحيح الدقيق على الحاسوب؛ لكن التمييز المهم هو بين `Int` والعائم و`String` و`Bool`.

يمكن لعدد صحيح وعدد عائم أن يختلطا في الحساب العادي؛ وترفع Julia النوع عند الحاجة. أما النص فليس عددًا ضمنًا. ونختار التحويلات صراحةً في الاتجاهين.

```
println(2 + 0.5)
text = "12.5"
mass = parse(Float64, text)   # kg
println(mass + 0.5)
println(string(mass, " kg"))
println(Int(7.0))
```

```
2.5
13.0
12.5 kg
7
```

يطلق `parse(Float64, "twelve")` خطأً لأن النص ليس عددًا عشريًا. ويطلق `Int(7.2)` خطأً أيضًا: فالتحويل إلى `Int` مسموح فقط عندما تكون القيمة صحيحة تمامًا. وهذا يحمينا من التخلص من المعلومة بصمت.

2.4 القيم المنطقية والقرارات التدريجية

تنتج المقارنات قيمًا منطقية. نسأل أولًا سؤالًا واحدًا، ثم نختار بين حالتين، ثم نضيف حالة ثالثة بـ`elseif`.

```
temperature = -2.0   # C
println(temperature < 0)
if temperature < 0
    println("below freezing")
else
    println("not below freezing")
end
```

```
true
below freezing
```

```
if temperature < 0
    state = "ice"
elseif temperature == 0
    state = "melting-point"
else
    state = "liquid"
end
println(state)
```

```
ice
```

`ice`. ولا يعمل من سلسلة `else/elseif/if` إلا فرع واحد بالضبط. و= يسند، و== يقارن.

نجمع الشروط بـ`&&` (كلاهما) و`||` (أحدهما على الأقل) و`!` (ليس). وغالبًا ما يسأل فحص الصلاحية هل القيمة عدد منتهٍ. ويعيد `isfinite(value)` القيمة true للأعداد المنتهية العادية وfalse لـ`NaN` و`Inf` و`-Inf`، وهي ليست قياسات قابلة للاستخدام. ويمكننا دمج ذلك الفحص مع شرط فيزيائي:

```
is_finite_positive = isfinite(2.0) && 2.0 > 0
println(is_finite_positive)
println(!is_finite_positive || 2.0 == 2)
```

```
true
true
```

2.5 الزوايا والمنطق والتحقق قبل الحساب

الشرط سؤال رياضي دقيق

يكون `a && b` صحيحًا فقط عندما يكون الشرطان صحيحين؛ ويكون `a || b` صحيحًا عندما يكون أحدهما على الأقل صحيحًا. ويكون `xor(a,b)` صحيحًا عندما يكون أحدهما بالضبط صحيحًا. وتفيد هذه القواعد في اختيار حالات متنافية، لكن على النموذج أن يحدّد ما يحدث عند المساواة. ومعنى متباينة متسلسلة مثل $0 \le \theta < 90^\circ$ أن يتحقق الحدّان معًا، ويمكننا كتابتها بمقارنتين موصولتين بـ`&&`.

تحتاج الزاوية إلى وحدة صريحة. يتوقّع `sin` و`cos` الراديان، بينما يتوقّع `sind` و`cosd` الدرجات، ويحوّل `rad2deg` زاوية من الراديان إلى الدرجات. أما الصيغة الثنائية `atan(y, x)` فتعيد زاوية النقطة (x, y) واعيةً بالربع: فهي تستخدم الإشارتين معًا، لا نسبتهما فقط، فتستطيع التمييز بين اتجاهين متعاكسين. وفيما يلي مثال صغير لكلٍّ منها:

```
println(true && false)
println(true || false)
println(xor(true, true))
println(xor(true, false))
println(sin(pi / 6))
println(sind(30.0))
println(round(rad2deg(atan(1.0, -1.0)); digits=1))
```

```
false
true
false
true
0.49999999999999994
0.5
135.0
```

تقع النقطة $(-1, 1)$ عند 135°، لا عند -45°؛ ولا تستطيع الصيغة ذات الأرجومنت الواحد `atan(y/x)` التمييز بينهما. وعند نقطة الأصل لا يوجد اتجاه فيزيائي؛ وعلى أي تطبيق أن يذكر اصطلاحًا. والفروق الصغيرة بين حسابات فاصلة مكافئة ليست فيزياءً مختلفة.

تحويل النص والتحقق الفيزيائي عمليتان منفصلتان

يعيد `readline()` نصًا. ويحوّل `parse(Float64, text)` النص العددي الصالح إلى عدد؛ والنص المشوّه يطلق خطأً. ولا يكفي نجاح التحويل: فـ`NaN` و`Inf` ليست قياسات منتهية، والكتلة السالبة المنتهية خارج نموذجنا. نتحقق قبل جذر أو قسمة أو بناء مثلثي. ويوقف `throw(ArgumentError("message"))` الحساب؛ أما مجرد طباعة رسالة خطأ فلا توقفه.

قبل اختبار مجال فيزيائي، نتأكد أن القيمة منتهية: يكون `isfinite(value)` صحيحًا فقط للأعداد المنتهية العادية، وخطأً لـ`NaN` أو `Inf`. وتحوّل الشيفرة العددية التالية نصًا وتتحقق من نتيجتها ككتلة:

```
text = "2.5"  # reproducible example of text supplied by readline()
mass = parse(Float64, text)   # kg
if !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite and positive"))
end
println("mass = ", mass, " kg")
```

```
mass = 2.5 kg
```

وتبقى أمثلة `nothing/tryparse` السابقة بأسلوب التعافي للقراءة الاختيارية في معالجة المدخلات. أما المسار الأساسي هذا الأسبوع فيستخدم `parse` وفحوص المجال الفيزيائي الصريحة، ولا يتطلب أي منهما تعريف دالة.

2.6 نموذج احتكاك متعدد القطع على منحدر

القوى والإشارات والافتراضات

نأخذ $+x$ نحو الأعلى بمحاذاة منحدر ثابت و$+y$ عمودياً عليه. ولجسم كتلته m (كغ) وزاوية θ (بالراديان في `cos/sin`) ومقدار جاذبية موجب g ($\mathrm{m/s^2}$)، للجاذبية مركّبة نزولية $mg\sin\theta$ وقوة عمودية $N = mg\cos\theta$. ويمكن للاحتكاك السكوني أن يعارض الحركة الوشيكة نحو الأسفل حتى $f_{s,\max} = \mu_s N$. وإذا لم يستطع التثبيت، يعمل الاحتكاك الحركي $f_k = \mu_k N$ نحو الأعلى ويكون $a = (-mg\sin\theta + \mu_k N)/m$. نفترض احتكاك كولوم وتلامسًا صلبًا ولا مقاومة هواء و$0 \le \theta < 90^\circ$. ولا يصف النموذج التدحرج أو تغيّر التلامس أو احتكاكًا يعتمد على السرعة. وعند $\theta = 0$ يكون الاحتكاك السكوني المطلوب صفرًا، وإذا كان $\mu_k = 0$، يعطي الانزلاق $a = -g\sin\theta$.

نتحقق قبل استخدام أي مدخل. يوقف `throw(ArgumentError(...))` الحساب فورًا ويخبر مستدعيه أن شرطًا موثّقًا في المدخل قد فشل. وهو يختلف عن `println` الذي يكتفي بعرض نص ثم يكمل. ويسأل اختبار العضوية `value isa T` هل تنتمي القيمة إلى النوع `T`، ويسمّي `Real` عائلة Julia للأنواع الحقيقية، التي تضم الأعداد الصحيحة والعائمة العادية. ويحتاج `Bool` إلى عناية خاصة هنا: فبما أن `true isa Real` صحيحة أيضًا، فإن فحص `Real` وحده سيقبل `true` ككتلة، لذا نرفضه صراحةً.

```
mass = 2.0      # kg
angle = 30.0    # deg
 μ_s = 0.60
 μ_k = 0.40
g = 9.81        # m/s^2
if !(mass isa Real) || mass isa Bool || !isfinite(mass) || mass <= 0
    throw(ArgumentError("mass must be finite, positive, and non-Boolean
"))
elseif !isfinite(angle) || !(0 <= angle < 90) || μ_k < 0 || μ_k > μ_s
    throw(ArgumentError("use 0 <= angle < 90 and 0 <= μ_k <= μ_s"))
end
 θ = deg2rad(angle)   # rad
normal = mass * g * cosθ()   # N
needed_static = mass * g * sinθ()   # N
if needed_static <= μ_s * normal
    state = "stuck"
    acceleration = 0.0   # m/s^2
else
    state = "sliding"
    acceleration = (-needed_static + μ_k * normal) / mass   # m/s^2
end
println(state, ", a = ", acceleration, " m/s^2")
```

```
stuck, a = 0.0 m/s^2
```

`stuck, a = 0.0 m/s^2`. هنا $mg\sin 30^\circ = 9.81$ N و$\mu_s N \approx 10.19$ N، فالاحتكاك السكوني كافٍ. وإذا غيّرنا `μ_s` وحده إلى `0.40`، تكون النتيجة `sliding` مع تسارع سالب: أي نحو الأسفل في اصطلاحنا المعلن. والمساواة تنتمي إلى فرع التثبيت، وهو اصطلاح العتبة الصريح عندنا.

2.7 التمرين: قوة سحب قد تعكس حركة وشيكة

نضيف إلى نموذج المنحدر قوة سحب معروفة نحو الأعلى P. وقبل اختيار إشارة الاحتكاك، نحسب النزعة بلا احتكاك $T = P - mg\sin\theta$. فالنزعة الموجبة T تعني حركة وشيكة نحو الأعلى، والسلبية تعني حركة وشيكة نحو الأسفل. ويعارض الاحتكاك تلك النزعة، ويمكن للاحتكاك السكوني أن يثبّت متى كان $|T| \le \mu_s N$.

نصل قادرين على قراءة سلسلة `else/elseif/if` وتحويل الدرجات إلى راديان. وفي الجزء الموجَّه القصير نتوقّع الاتجاه، ونتتبّع فرعًا واحدًا، ونختبر $T = 0$. ثم نبدأ واجب خلوص الجدار المستقل. والفكرة الجوهرية طريقة قابلة للتكرار: نتحقق من المدخلات، ونصوغ شرطًا فيزيائيًا، ونحسب الحالة المنطبقة، ونفحص حدّها.

2.8 الواجب الأساسي: هل يعبر المقذوف جدارًا؟

نطلق من مستوى الأرض بسرعة v_0 وزاوية θ ومقدار جاذبية $g > 0$، مع $+x$ نحو الجدار و$+y$ نحو الأعلى. وفي نموذج الجاذبية المنتظمة دون مقاومة هواء،

$$v_x = v_0\cos\theta, \quad t_w = x_w / v_x, \quad y_w = v_0\sin\theta\, t_w - \tfrac{1}{2} g t_w^2, \quad v_{y,w} = v_0\sin\theta - g t_w.$$

ونشترط $v_x > 0$ قبل القسمة. نقارن y_w بارتفاع الجدار، ونستخدم $v_{y,w}$ لتصنيف الصعود أو الهبوط. والمساواة في الارتفاع تُحتسب عبورًا؛ والسرعة الرأسية الصفرية تنتمي إلى فئة الهبوط المنشورة. وقد يكون الارتفاع المحسوب في الطيران الحر سالبًا: وبالنسبة لارتفاع جدار غير سالب يعني ذلك فشل العبور، لا استمرار الحركة فيزيائيًا عبر الأرض.

نكتب أولًا تنبؤًا لحالة عادية وأخرى حدّية. الواجب 2 برنامج كامل يُسلَّم في CodeRunner: يطبع طلبات الإدخال الخمسة، ويقرأ القيم باستخدام `readline()`، ويتحقق من المجال الفيزيائي، ويطبع سطر نتيجة واحدًا عند السقوط قبل الجدار أو ثلاثة أسطر خلاف ذلك. يزوّد CodeRunner البرنامج بخمسة أسطر إدخال ويقارن الإخراج المطبوع كاملًا، بما فيه طلبات الإدخال. نقرّب الارتفاع المعروض إلى أربع منازل عشرية، ونستخدم القيمة الكاملة قبل التقريب لتحديد الحالة والنتيجة. ويظل قرار العبور منفصلًا عن تصنيف الحركة الرأسية: فقد يعبر المقذوف وهو هابط. لا حاجة إلى دوال أو حلقات يعرّفها الطالب.

2.9 مكافأة اختيارية: الوحدات وأطوار التيار المتردد

هذه المكافأة اختيارية، وليست شرطًا مسبقًا لعمل لاحق، وغير ممتحنة. وتساوي **ثلث نقطة من العلامة النهائية**.

تطوّر القراءة المستقلة الأعداد المركبة والمقدار والطور والمرافق وعلاقة أويلر قبل استخدام `cis`. نختار سعات الذروة واصطلاح الزمن $\exp(+i\omega t)$، ونضع طور الجهد صفرًا، ونبني $Z = |Z|\exp(i\phi_Z)$، ونحسب $I = V/Z$. والتيار القابل للقياس هو الجزء الحقيقي من $I\exp(i\omega t)$؛ فالعدد المركب تمثيل، لا تيار مقيس إضافي.

نتبع أمثلة Unitful المُعطاة، ونتحقق من أن قسمة الجهد على المقاومة لها وحدة التيار، ونفحص مقاومًا بطور صفري قبل تجربة طور ربع دورة. ثم نقدّم الشيفرة العددية التي تسند التيار المركب بالأمبير ونشغّل فحوص الوحدات المكافئة والمدخلات غير الصالحة المنشورة. ونشرح لماذا يعطي طور معاوقة موجب طور تيار سالبًا. هذا نموذج أحادي التردد خطي في الحالة المستقرة؛ والانتقالات العابرة والمكوّنات غير الخطية خارج نطاقه.

2.10 استكشاف إضافي: الانعكاس الداخلي الكلي

نستخدم وصفة قانون سنيل المُعطاة $n_1 \sin\theta_1 = n_2 \sin\theta_2$، مع قرائن انكسار موجبة وزوايا مقيسة من العمودي على السطح الفاصل. وعندما يكون $n_1 > n_2$، يعطي السقوط بعد $\theta_c = \arcsin(n_2/n_1)$ انعكاسًا داخليًا كليًا. ونصنّف الأمثلة دون الحرجة والحرجة وفوق الحرجة قبل حساب جيب معكوس؛ فحساب `asin` لعدد أكبر من واحد دون تفكير ليس حسابًا فيزيائيًا.

نرسم انعكاسات مثالية متكررة في ليف مستقيم ونغيّر زاوية الإطلاق. ونسجّل الخيارات التي تبقى موجَّهة ونشرح العمودي المستخدم عند كل سطح فاصل. وأي حلقة تتبع مُعطاة هي معاينة اختيارية، لا عمل أساسي في الأسبوع الثاني. وتتطلب أنماط الدليل الموجي والتشتت وفقد الانحناء والامتصاص فيزياءً أكثر من نموذج الأشعة هذا. وهذا الاستقصاء منفصل عن مكافأة الطور، وهو غير ممتحن وغير شرط مسبق.



3. الحلقات والمتجهات والحركة المرصودة

تسأل مسائل فيزيائية كثيرة عن الحساب نفسه في أزمنة أو مواضع كثيرة. ونجعل الآن التكرار صريحًا ونخزّن تاريخه في متجهات. المتجه مجموعة مرتبة من القيم، والحلقة تكرّر كتلة من التعليمات، ومعًا تعطياننا طريقةً نخطو بها بنموذج إلى الأمام في الزمن. وهدفنا أن نفهم أيّ التعليمات تتكرر، وما الذي يتغير في كل خطوة، وما الدليل الذي يخبرنا أن نتوقف. ونستدعي أدوات قائمة، أما تعريف دوالنا فيعود إلى الأسبوع الخامس.

3.1 المتجهات تخزّن تواريخًا مرتبة

3.1.1 إنشاء المتجهات

المتجه مجموعة مرتبة تُكتب بأقواس مربعة. ويمكننا كتابة متجه مباشرة، أو البدء بمتجه فارغ، أو طلب متجه جاهز من أصفار أو وحدات:

```
v = [10, 20, 30, 40]
empty_int = Int[]
println(v)
println(empty_int)
println(zeros(5))
println(ones(4))
println(collect(1:5))
println(range(0, 1; length=6))
```

```
[10, 20, 30, 40]
Int64[]
[0.0, 0.0, 0.0, 0.0, 0.0]
[1.0, 1.0, 1.0, 1.0]
[1, 2, 3, 4, 5]
0.0:0.2:1.0
```

التعبير `1:5` هو مدى (range)، لا متجه: فهو يصف القيم من 1 إلى 5 دون تخزينها. ونستدعي `collect` عندما نريد متجهًا فعليًا. ويظهر المدى `0.0:0.2:1.0` الناتج عن `range(0, 1; length=6)` كمدى لأن ست نقاط متباعدة بالتساوي من 0 إلى 1 تفصل بينها 0.2. ويعطي `zeros(5)` متجهًا من خمسة أصفار بفاصلة عائمة، ويعطي `ones(4)` أربع وحدات. و `Int[]` متجه فارغ ستكون عناصره أعدادًا صحيحة. ومصفوفات Julia قابلة للتغيير: يمكننا تغيير العناصر أو الطول بعد الإنشاء، كما سنرى قريبًا.

3.1.2 الفهرسة والتقطيع

تبدأ المتجهات العادية في Julia من الفهرس 1. ويحدد الفهرس موضعًا لا القيمة المخزنة فيه، و `end` داخل تعبير فهرسة يحدد الموضع الأخير.

```
v = [10, 20, 30, 40, 50]
println(v[1])
println(v[end])
println(v[2:4])
println(v[2:end])
v[2:3] = [99, 88]
println(v[[1, 3, 5]])
indices = [2, 3]
println(v[indices])
println(v)
```

```
10
50
[20, 30, 40]
[20, 30, 40, 50]
[10, 88, 50]
[99, 88]
[10, 99, 88, 40, 50]
```

تشمل التقطيعة مثل `v[2:4]` الطرفين. ويمكننا الإسناد إلى تقطيعة، ويمكننا تحديد مواضع متعددة دفعة واحدة بتمرير متجه من المواضع إلى الفهرس. وتخبرنا الدوال `length(v)` و`firstindex(v)` و`lastindex(v)` بعدد العناصر وبأول وآخر موضع صالح.

التقطيعة نسخة؛ والاسم البديل هو المتجه نفسه

بالنسبة لمتجه عادي، ينسخ `part = height[2:3]` تلك العناصر. وتغيير `part[1]` لا يغير `height`. وفي المقابل، يجعل `same = height` اسمًا ثانيًا يشير إلى المتجه نفسه، وتغيير `same[1]` يغير الكائن المشترك. وينشئ `copy(a)` نسخة حقيقية مستقلة. أما المنظر (view) فيشارك التخزين مع الأصل دون نسخ: يسمح لنا `@view a[1:2]` بقراءة وكتابة أول عنصرين من `a` عبر اسم جديد. وعلامة `@` تدل على ماكرو، وهو تحويل صغير مدمج للتعبير الذي يليه؛ نستخدمه هنا فقط لطلب المنظر، ولا نحتاج إلى كتابة ماكروهات بأنفسنا.

```
a = [1, 2, 3]
b = a
c = a[1:2]
d = copy(a)
e = @view a[1:2]
a[1] = 2
println("a[1]=", a[1], " b[1]=", b[1], " c[1]=", c[1], " d[1]=", d[1], "
 e[1]=", e[1])
```

```
a[1]=2 b[1]=2 c[1]=1 d[1]=1 e[1]=2
```

يتبع الاسم البديل `b` والمنظر `e` التغيير، أما النسخة `d` والتقطيعة `c` فلا تتبعانه. ويعتمد السلوك الذي نريده على ما إذا كنا نقصد التخزين نفسه أم لقطة منه.

3.1.3 تغيير متجه

علامة التعجب ! في نهاية اسم دالة هي اصطلاح Julia للعملية المغيِّرة: فقد تغيّر الوسيط نفسه. تضيف push! قيمة واحدة، وتضيف append! كل عنصر من مجموعة، وتزيل pop! العنصر الأخير وتعيده. وتغيّران insert! وdeleteat! موضعًا مختارًا.

```
v = [1, 2, 3]
push!(v, 4); println(v)
append!(v, [5, 6]); println(v)
pop!(v); println(v)
insert!(v, 2, 99); println(v)
deleteat!(v, 2); println(v)
w = vcat(v, [7, 8]); println(w); println(v)
```

```
[1, 2, 3, 4]
[1, 2, 3, 4, 5, 6]
[1, 2, 3, 4, 5]
[1, 99, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5, 7, 8]
[1, 2, 3, 4, 5]
```

لاحظوا الفرق: تضيف push! كائنًا واحدًا، وتضيف append! كل عنصر من مجموعة، بينما يبني vcat متجهًا جديدًا موصولًا ويترك v دون تغيير. وفي أمثلة الحلقات القادمة سننمي التواريخ بـ push!، وهي الطريقة الطبيعية لتخزين تسلسل عينات أثناء حسابها.

3.1.4 حساب المتجهات والدوال المفيدة

يعمل الحساب على المتجهات عنصرًا عنصرًا، وتلخص بعض الدوال متجهًا كاملًا:

```
a = [1.0, 2.0, 3.0]
b = [4.0, 5.0, 6.0]
println(a + b)
println(2a)
println(sum(a)); println(maximum(a)); println(minimum(a)); println(
length(a))
println(a .* b)
println(a .^ 2)
println(sqrt.(a))
using LinearAlgebra
println(dot(a, b))
println(round(norm([1,1]); digits=4))
```

```
[5.0, 7.0, 9.0]
[2.0, 4.0, 6.0]
6.0
3.0
1.0
3
[4.0, 10.0, 18.0]
[1.0, 4.0, 9.0]
[1.0, 1.4142135623730951, 1.7320508075688772]
32.0
1.4142
```

تشير النقطة إلى عملية عنصرًا-عنصر: يضرب `a .* b` العناصر المتطابقة، بينما يكون `a * b` خطأً لهذه الأشكال. وتعمل المؤثرات المنقوطة عنصرًا عنصرًا، لذا يربّع `a .^ 2` كل عنصر، ويطبق `sqrt.(a)` الجذر التربيعي على كل عنصر. وتبلغ `sum` و`maximum` و`minimum` و`length` عن عدد واحد للمتجه كله. وتوفر الوحدة القياسية `LinearAlgebra` الجداء السلمي `dot(a, b)` والطول `norm(a)`؛ ومثل `Statistics`، هي جزء من Julia ولا تحتاج إلى تركيب.

تمرين سريع

أنشئوا متجهًا من خمس قيم، واستبدلوا عنصره الثالث، وأضيفوا قيمتين إضافيتين، واحسبوا متوسطه باستخدام `sum(v) / length(v)`. ثم تحققوا من المتوسط بـ `using Statistics; mean(v)`.

3.2 تكرار مخطَّط: حلقة for

تكرّر الحلقة كتلة من التعليمات. وتزور حلقة `for` مجموعة قيم مُعطاة بالترتيب: `for value in collection`، ثم كتلة، ثم `end`. وقبل تشغيلها نسرد القيم التي سيأخذها متغير الحلقة. ويبدأ التعليق بالرمز #؛ وهو يشرح خيارًا لقارئ ولا يُنفَّذ.

```
for n in 1:3
    println(n)
end
```

```
1
2
3
```

يشمل المدى `1:3` الطرفين. أما الصيغة الأعم `start:step:stop` فتزور القيم المتفاصلة بالخطوة دون تجاوز حد التوقف. ونستخدم خطوة سالبة للعدّ نحو الأسفل. ولا يُشمَل حد التوقف نفسه إلا عندما تصل إليه المسافة بين القيم.

```
for time in 0:2:10   # s
    println("time = $time s")
end
```

```
time = 0 s
time = 2 s
time = 4 s
time = 6 s
time = 8 s
time = 10 s
```

تُدرج علامة الدولار قيمة في سلسلة نصية. ولحساب بدل اسم مفرد نستخدم `$(expression)`، كما في `println("square = $(n^2)")`.

ثمة طرق عدة لزيارة عناصر متجه. يمكن لمتغير الحلقة أن يأخذ قيمًا صحيحة، أو القيم نفسها، أو فهارس صالحة، أو زوج فهرس-قيمة:

```
v = [10, 20, 30]
for i in 1:3
    println(i)
end
for x in v
    println(x)
end
for i in eachindex(v)
    println("index = ", i, ", value = ", v[i])
```

```
end
for (i, x) in enumerate(v)
    println(i, " => ", x)
end
```

```
1
2
3
10
20
30
index = 1, value = 10
index = 2, value = 20
index = 3, value = 30
1 => 10
2 => 20
3 => 30
```

يعتمد اختيار الصيغة على ما يحتاج إليه الجسم. فعندما تكون القيم فقط هي المهمة، يكون `for x in v` أوضح. وعندما نفهرِ أو نعدّل عناصر، يوفر `eachindex(v)` الفهارس الصالحة لذلك المجموع، فلا نضطر إلى افتراض مدى فهارس معين. ويزود `enumerate` عدّادًا إلى جانب كل عنصر؛ وعدّاده هو الموضع العادي الذي يبدأ من 1 هنا، وهذا كل ما نحتاجه.

المراكم قيمة تُحمَل بين التكرارات

لجمع قيم عدة، نهيّئ مجموعًا، ونضيف مساهمة واحدة، ونحتفظ بالمجموع الجديد للخطوة التالية. وعلينا التهيئة خارج الكتلة المكررة؛ فإعادة ضبطها داخلها ستُهدر المساهمات السابقة. ويسجّل العدّاد كم خطوة نفّذنا، بينما يسجّل المراكم نتيجتها المجمّعة. وتجمّع الأمثلة أدناه بياناتها في كتلة `begin ... end`، فتقرأ كل دالة كوحدة واحدة ويمكن لصقها وتشغيلها كما هي.

```
begin v = [3, 5, 8, 2]
    total = 0
    for x in v
        total += x
    end
    println(total)
    println(sum(v))
end
```

```
18
18
```

تعيد الحلقة إنتاج `sum(v)` بإضافة واحدة في كل مرة. وهذا هو الهيكل الأساسي لمعظم عمليات التجميع في هذا المقرر: ابدأ بقيمة حيادية، وادمج عنصرًا واحدًا في كل تكرار، واحتفظ بالنتيجة للخطوة التالية. والمضروب هو النمط نفسه مع ضرب بدل الجمع:

```
begin f = 1
    for n in 1:5
        f = f * n
    end
    println(f)
end
```

```
120
```

العدّاد مراكم للخطوات التي تحقق شرطًا:

```
begin v = [-2, 3, 0, 7, -1]
    count_positive = 0
    for x in v
        if x > 0
            count_positive += 1
        end
    end
    println(count_positive)
end
```

```
2
```

إيجاد قيمة عظمى دون `maximum` يستخدم مراكمًا يتحسن فقط:

```
begin v = [4, 9, 2, 7]
    largest = v[1]
    for x in v
        if x > largest
            largest = x
        end
    end
    println(largest)
end
```

```
9
```

بناء متجه من مجاميع جارية يجمع مراكمًا مع `!push`:

```
begin v = [2, 3, 5, 7]
    running = 0
    cumulative = Int[]
    for x in v
        running += x
        push!(cumulative, running)
    end
    println(cumulative)
    println(cumsum(v))
end
```

```
[2, 5, 10, 17]
[2, 5, 10, 17]
```

هنا يسجّل المتجه `cumulative` الحالة في كل خطوة، لا الأخيرة فقط. وهذا هو النمط الذي سنستخدمه لتخزين مسار: ادفع كل عينة جديدة أثناء حسابها، وقارن بنتيجة معلومة.

عملية متجهية شائعة هي الجداء السلمي، التي تستخدم أيضًا مراكمًا عبر فهارس متطابقة:

```
using LinearAlgebra
begin
    a = [1.0, 2.0, 3.0]
    b = [4.0, 5.0, 6.0]
    s = 0.0
```

```
    for i in eachindex(a, b)
        s += a[i] * b[i]
    end
    println(s)              # 32.0
    println(dot(a, b), " ", ·ab)   # 32.0 32.0
end
```

```
32.0
32.0 32.0
```

يجب أن يكون للمتجهين الطول نفسه حتى تكون الحلقة منطقية، وهذا بالضبط ما يشترطه `eachindex(a, b)`. وتجميع مجموع بـ `s += a[i] * b[i]` هو النسخة اليدوية للجداء السلمي.

3.3 الملخصات: المتوسط والانحراف المعياري للعينة

الموقع والانتشار يجيبان عن سؤالين مختلفين

بالنسبة لـ n مشاهدة q_i، المتوسط هو $\bar{q} = \sum_i q_i/n$. والانحراف المعياري المصحح للعينة هو

$$s = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(q_i - \bar{q})^2}, \qquad n \geq 2.$$

ولكل منهما وحدات q. يحدد المتوسط موقع العينة، ويقيس الانحراف المعياري انتشارها. ولا يستبعد رقم عشري أصغر ولا انتشار صغير خطأً منهجيًا في القياس. وتفسير $s/\sqrt{n}$ بعدم يقين للمتوسط يتطلب افتراضات إضافية عن قياسات متكررة مستقلة.

`Statistics` وحدة مكتبة قياسية تأتي مع Julia، فلا تحتاج إلى تركيب حزمة. ويجعل الإعلان `using Statistics` الدالتين `mean` و`std` متاحتين:

```
using Statistics
measurements = [1.0, 2.0, 3.0]   # m
println(mean(measurements))
println(std(measurements))
```

```
2.0
1.0
```

تستخدم `std` الافتراضية التصحيح أعلاه، وتعمل الدالتان على متجه كامل. والمسار ليس تلقائيًا مجموعة قياسات متكررة مستقلة؛ فالانتشار في المواضع لا يقدّر بذاته عدم يقين آلي.

3.4 تكرار متحكم فيه بشرط: while

تفحص حلقة `while` شرطها قبل كل تكرار. ونحدد تهيئتها وشرطها وتحديثها وحجة إنهائها. وقد تُنفَّذ صفر مرات. ويغادر `break` الحلقة الأقرب فورًا، وفي بحث محدود يسمح لنا بالتوقف عند أول نجاح.

```
for n in 1:10
    if n^2 >= 20
        println("first square at least 20 is $(n^2)")
        break
    end
```

```
end
```

```
first square at least 20 is 25
```

التوقف جزء من النموذج العلمي

نميّز بين أول عينة ناجحة والزمن الدقيق الذي يحدث فيه حدث متصل. ونحتفظ بحد أقصى لعدد الخطوات عندما لا يكون الإنهاء مثبتًا. ويمكن لتحديث مفقود أن يصنع حلقة لا نهائية؛ ويمكن للامساواة خاطئة أن تتجاوز حدثًا ابتدائيًا أو حديًا. وبرامج إدخال الحارس وتمارين `continue` المتقدمة قراءة اختيارية لاحقة، لا شروط مسبقة هنا.

3.5 من المشتقة إلى تحديث أويلر

حركة المقذوف: قوانين وحدود

نأخذ $+x$ يمينًا و$+y$ إلى الأعلى. والمقذوف النقطي في جاذبية منتظمة تقريبًا، بلا مقاومة هواء، يحقق $\dot{x} = v_x$، $\dot{y} = v_y$، $\dot{v}_x = 0$، و$\dot{v}_y = -g$. والمواضع بالمتر، والسرعات بـ m/s، والزمن بالثواني، و$g > 0$ بـ $\mathrm{m/s}^2$. ويجب أن تبقى السرعة الأفقية ثابتة. وتستبعد المقاومة والارتداد وتغير الجاذبية وانحناء الأرض؛ ولا يمكن لخطوة عددية استعادة هذه التأثيرات المفقودة.

القيم القديمة تحدد الحالة التالية

يعطي تعريف المشتقة $[y(t+\Delta t) - y(t)]/\Delta t \approx v_y(t)$ عبر خطوة صغيرة. وهكذا يستخدم أويلر الصريح:

$$x_{n+1} = x_n + v_{x,n}\Delta t, \quad y_{n+1} = y_n + v_{y,n}\Delta t, \quad v_{y,n+1} = v_{y,n} - g\Delta t.$$

وتحديث الموضع بالسرعة الجديدة بدلًا من ذلك يعطي خوارزمية مختلفة. ويتضمن الارتفاع الدقيق في هذا المثال ذي الجاذبية الثابتة الحد الإضافي $-g\Delta t^2/2$ في كل خطوة. ويتجاهل أويلر ذلك الحد، وتنتج الإهمالات الصغيرة المتكررة خطأ خطوة زمنية عامًا.

نخزّن الموضع والسرعة في متجهين ذوي مركبتين، $\mathbf{r} = (x, y)$ و$\mathbf{v} = (v_x, v_y)$، مع $\mathbf{a} = (0, -g)$. وفي كل فترة زمنية قصيرة نستبدل المشتقة بفرق منتهٍ:

$$\frac{dx}{dt} \approx \frac{x(t+\Delta t) - x(t)}{\Delta t} \quad \Longrightarrow \quad x(t+\Delta t) \approx x(t) + v_x(t)\Delta t.$$

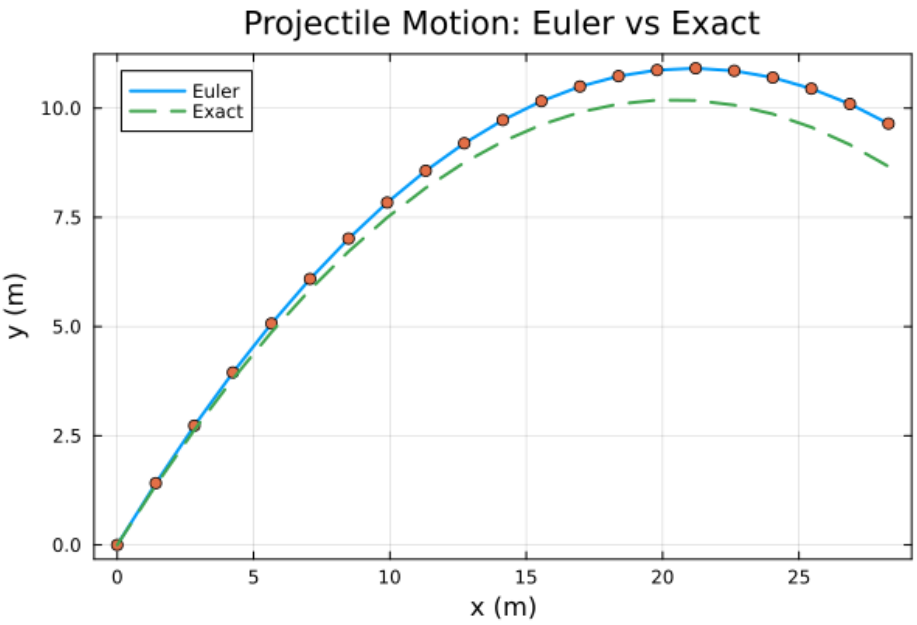
وبالطريقة نفسها نحصل على $\mathbf{r}_{n+1} = \mathbf{r}_n + \mathbf{v}_n\Delta t$ و$\mathbf{v}_{n+1} = \mathbf{v}_n + \mathbf{a}_n\Delta t$. تُحسب كل حالة جديدة من الحالة السابقة. وتحديثا مركبتي السرعة هما $v_{x,n+1} = v_{x,n} + a_{x,n}\Delta t$ و$v_{y,n+1} = v_{y,n} + a_{y,n}\Delta t$.

لنتابع الإطلاق مدة $T = 2$ ثانية، قبل أن يصل المقذوف إلى الأرض. تخزّن أربعة متجهات متنامية المركبتين الأفقية والعمودية للموضع والسرعة. يستخدم الرسم العينات المخزّنة؛ فلا نحتاج إلى التكامل مرة أخرى لرسمها. ونحسب القطع المكافئ للمقارنة من الموضع والسرعة الابتدائيين.

تجمع الكتلة المزوّدة `let ... end` المحاكاة ورسمها في مساحة عمل محلية واحدة. شغّلوا الكتلة كاملة معًا. سنعود إلى المتغيرات المحلية عند تقديم الدوال؛ أما الآن فيجعل هذا الغلاف المثال العددي يعمل بصورة متسقة في السكربت والدفتر. وتبقى أمثلة `begin ... end` السابقة تمارين للاستخدام التفاعلي. ويعطي `round(Int, T / dt)` عدد الخطوات الصحيح، وهنا $T/\Delta t = 20$ بالضبط.

```
using Plots
let
    trajectory_x = Float64[]
    trajectory_y = Float64[]
    velocities_x = Float64[]
    velocities_y = Float64[]
    g = 9.81;         # gravity in m/s^2
    speed = 20.0;     # initial speed in m/s
    theta = pi / 4;  # initial angle in radians
    dt = 0.1;         # time step in seconds
    T = 2.0           # total time in seconds
    r = [0.0, 0.0]   # m
    v = [speed * cos(theta), speed * sin(theta)]   # m/s
    a = [0.0, -g]    # m/s^2
    N = round(Int, T / dt)  # number of time steps
    trajectory_x = [r[1]]
    trajectory_y = [r[2]]
    velocities_x = [v[1]]
    velocities_y = [v[2]]
    for step in 1:N
        r = r + v * dt
        v = v + a * dt
        push!(trajectory_x, r[1])
        push!(trajectory_y, r[2])
        push!(velocities_x, v[1])
        push!(velocities_y, v[2])
    end
    println("Euler final position: ", round(r[1]; digits=4), ", ", round
(r[2]; digits=4))
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    p = plot(title="Projectile Motion: Euler vs Exact", xlabel="x (m)",
ylabel="y (m)", frame=:box)
    p = plot!(p, trajectory_x, trajectory_y, label="Euler", lw=2)
    p = scatter!(p, trajectory_x, trajectory_y, label=:none, lw=2)
    exact_trajectory_x = Float64[]
    exact_trajectory_y = Float64[]
    ts = 0:dt:T
    for t in ts
        push!(exact_trajectory_x, r0[1] + v0[1] * t + 0.5 * a[1] * t^2)
        push!(exact_trajectory_y, r0[2] + v0[2] * t + 0.5 * a[2] * t^2)
    end
    p = plot!(p, exact_trajectory_x, exact_trajectory_y, label="Exact",
lw=2, ls=:dash)
    display(p)
end
```

```
Euler final position: 28.2843, 9.6453
```



تبيّن العلامات عينات أويلر المخزّنة، ويبيّن الخط المتقطع المسار الدقيق تحت جاذبية ثابتة. عند $t = T$ يكون ارتفاع أويلر 9.6453 م، مقابل الارتفاع الدقيق 8.6643 م. والفرق هو $gT\Delta t/2 = 0.981$ م. والخطوط الواصلة بين العينات دليل بصري. يسأل المثال التالي كيف يتغير هذا الفرق عند تصغير الخطوة الزمنية.

3.6 التنقيح: المقارنة مع جواب معلوم

بالنسبة لموضع ابتدائي $\mathbf{r}_0$ وسرعة ابتدائية $\mathbf{v}_0$، الحركة الدقيقة تحت تسارع ثابت هي

$$\mathbf{r}_{\text{exact}}(t) = \mathbf{r}_0 + \mathbf{v}_0 t + \tfrac{1}{2}\mathbf{a}t^2.$$

لنشّغل التكامل نفسه لعدة خطوات زمنية، مع إبقاء $T/\Delta t$ عددًا صحيحًا:

```
using LinearAlgebra
let
    g = 9.81; speed = 20.0; theta = pi / 4; T = 2.0
    r0 = [0.0, 0.0]
    v0 = [speed * cos(theta), speed * sin(theta)]
    a = [0.0, -g]
    for dt in (0.2, 0.1, 0.05, 0.02)
        r = copy(r0); v = copy(v0)
        N = round(Int, T / dt)
        for step in 1:N
            r = r + v * dt
            v = v + a * dt
        end
        exact = r0 + v0 * T + 0.5 * a * T^2
        println("dt = $dt, error = ", round(norm(r - exact); digits=4),
" m")
    end
end
```

```
dt = 0.2, error = 1.962 m
dt = 0.1, error = 0.981 m
dt = 0.05, error = 0.4905 m
dt = 0.02, error = 0.1962 m
```

تنصيف Δt ينصّف خطأ الموضع. وفي تسارع ثابت وأويلر صريح، تكون السرعة عند الأزمنة المتقطعة دقيقة حتى تقريب الفاصلة العائمة، بينما يحمل الموضع خطأً منهجيًا:

$$\mathbf{r}_N - \mathbf{r}_{\text{exact}}(T) = -\tfrac{1}{2}\mathbf{a}\,T\Delta t.$$

في الجاذبية الهابطة، يقع المسار العددي فوق المسار الدقيق عند زمن ثابت، ومقدار الخطأ هو $gT\Delta t/2$، بالضبط القيم المطبوعة أعلاه.

لماذا يتدرج الخطأ مع Δt؟ يحذف كل تحديث موضع لأويلر الحد $\frac{1}{2}a(\Delta t)^2$، لذا يتناسب الخطأ الموضعي المحلي مع $(\Delta t)^2$. ويوجد $N = T/\Delta t$ خطوة. وفي هذا المثال ذي التسارع الثابت، تتراكم المساهمات المحذوفة بشكل متماسك، معطية خطأً عامًا يتناسب مع $N(\Delta t)^2 = T\Delta t$. وهذا هو التقارب من الرتبة الأولى. والمحاكاة ليست الفيزياء الدقيقة، لكن الخطأ يتقلص بشكل متوقع، ويمكننا رؤيته يتقلص.

سؤال ختامي: ماذا يحدث للخطأ إذا جعلنا الخطوة الزمنية أصغر بعشر مرات؟ اختبروا ذلك عدديًا قبل الشرح.

تكامل من رتبة أعلى: نظرة أولى (اختياري)

خطأ الرتبة الأولى لأويلر ليس نهاية القصة. فكرة أفضل: خذ نصف خطوة، وقدّر المشتقة عند نقطة المنتصف تلك، واستخدمها لاتخاذ الخطوة الكاملة:

$$k_1 = f(y_n), \qquad k_2 = f(y_n + \tfrac{1}{2}\Delta t\, k_1), \qquad y_{n+1} = y_n + \Delta t\, k_2.$$

الطريقة التي تستخدم مشتقة واحدة في الفترة الوسطى هي طريقة من الرتبة الثانية؛ وفي تسارع ثابت تكون دقيقة بالصدفة. وأخذ عينات إضافية من المشتقة في مواضع ذكية يعطي رتبًا أعلى، حتى طريقة Runge–Kutta الكلاسيكية من الرتبة الرابعة ("RK4"). هذه الصيغ خلفية اختيارية، وليست شيئًا نحتاج إلى كتابته في هذا المقرر.

3.7 تمرين: أول زمن تبريد مرصود

نستخدم نموذج تبريد نيوتن، $\dot{T} = -k(T - T_{\text{room}})$، لجسم ذي حرارة داخلية منتظمة تقريبًا في بيئة ثابتة. والمعدل $k > 0$ بـ s^{-1}، ويمكن قياس فروق الحرارة بالكلفن أو بالدرجات المئوية. ونفترض أن فقد الحرارة يتناسب تقريبًا مع ذلك الفرق، لا أن النموذج يغطي التبخر أو تأثيرات إشعاعية كبيرة.

توفر دفتر التمرين التحديث الفيزيائي. ونحدد التهيئة والشرط والتحديث والعدّاد، ثم نستخدم حلقة `while` لإيجاد أول عينة عند العتبة أو أدنى منها. والمساواة تُعدّ وصولًا إلى العتبة. ونتنبأ بما يحدث إذا كانت العينة الابتدائية مؤهلة أصلًا، ولماذا لا يمكن بلوغ عتبة أدنى من حرارة الغرفة بتبريد رتيب من الأعلى. وبعد مناقشة 20 دقيقة نبدأ واجب المطرقة، الذي تتوقف حلقته عند أول عينة تحت الأرض.

3.8 الواجب الرئيسي: رمي مطرقة مع مقاومة هواء

نحاكي رمي مطرقة ثنائي الأبعاد من الأصل مع مقاومة هواء تربيعية، ونخزّن تاريخها المرصود. النموذج لكل وحدة كتلة هو

$$\dot{x} = v_x, \quad \dot{y} = v_y, \quad \dot{v}_x = -b v v_x, \quad \dot{v}_y = -g - b v v_y, \qquad v = \sqrt{v_x^2 + v_y^2},$$

حيث $g = 9.81\ \text{m/s}^2$ ومعامل السحب $b \approx 0.001\ \text{m}^{-1}$. يقرأ البرنامج السرعة الابتدائية وزاوية الرمي بالدرجات والزمن الكلي T والخطوة الزمنية Δt وb، ثم يقدم الحالة بأويلر صريح ويخزّن تواريخ متطابقة لـ x, y, v_x, v_y. ويتوقف عند أول عينة مقترحة بشرط $y < 0$، أو بعد T، ويطبع المسافة الأفقية المقطوعة.

نرسم أولًا المسار المتوقع ونحدد الوحدات والإشارات. ثم نبني المتجهات والحلقة، مع الحرص على تحديث الموضع من السرعة القديمة. ونفحص التحديث الأول يدويًا ونتحقق من أن كل متجه ينمو بعنصر واحد في كل خطوة، وأن السرعة الأفقية تتحلل عبر السحب فقط، وأن التسارع العمودي يشمل الجاذبية وحد السحب معًا. ونشغّل البرنامج بالمثال المعطى، ونقارن المسافة المطبوعة بتقدير محسوب يدويًا، وننقّح Δt لنرى كيف تتحرك المسافة المرصودة. ونحفظ رسم المسار كأثر. ويجب أن يذكر التقرير القاعدة التي أوقفت الحلقة: عبور الأرض أو نهاية نافذة المحاكاة. والامتداد الاختياري في الورقة، الذي يختبر ما إذا كانت 45° ما زالت تعظم المدى، غير مطلوب. ويبقى تعريف الدوال مادة الأسبوع الخامس.

3.9 مكافأة اختيارية: ثلاث صيغ تكامل

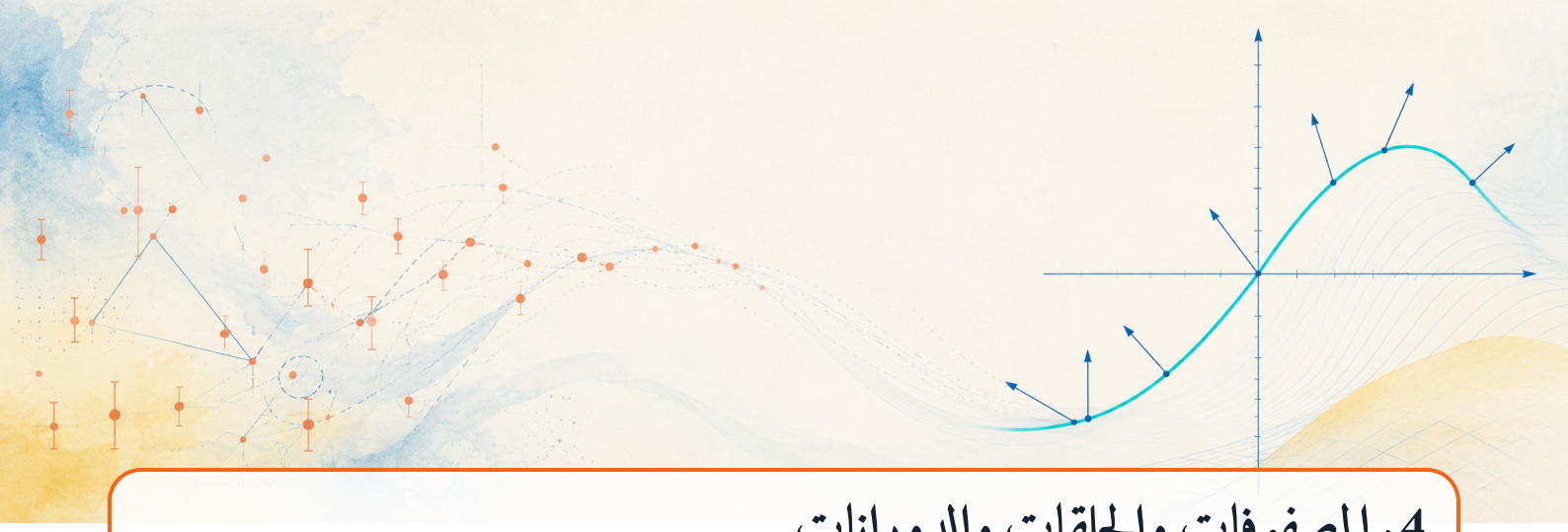
هذه المكافأة اختيارية، وليست شرطًا مسبقًا لعمل لاحق، وليست مُختبرة. وهي تساوي **ثلث نقطة في الدرجة النهائية.**

نقارن أويلر الأمامي وأويلر-كرومر وصيغة velocity-Verlet (نقطة المنتصف) على سقوط حر بتتبع الطاقة الميكانيكية لكل وحدة كتلة. ونقرأ الزمن النهائي والخطوة الزمنية والارتفاع الابتدائي والسرعة الابتدائية، ونشغّل الصيغ الثلاث على الشبكة نفسها، ونطبع الطاقات النوعية النهائية الثلاث. ثم نرسم الطاقة النوعية مقابل الزمن للصيغ الثلاث مع القيمة المحفوظة الدقيقة. وسؤالان نظريان قصيران يطلبان اشتقاق تغير الطاقة في كل خطوة لكل صيغة وقول أي صيغة أفضل ولماذا.

3.10 استكشاف إضافي: من مشية واحدة إلى انتشار

يمكن لمسار واحد أن ينجرف عائدًا نحو أصله. والانتشار عبارة عن مجموعة: بالنسبة لخطوات وحدة مستقلة وغير منحازة، يحقق الإزاحة المربعة المتوسطة $\langle r_n^2 \rangle = n$ بوحدات الشبكة، لأن حدود التقاطع تتلاشى في المتوسط. وإذا استغرقت الخطوة الواحدة τ وكان طولها ℓ، فإن معامل الانتشار ثنائي الأبعاد المقابل هو $D = \ell^2/(4\tau)$.

باستخدام وصفة مجموعة مُعطاة، نقارن مجموعة صغيرة وأخرى أكبر من المشيات، ونرسم الإزاحة المربعة المتوسطة مقابل رقم الخطوة، وندرس التقلبات حول الخط المستقيم المتوقع. ونشرح لماذا مربع متوسط الإزاحة كمية مختلفة ويمكن أن يظل قريبًا من الصفر. ونسجّل إعداد المولّد وحجم المجموعة مع الشكل. هذا الاستكشاف اختياري وغير مُختبر؛ والآلية المجموعية المُعطاة ليست شرطًا مسبقًا للعمل الأساسي.



4. المصفوفات والحلقات والدورانات

تُرتَّب كميات فيزيائية كثيرة في جداول من الأعداد بدلًا من أن تكون أعدادًا مفردة. تخزّن المصفوفة مثل هذا الجدول، ويُعد الدوران من أهم التحويلات التي يمكننا تطبيقها عليه. سنتعلم قراءة المصفوفة وتحديد أبعادها، ثم نستخدم الحلقات لملء المصفوفات ودمجها، وأخيرًا ندير الأشكال والمتجهات بالمصفوفات. وعلى امتداد الفصل سنكتب العمليات الحسابية بأنفسنا، عنصرًا عنصرًا، لنتبين ما تنفذه العمليات الجاهزة فعلًا. سنتعلم كتابة دوالنا في الأسبوع الخامس؛ أما هنا فنكتب كل شيء باستخدام الحلقات ومساحات العمل `let`.

4.1 كتابة المصفوفات وأبعادها

المصفوفة جدول مستطيل من الأعداد المرتبة في صفوف وأعمدة. عند كتابة مصفوفة في Julia، تفصل المسافات عناصر الصف الواحد، وتفصل الفاصلة المنقوطة بين الصفوف:

```
A = [1 2 3; 4 5 6]      # a 2-by-3 matrix: 2 rows, 3 columns
println(A)
println(size(A))         # (2, 3)
println(size(A, 1))      # 2   (number of rows)
println(size(A, 2))      # 3   (number of columns)
println(length(A))       # 6   (total number of entries)
```

```
[1 2 3; 4 5 6]
(2, 3)
2
3
6
```

للمصفوفة A صفّان وثلاثة أعمدة، ولذلك فإن أبعادها هي «2 في 3». تعيد `size(A)` زوج الأبعاد، وتعطي `size(A, 1)` عدد الصفوف، و`size(A, 2)` عدد الأعمدة. والعدد الكلي للعناصر هو حاصل ضرب البعدين، وتعيده `length`. غالبًا ما نتحقق من الأبعاد أولًا، إذ لا يمكن دمج مصفوفتين إلا عندما تكون أبعادهما متوافقة.

4.1.1 فهرسة المصفوفة

نحدد عنصرًا مفردًا بفهرس صفه أولًا، ثم فهرس عموده:

```
println(A[2, 3])         # 6
println(A[1, :])         # [1 2 3]   the first row (a 1-by-3 matrix)
```

```
println(A[:, 2])          # [2, 5]    the second column (a vector)
```

```
6
[1 2 3]
[2, 5]
```

`A[2, 3]` هو العنصر الواقع في الصف 2 والعمود 3. وتحدد النقطتان الرأسيتان جميع العناصر على ذلك المحور: `A[1, :]` هو الصف الأول كاملًا، و`A[:, 2]` هو العمود الثاني كاملًا. لاحظوا اختلاف شكل الناتج: يحافظ الصف على ترتيبه الأفقي، بينما يظهر العمود متجهًا عاديًا. وتظل المصفوفة ذات العمود الواحد مصفوفة، وهذا ما تعكسه طريقة طباعة Julia لها.

الصفوف أولًا، ثم الأعمدة

يسمي كل تعبير فهرسة الصف قبل العمود: `A[i, j]` يعني الصف i والعمود j. أما النقطتان الرأسيتان `:` فتعنيان «كل الفهارس على هذا المحور». ويعيد اقتطاع صف أو عمود نتيجةً بالاتجاه الصحيح، فيمكننا استخدامها مباشرةً في جداء أو رسم.

4.2 حلقات while: نكرّر ما دام الشرط صحيحًا

قد لا نعرف مسبقًا عدد مرات تكرار بعض العمليات. تكرر حلقة `while` ما دام شرطها صحيحًا، وتختبره قبل كل تكرار. وعلى خلاف حلقة `for`، لا تحمل هذه الحلقة عدّادًا خاصًا بها؛ لذا ينبغي أن يغيّر أمر داخل جسمها قيمة الشرط، وإلا فلن تنتهي الحلقة.

```
let
    n = 0
    while n < 5
        n += 1
        println(n)
    end
end                     # prints 1, 2, 3, 4, 5
```

```
1
2
3
4
5
```

هنا يكون `n` هو العدّاد، و`n += 1` هو التحديث، أما `n < 5` فهو الشرط. تختبر الحلقة الشرط، ثم تنفذ الجسم وتختبره من جديد، وتتوقف فور إخفاق الاختبار.

4.2.1 إدخال قيمة حارسة: متوسط الصف

من الاستخدامات الشائعة لحلقة `while` قراءة المدخلات حتى تظهر قيمة خاصة تُسمى قيمة حارسة. نقرأ أدناه الدرجات إلى أن يُدخل المستخدم -1. تحدد هذه القيمة نهاية الإدخال، ولا تُعد درجةً.

```
let
    sum_grades = 0      # total of the grades
    n_grades = 0        # how many grades were entered
    print("Enter grade (-1 to stop): ")
    grade = parse(Int, readline())
    while grade != -1
```

```
        sum_grades += grade
        n_grades += 1
        print("Enter grade (-1 to stop): ")
        grade = parse(Int, readline())
    end
    if n_grades != 0
        avg = sum_grades / n_grades
        println("Class average is ", round(avg; digits = 2),
                "; number of grades is ", n_grades)
    else
        println("No grades were entered")
    end
end
```

تحدث القراءة الأولى قبل بدء الحلقة، وينتهي كل تكرار بقراءة جديدة. فإذا أدخل المستخدم -1 مباشرةً، فلن تنفذ الحلقة، وسنذكر عدم إدخال أي درجات بدلًا من القسمة على صفر.

القيمة الحارسة بيانات وليست خطأً

تحدد القيمة الحارسة نهاية سلسلة المدخلات، ولذلك ينبغي تمييزها عن البيانات الفعلية. تصلح -1 هنا لأن الدرجات تقع بين 0 و100. ويفحص شرط `while` وجود القيمة الحارسة، ولا يعدّ التكرارات.

4.2.2 التحقق من صحة المدخلات

وهناك نمط شائع آخر يواصل طلب الإدخال حتى تكون الإجابة مقبولة. تستمر الحلقة ما دام الإدخال غير صالح؛ لذا يكون شرطها نفي الحالة التي نريدها:

```
let
    print("Please enter a positive number 20..100: ")
    flush(stdout) # Make sure the prompt appears before waiting for
input.
    number = parse(Int, readline())
    while number <= 20 || number >= 100
        print("A positive number 20..100, try again: ")
        number = parse(Int, readline())
    end
    println("Your positive number is ", number)
end
```

يُقرأ الشرط `number <= 20 || number >= 100` «ما زالت القيمة غير صالحة». ولا يوقف الحلقة إلا عدد يقع بين 20 و100 دون الطرفين. انتبهوا إلى الحدّين: باستخدام => و=< تُرفض القيمتان 20 و100 نفسيهما. ولقبولهما نستخدم > و<. تدفع الدالة `flush(stdout)` الموجه إلى الشاشة قبل انتظار البرنامج للإدخال، وهذا مهم في الطرفيات التي تخزن المخرجات مؤقتًا.

4.2.3 لعبة تخمين

تجمع لعبة التخمين الفكرتين. يختار الحاسوب عددًا، ونواصل قراءة التخمينات مع تلقي تلميح في كل مرة، حتى يصيب أحدها العدد:

```
let
    number = rand(1:16)
    print("please guess my number between 1 and 16 ")
    flush(stdout) #
```

```
    guess = parse(Int, readline())
    while guess != number
        if number > guess
            print("My number is bigger, please try again ")
        else
            print("My number is smaller, please try again ")
        end
        flush(stdout)
        guess = parse(Int, readline())
    end
    println("You guessed it! The number is ", number)
end
```

قد لا تنفذ الحلقة مطلقًا إذا كان التخمين الأول صحيحًا، وقد تتكرر مرات عديدة. يعطي كل تخمين خاطئ تلميحًا يضيّق مجال البحث؛ لذلك نادرًا ما يحتاج اللاعب إلى أكثر من بضع محاولات من أصل ست عشرة إمكانية.

4.2.4 توقيت حلقة while: المتسلسلة التوافقية

تنمو المتسلسلة التوافقية $1 + \frac{1}{2} + \frac{1}{3} + \dots$ ببطء شديد، وبمعدل يقارب $\ln n$. لذا يتطلب بلوغ مجموع متواضع إضافة حدود كثيرة، وهذا يتيح لنا تكوين فكرة عن سرعة تنفيذ Julia. نضيف الحدود حتى يتجاوز المجموع حدًا معينًا، ونقيس زمن الحلقة:

```
let
    print("Enter positive limit: ")
    flush(stdout)
    limit = parse(Float64, readline())

    max_terms = 100_000_000   # safety cap on the number of terms
    t1 = time_ns()
    series_sum = 0.0
    n = 0
    while series_sum <= limit
        n += 1
        series_sum += 1.0 / n
        if n >= max_terms
            println("Limit ", limit, " too large: stopped after ", n,
                    " terms without exceeding it.")
            break
        end
    end
    t2 = time_ns()

    println("Number of terms needed: ", n)
    println("Elapsed seconds: ", round((t2 - t1) / 1.0e9; digits = 3))
    # For limit 10: 12367 terms (instant).
    # For limit 20: about 272,400,600 terms — more than the cap, so the
loop
    # stops at 100,000,000 terms and prints that the limit was too large.

end
```

عندما يكون الحد 10، نحتاج إلى 12367 حدًا، وينتهي الحساب فورًا. أما الحد 20 فيتطلب نحو 272 مليون حد، متجاوزًا سقف الأمان البالغ مئة مليون؛ لذا تتوقف الحلقة مبكرًا وتنبّه إلى أن الحد كبير جدًا. إن شرط `if n >= max_terms` مع `break` عادة نافعة. فعندما يُحتمل أن تستمر الحلقة طويلًا، نضع لها حدًا ونحدد ما ينبغي فعله عند بلوغه. ونحصل على الأزمنة باستخدام `time_ns()`, التي تقرأ ساعة بالنانوثانية؛ لذا تعطي قسمة الفرق على 10^9 الزمن بالثواني.

التوقف جزء من التصميم

تحتاج حلقة `while` إلى تهيئة، وشرط يمكن أن يصبح خاطئًا، وتحديث يغيّر الشرط. يؤدي غياب أي واحد منها إلى حلقة لا نهائية. ويحمينا تحديد عدد أقصى للتكرارات من الأخطاء، ويحوّل البحث المفتوح إلى بحث محدود.

تعمل هذه الأمثلة التفاعلية بموثوقية أكبر عند تشغيلها ملفاتٍ مستقلة بدل تنفيذها خليةً خليةً في محرر، لذلك جمعنا أمثلة الإدخال الأربعة أيضًا في المجلد `week04_readline_examples` داخل examples. ومن REPL، بعد الانتقال إلى مجلد examples، يمكننا تنفيذ `include("week04_readline_examples/class_average_sentinel.jl")`، وبالطريقة نفسها نشغّل الملفات الثلاثة الأخرى.

4.3 جداء المصفوفة بالمتجه وبالمصفوفة

تؤثر المصفوفة في متجه أو مصفوفة أخرى، وتكتب Julia هذه الجداءات باستخدام الرمز المعتاد *:

```
M = [1 2; 3 4]
v = [5, 6]
println(M * v)          # [17, 39]   matrix * vector: linear
combination of columns
println(M .* M)         # [1 4; 9 16]  elementwise multiplication (
dotted)
println(M * M)          # [7 10; 15 22]  true matrix product
```

```
[17, 39]
[1 4; 9 16]
[7 10; 15 22]
```

السطر الأول جداء مصفوفة بمتجه: تُدمج أعمدة `M` مع عناصر `v` لإنتاج متجه جديد. أما السطر الثاني فهو ضرب عنصرًا بعنصر، ونكتبه بنقطة؛ إذ يُضرب كل عنصر من `M` في العنصر المناظر له في الموضع نفسه. ويمثل السطر الثالث جداء المصفوفتين المعتاد، وسنعيد بناءه يدويًا بالحلقات بعد قليل.

ضرب المصفوفات ليس ضربًا عنصرًا بعنصر

العنصر الواقع في الصف i والعمود j من الجداء `A * B` هو الجداء السلمي للصف i من `A` والعمود j من `B`:

$$(AB)_{ij} = \sum_{k} A_{ik} B_{kj}.$$

كي يكون هذا المجموع معرّفًا، ينبغي أن يتساوى البعدان الداخليان: عدد أعمدة `A` يجب أن يساوي عدد صفوف `B`. أما الضرب عنصرًا بعنصر `A .* B` فيقرن العناصر المتناظرة، ويتطلب تطابق الأبعاد. إنها عملية مختلفة عن جداء المصفوفات.

4.4 الحلقات المتداخلة: جدول الضرب

تكرر الحلقة كلّةً من التعليمات. وتضع الحلقة المتداخلة حلقةً داخل أخرى، فتكتمل الحلقة الداخلية لكل قيمة تأخذها الحلقة الخارجية. وتناسب الحلقات المتداخلة ملء المصفوفة صفًا بعد صف. وأبسط مثال جدول ضرب 10×10، حيث يخزن كل عنصر `table[i, j]` القيمة `i * j`:

```
let
    table = zeros(Int, 10, 10)
    for i in 1:10
```

```
        for j in 1:10
            table[i, j] = i * j
        end
    end
    println(table[3, 4])       # 12
    println(table[10, 10])     # 100
    println(table[3, 1:5])     # [3, 6, 9, 12, 15]   (row 3)
    println(table)
end
```

```
12
100
[3, 6, 9, 12, 15]
```

تتبّعوا ترتيب تنفيذ الحلقتين. تعيّن الخارجية $i = 1$، وتمسح الداخلية القيم $j = 1, \ldots, 10$ لتملأ الصف الأول كاملًا. ثم تنتقل الخارجية إلى $i = 2$، فتملأ الداخلية الصف الثاني. تتغير قيمة الحلقة الداخلية أسرع، وهذا يطابق قراءتنا للجدول: من اليسار إلى اليمين، صفًا بعد صف. وتطبع `println(table)` الأخيرة في المثال المصفوفة كاملة بترميز Julia المختصر: سطر طويل جدًا تفصل فيه الفواصل المنقوطة بين الصفوف. الناتج صحيح، لكن قراءته صعبة للغاية.

طباعة مقروءة للمصفوفة (اختياري)

تكفينا طباعة المصفوفة العادية، لكن قراءة المصفوفات العريضة تصبح صعبة. إذا أردتم محاذاة الأعمدة، توفر الحزمة PrettyTables.jl الدالة `pretty_table(table)` التي ترسم جدولًا ذا حدود. هذه الحزمة ليست ضمن بيئة المقرر ولا نحتاج إليها في أي تمرين، كما أن تثبيت حزم إضافية خارج نطاق المقرر؛ لذلك سنكتفي بطباعة المصفوفة مباشرةً.

4.5 الحلقات المتداخلة: ضرب المصفوفات

يمكننا الآن إعادة إنشاء الجداء بأنفسنا. كل عنصر في الناتج هو جداء سلمي لصف بعمود؛ لذا تزور حلقتان متداخلتان كل عناصر الناتج، بينما تجمع حلقة داخلية حدود المجموع على البعد المشترك. وإذا طال السطر، نوزع المصفوفة على أسطر عدة ونحاذي صفوفها كي تبقى بنيتها واضحة:

```
let
    A = [1.0 2.0 3.0;
         4.0 5.0 6.0]   # 2-by-3
    B = [1.0 0.0 2.0 1.0;
         0.0 1.0 1.0 0.0;
         2.0 1.0 0.0 1.0]   # 3-by-4
    m, k = size(A)
    k2, n = size(B)
    @assert k == k2 "inner dimensions must match"

    C = zeros(m, n)
    for i in 1:m          # output row
        for j in 1:n      # output column
            for r in 1:k  # the shared inner dimension
                C[i, j] += A[i, r] * B[r, j]
            end
        end
    end
```

```
    println(C)             # 7.0  5.0  4.0  4.0  /  16.0  11.0  13.0
10.0
    println(A * B)         # same numbers, built by Julia's own matrix
product
    println(isapprox(C, A * B))   # true
end
```

```
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
[7.0 5.0 4.0 4.0; 16.0 11.0 13.0 10.0]
true
```

هذه ثلاث حلقات متداخلة. تختار الحلقتان الخارجيتان عنصر الناتج (i, j)، وتضيف الحلقة الأعمق الحدود $A[i,1]B[1,j] + A[i,2]B[2,j] + \ldots$ حتى تستنفد البعد المشترك. لا يغير توزيع المصفوفات على أسطر عدة قيمها؛ بل يُظهر بنية الصفوف، ويذكّرنا التعليق الختامي بأبعاد المصفوفة. ويعمل السطر `@assert` كحارس؛ فإذا اختلف البعدان الداخليان توقف البرنامج برسالة واضحة بدل إجراء حساب غير ذي معنى. ويؤكد تطابق الناتج مع `A * B` أننا فهمنا القاعدة.

4.6 التحقق من تماثل المصفوفة

تكون المصفوفة متماثلة إذا تحقق $A_{ij} = A_{ji}$ لكل زوج من الفهارس؛ أي إن عنصر الصف i والعمود j يساوي عنصر الصف j والعمود i. ويمكننا فحص ذلك بحلقة متداخلة. يكفي تفقد المثلث العلوي، لأن عناصر القطر تساوي نفسها دائمًا:

```
let
    M = [1 2 3;
         2 5 6;
         3 6 9]    # symmetric by construction
    symmetric = true
    for i in 1:size(M, 1)
        for j in i:size(M, 2)    # j starts at i, so we visit the upper
triangle
            if M[i, j] != M[j, i]
                symmetric = false
            end
        end
    end
    println("Matrix M is symmetric: $symmetric")           # true

    N = [1 2 3;
         4 5 6;
         7 8 9]   # not symmetric
    symmetric = true
    for i in 1:size(N, 1), j in i:size(N, 2)
        if N[i, j] != N[j, i]
            symmetric = false
        end
    end
    println("Matrix N is symmetric: $symmetric")           # false
end
```

```
Matrix M is symmetric: true
Matrix N is symmetric: false
```

كُتبت المصفوفتان صفًا في كل سطر، فيظهر تماثل `M` من النظرة الأولى: فالأعداد في الصف هي نفسها الأعداد في العمود المناظر. وتفحص الحلقة هذه الخاصية مباشرةً. وعندما نبدأ الحلقة الداخلية عند `j in i:…` نتجاوز المثلث السفلي والقطر، فنقلل العمل

إلى نحو النصف. وتستخدم الحلقة الثانية الصيغة المختصرة `for i in …, j in …`، وهي مكافئة لحلقتين متداخلتين.

لدينا طريقة مختصرة أيضًا. يبدّل المنقول `M'` الصفوف والأعمدة، ولذلك تكون المصفوفة متماثلة بالضبط عندما `M == M'`. لكن تطبيق هذا الاختبار على مصفوفة شبه متماثلة بعناصر فاصلة عائمة قد يفشل، لأن التقريب يغيّر أحد الزوجين بنحو 10^{-14}. في هذه الحالة تكون `isapprox` مع سماحية هي الأداة المناسبة، كما يبيّن المقطع الأخير.

```
# A shortcut: M' is the transpose. Comparing M with its transpose is the
 same check.
M = [1 2 3; 2 5 6; 3 6 9]
N = [1 2 3; 4 5 6; 7 8 9]
println(M == M')         # true
println(N == N')         # false
```

```
true
false
```

ونجري المقارنة نفسها بأعداد فاصلة عائمة، مع سماحية:

```
let
    P = [1.0 2.0;
         2.0 1.0]
    Q = P + [0.0 1e-14;
             0.0 0.0]   # nearly symmetric, but not exactly
    println(P == P')                # true
    println(Q == Q')                # false  (machine noise breaks exact
 equality)
    println(isapprox(Q, Q'; atol = 1e-12))    # true
    println(maximum(abs.(Q - Q')))            # 1.0e-14
end
```

```
true
false
true
1.0e-14
```

لا تختلف `Q` عن منقولها إلا بمقدار 10^{-14}، وهو مقدار ضئيل فيزيائيًا، لكنه يكفي لجعل `==` يفشل. وتقبلها `isapprox` باستخدام `atol = 1e-12`. وتعرض `maximum(abs.(Q - Q'))` مقدار الفرق مباشرةً، والنقطة في `abs.` توزّع عملية القيمة المطلقة على جميع العناصر.

4.7 مصفوفات الدوران في بعدين

مصفوفة تدير متجهًا عكس اتجاه عقارب الساعة بزاوية θ هي

$$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}.$$

عند استخدام متجهات عمودية، تكون النقطة بعد الدوران $R\mathbf{p}$. وينبغي أن يحوّل تدوير المتجه $(1,0)$ بزاوية $90°$ إياه إلى $(0,1)$:

```
using LinearAlgebra
let
    θ = π / 2
    R = [cosθ() -sinθ();
         sinθ() cosθ()]
```

```
    println(R)                          # 0.0  -1.0  /  1.0  0.0
    p = [1.0, 0.0]
    println(R * p)                      # [0.0, 1.0]   (a quarter turn)
end
```

```
[0.0 -1.0; 1.0 0.0]
[0.0, 1.0]
```

كتبنا الصف الثاني في سطر مستقل كي يظهر الصفان بوضوح. تعرض Julia العنصر 6.12e-17 بدلًا من الصفر تمامًا، لأن $\cos(\pi/2)$ لا يمكن تمثيله بدقة تامة بأعداد الفاصلة العائمة؛ وتُقرّب الطباعة القيمة إلى 0.0. وهذا الباقي الضئيل هو سبب مقارنة الكميات في المقطع التالي بسماحية بدلًا من استخدام ==.

4.7.1 التحقق من أن المصفوفة تمثل دورانًا فعلًا

تتميز مصفوفة الدوران بخاصيتين تفرّقانها عن المصفوفة العامة. فهي تحفظ المسافات، ولذلك لا تتغير المسافة بين نقطتين. كما أن منقولها معكوسها، $R'R = I$، وتحفظ الاتجاه، فيكون محددها 1. ويمكننا اختبار الخاصيتين الأوليين مباشرةً:

```
let
    p = [1.0, 2.0]
    q = [-0.5, 3.0]
    θ = π / 6
    R = [cosθ() -sinθ();
         sinθ() cosθ()]

    println(isapprox(R' * R, I; atol = 1e-14))   # true  (orthogonal)

    d_before = norm(p - q)          # distance before rotation
    d_after = norm(R * p - R * q)  # distance after rotation
    println(round(d_before; digits = 6))         # 1.802776
    println(round(d_after; digits = 6))          # 1.802776   (same
distance)
end
```

```
true
1.802776
1.802776
```

تعطي R' * R مصفوفة الوحدة بدقة تصل إلى 10^{-14}، وتساوي المسافة بين النقطتين بعد الدوران المسافة بينهما قبله. وهذان الاختباران يحسمان سريعًا ما إذا كان تحويل أنشأناه دورانًا بالفعل.

4.8 دوران مستطيل في المستوى

يعني تدوير شكل أن ندير كل رأس ثم نصل الرؤوس الجديدة بعضها ببعض. نخزن المستطيل في مصفوفة، بحيث يمثل كل صف أحد أركانه الأربعة. ندير المستطيل أدناه بزوايا $0°, 30°, 60°, 90°$، ونطبع بعد كل دوران موضع أحد أركانه:

```
let
    θs = (0.0, 30.0, 60.0, 90.0)
    rect = [0.0 0.0;
            2.0 0.0;
            2.0 1.0;
            0.0 1.0]
    for θ in θs
        R = [cosd(θ) -sind(θ);
```

```
             sind(θ) cosd(θ)]
        rotated = zeros(size(rect))  # same shape as rect
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]   # the corner
as a vector
            moved = R * corner                            # the rotated
corner
            rotated[vertex, 1] = moved[1]
            rotated[vertex, 2] = moved[2]
        end
        println(θ," = ", θ, "°  corner (2,0) -> [", round(rotated[2, 1];
digits = 3),
                ", ", round(rotated[2, 2]; digits = 3), "]")
    end
    # θ = 0.0°   corner (2,0) -> [2.0, 0.0]
    # θ = 30.0°  corner (2,0) -> [1.732, 1.0]
    # θ = 60.0°  corner (2,0) -> [1.0, 1.732]
    # θ = 90.0°  corner (2,0) -> [0.0, 2.0]
end
```

يمثل كل صف من rect ركنًا واحدًا، وقد كتبنا الأركان على أسطر منفصلة لتمييزها. تقرأ الحلقة الداخلية إحداثيي ركن واحد بفهرسين منفردين، rect[vertex, 1] وrect[vertex, 2]، وتجمعهما في متجه من عنصرين، ثم تضرب المصفوفة في هذا المتجه. وهذا هو جداء المصفوفة في المتجه نفسه من القسم السابق، مطبَّقًا مرة واحدة لكل ركن، فلا حاجة إلى معرفة شيء عن الشرائح. ويُكتب إحداثيا الناتج في الصف المقابل من rotated الذي هيأه zeros(size(rect)) بالأبعاد نفسها. يتحرك الركن $(2, 0)$ على دائرة نصف قطرها اثنان، ويمر بالنقطة $(1.732, 1.0)$ عند $30°$، ثم ينتهي على محور y عند $90°$؛ وهذا ما نتوقعه من ربع دورة. وتأخذ الدالتان cosd وsind الزوايا بالدرجات مباشرةً، فلا نحتاج إلى تحويل صريح.

يسهّل رسم النسخ معًا رؤية الدوران أكثر من تتبع الإحداثيات:

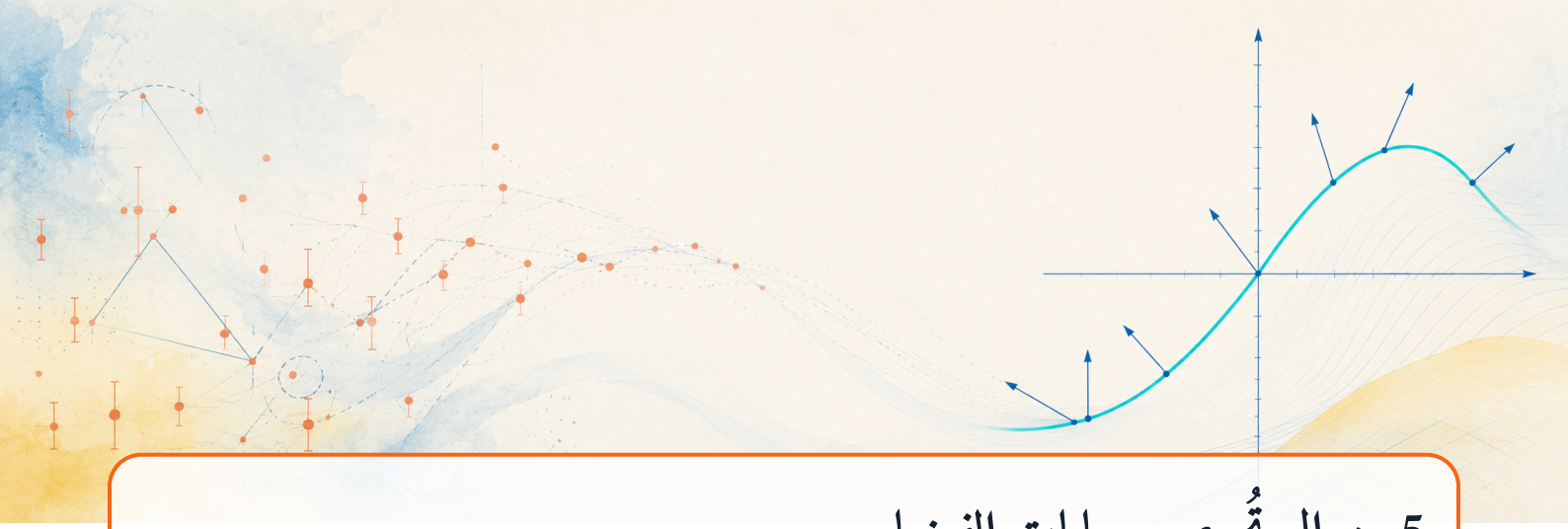
```
using Plots
let
    rect = [0.0 0.0; 2.0 0.0; 2.0 1.0; 0.0 1.0]

    p = plot(title = "Rectangle rotated in the xy plane",
             xlabel = "x", ylabel = "y",
             legend = false, aspect_ratio = :equal)

    for θ in 0:30:330
        R = [cosd(θ) -sind(θ); sind(θ) cosd(θ)]
        xs = Float64[]; ys = Float64[]
        for vertex in 1:4
            corner = [rect[vertex, 1], rect[vertex, 2]]
            moved = R * corner
            push!(xs, moved[1])
            push!(ys, moved[2])
        end
        # Close the polygon back to its first vertex.
        push!(xs, xs[1]); push!(ys, ys[1])
        plot!(p, xs, ys; lw = 2)
    end

    display(p)
end
```

ينبغي أن نلاحظ أمرين في هذا المقطع. تنتقل الحلقة الخارجية بالزوايا من $0°$ إلى $330°$ بزيادات مقدارها ثلاثون، فترسم مستطيلًا مدوّرًا في كل تكرار. وفي الداخل يتحول كل ركن إلى متجه من عنصرين، ويُدار بجداء المصفوفة في المتجه نفسه كما سبق، ويُجمع إحداثياه في xs وys باستخدام !push بدلًا من تعبير إنشاء قائمة، إذ نتمرن هنا على كتابة الحلقات صراحةً. ونضيف النقطة المحسوبة الأولى مرة أخرى لإغلاق المضلع؛ فبدون ذلك يغيب الضلع الأخير ويبدو الشكل مسارًا مفتوحًا.



5. دوال تُجري حسابات الفيزياء

أمضينا شهرًا نبني الحسابات باستخدام الحلقات والمتجهات والمصفوفات. وكانت كل مسألة جديدة تستدعي نسخ مقطع من الشيفرة وتغيير عدد أو عددين فيه. هذا الأسبوع سنجمع خطوات الحساب مرةً واحدة، ونمنحها اسمًا، ثم نستدعيها كلما احتجنا إليها. هذا ما تفعله الدالة، ومعها تبدأ برامجنا بتجاوز مرحلة النصوص البرمجية التي نكتبها لاستخدام واحد.

مثالنا الفيزيائي هو بندول نُطلقه من زاوية كبيرة. لا يمكن التعبير عن زمنه الدوري بدوال أولية، ولذلك نكامل معادلة حركته عدديًا. وتتيح لنا الدالة نفسها بعد ذلك أن نسأل كيف يعتمد الزمن الدوري على زاوية الإطلاق؛ وهو سؤال يستحق أن نرسم إجابته ونتأملها.

5.1 من حساب إلى دالة

لنبدأ بوسيط واحد. تأخذ الدالة القيم التي نمررها إليها، وتُسمى وسائط، وتتعامل معها محليًا، ثم تعيد إلينا النتيجة باستخدام `return`:

```
function double_number(x)
    return 2 * x
end

println(double_number(3))      # 6
```

```
6
```

يوجد الاسم `x` داخل الدالة فقط. وبعد أن تعيد الدالة نتيجتها، يمكننا استدعاؤها مجددًا بعدد مختلف، من دون أن يترك الاستدعاء السابق شيئًا وراءه. ولأن الحساب أصبح في موضع واحد، تستطيع الحلقة استخدامه مباشرةً:

```
for n in 1:5
    println(double_number(n))
end
```

```
2
4
6
8
10
```

من المفيد أن نعتاد هذا التقسيم للعمل: تحدد الحلقة أيّ القيم نستخدم، وتحدد الدالة ما نفعله بقيمة واحدة.

تعمل الدالة ذات الوسيطين بالطريقة نفسها. وتكون الأسماء داخل جسمها محليةً أيضًا:

```
function rectangle_area(width, height)
    area = width * height
    return area
end

println(rectangle_area(3, 4))   # 12
width = 100
area = -1
println(rectangle_area(3, 4))   # 12
println(width)                  # 100
println(area)                   # -1
```

```
12
12
100
-1
```

للدالة متغيراها المحليان `width` و`area`، أما الاسمان خارجها فيحتفظان بقيمتيهما. يستخدم الطولان هنا الوحدة الاختيارية نفسها، وتكون وحدة المساحة المعادة مربع تلك الوحدة.

يمكن كتابة الحساب القصير في سطر واحد

عندما يتكون الجسم من تعبير واحد، يمكن اختصار التعريف:

```
square(x) = x * x
println(square(5))       # 25
```

تمثل الصيغتان الدالة نفسها. استخدموا الصيغة الطويلة عندما يحتاج الجسم إلى عدة خطوات أو إلى تعليق.

5.2 النطاق: أين يوجد الاسم؟

يكتسب الاسم معناه داخل نطاق معين. ففي المستوى الأعلى من النص البرمجي أو في البيئة التفاعلية REPL، ينشئ الإسناد اسمًا عامًا. أما داخل الدالة، فينشئ اسمًا محليًا. ويظهر الفرق فور استخدام التهجئة نفسها للاسمين:

```
x = 10
println("Global x = ", x)              # Global x = 10

function scope_function_x()
    x = 20
    return x
end
println("Function's local x = ", scope_function_x())   # 20
println("After function, global x = ", x)              # 10
```

لم يؤثر الإسناد داخل الدالة في x العام. وهذا ما نريده عادةً: فلا تعيد الدالة كتابة بقية البرنامج من دون أن ننتبه.

تختلف كتلة `begin` عن ذلك. فهي تجمع التعليمات من دون إنشاء نطاق، ولذلك تغيّر الاسم العام نفسه عندما تقع في المستوى الأعلى:

```
begin
    x = 30
```

```
    println("Inside top-level begin, x = ", x)   # 30
end
println("After begin, global x = ", x)           # 30
```

أما كتلة `let` فتفتح نطاقًا محليًا:

```
let
    x = 40
    println("Inside let, local x = ", x)   # 40
end
println("After let, global x = ", x)       # 30
```

وعندما نضع `begin` داخل دالة، فإنها تشترك في نطاق تلك الدالة:

```
function scope_function_begin_x()
    x = 50
    begin
        x = 60
    end
    return x
end
println(scope_function_begin_x())   # 60
println(x)                          # 30
```

لا تجعل كتابة `begin` الكتلةَ عامةً؛ فهي تستخدم النطاق المحيط بها. ويتعلق النطاق المحلي بالأسماء، وليس بنسخ الكائنات: فقد يشير اسم محلي إلى المصفوفة نفسها التي يشير إليها اسم عام، كما سنرى في الأقسام التالية.

5.3 الوسائط الموضعية والمسمّاة وقيمها الافتراضية

تُطابق الوسائط الموضعية بحسب ترتيبها، ولذلك فالترتيب مهم:

```
function difference(a, b)
    return a - b
end
println(difference(10, 3))    # 7
println(difference(3, 10))    # -7
```

يمكن أن تكون للوسيط الموضعي قيمة افتراضية تُستخدم عندما لا يمرر المستدعي قيمةً له:

```
function shifted(x, amount = 1)
    return x + amount
end
println(shifted(5))       # 6
println(shifted(5, 10))   # 15
```

عند وجود عدة قيم افتراضية، تُستكمل القيم المحذوفة من اليمين؛ فلا يستطيع المستدعي تجاوز وسيط موضعي في الوسط:

```
function confusing(x, y = 10, z = 100)
    return x + y + z
end
confusing(3, 1)   # 3 + 1 + 100 = 104
confusing(3)      # 3 + 10 + 100 = 113
```

تُذكر أسماء الوسائط المسمّاة عند الاستدعاء. وتحدد الفاصلة المنقوطة في التعريف بداية هذه الوسائط:

```
function affine(x; scale = 1, offset = 0)
    return scale * x + offset
```

```
end
println(affine(3))                          # 3
println(affine(3; scale = 2, offset = 1))   # 7
println(affine(3; offset = 1, scale = 2))   # 7
```

لا يهم ترتيب الوسائط المسمّاة في Julia، ويمكن عند الاستدعاء استبدال الفاصلة المنقوطة بفاصلة عادية: فالتعبير `affine(3, offset = 1)` يعطي 4. وتلزم الفاصلة المنقوطة في التعريف لتحديد بداية الوسائط المسمّاة. استخدموا هذه الوسائط عندما يكون معنى عدد مجرد عند الاستدعاء ملتبسًا.

5.4 إعادة قيمة

تعيد الدالة التي لا تحتوي على `return` قيمة آخر تعبير فيها:

```
function cube(x)
    x^3
end
println(cube(3))      # 27
```

تؤدي `return` أيضًا إلى مغادرة الدالة فورًا، حتى من داخل حلقة. وبذلك نستطيع التوقف بمجرد معرفة الإجابة، والإشارة إلى غياب إجابة باستخدام `nothing`:

```
function first_positive(values)
    for value in values
        if value > 0
            return value
        end
    end
    return nothing
end
println(first_positive([-2, 0, 5, 9]))   # 5
println(first_positive([-2, 0]))         # nothing
```

القيمة `nothing` قيمة قائمة بذاتها، فهي ليست العدد صفرًا ولا سلسلة نصية فارغة. ويمكن اختبارها مباشرةً. وتحتوي Julia على قيم خاصة أخرى أيضًا:

```
a = nothing
if a == nothing
    println("a is nothing")
end
a = 1 / 0
println(a, "   is a > 0 ?  ", a > 0)   # Inf   is a > 0 ?  true
println(-1 / 0)                        # -Inf
println(0 / 0)                         # NaN
```

تتصرف `Inf` في المقارنات كعدد موجب بالغ الكبر، بينما تُخفق `NaN` في كل مقارنة، بما فيها `NaN == NaN`. ولهذا نستخدم `isfinite` عند فحص البيانات بدلًا من مقارنتها بالصفر.

وأخيرًا، إعادة القيمة وطباعتها عمليتان مختلفتان. فالدالة التي تطبع ترسل إجابتها إلى الشاشة، ولا تعيدها إلى المستدعي:

```
function print_square(x)
    println(x^2)
end
answer = print_square(4)   # prints 16
println(answer)            # nothing
answer = square(4)
```

```
println(answer + 1)        # 17
```

جعل الاستدعاء الأول `answer` يحمل `nothing`، أما الثاني فجعله يحمل عددًا يمكننا أن نضيف إليه. أعيدوا النتائج من الدوال، واطبعوها عند الاستدعاء عندما تريدون رؤيتها.

5.5 التعديل في المكان واصطلاح علامة !

يمكن للدالة تغيير مصفوفة مررها إليها المستدعي. ويُتّبع اصطلاح وضع علامة ! في اسم مثل هذه الدالة:

```
function double_values!(values)
    for i in eachindex(values)
        values[i] = 2 * values[i]
    end
    return values
end
values = [1, 2, 3]
another_name = values
double_values!(values)
println(values)          # [2, 4, 6]
println(another_name)    # [2, 4, 6]
```

يطبع الاسمان الشيء نفسه لأنهما يشيران إلى متجه واحد. وتنشئ `copy` متجهًا مستقلًا:

```
values = [1, 2, 3]
another_name = values
another_vector = copy(values)
double_values!(values)
println(values)          # [2, 4, 6]
println(another_name)    # [2, 4, 6]
println(another_vector)  # [1, 2, 3]
```

أما إعادة إسناد اسم فلا تعدّل الكائن نفسه: بل تجعل ذلك الاسم يشير إلى كائن مختلف، وتترك الأسماء الأخرى من دون تغيير.

علامة التعجب وعد للقارئ

لا تفرض Julia استخدام !؛ فهي اصطلاح يخبر القارئ بأن أحد الوسائط سيتغير. ينبغي للدالة التي تغيّر متجهًا أن تصرّح بذلك في اسمها. أما الدالة التي تحسب متجهًا جديدًا فلا تستخدم هذه العلامة.

5.6 البث: التطبيق عنصرًا عنصرًا

يمكن تطبيق معظم الحسابات العددية المفردة على كل عنصر في مصفوفة دفعةً واحدة. وتحقق ذلك نقطة توضع قبل العامل أو قبل قوسي الاستدعاء:

```
xs = [1, 2, 3]
println(square.(xs))                  # [1, 4, 9]
println(sqrt.([1.0, 4.0, 9.0]))       # [1.0, 2.0, 3.0]
println(shifted.(xs, 10))             # [11, 12, 13]
println(affine.(xs; scale = 2, offset = 1))   # [3, 5, 7]
println(xs .^ 2 .+ 1)                 # [2, 5, 10]
println(xs .* [10, 20, 30])           # [10, 40, 90]
```

يستدعي التعبير `shifted.(xs, 10)` الدالة ذات الوسيطين مرةً لكل عنصر من `xs`، مع إعادة استخدام العدد المفرد 10. وتُمرر الوسائط المسمّاة من دون تغيير إلى كل استدعاء.

يضيف الماكرو .@ النقاط بدلًا منا، فيمكن كتابة الصيغة العددية بصورتها المعتادة:

```
grid = [-2.0, -1.0, 0.0, 1.0, 2.0]           # dimensionless coordinates
gaussian_macro = @. exp(-0.5 * grid^2)
println(round.(gaussian_macro; digits = 4))   # [0.1353, 0.6065, 1.0,
0.6065, 0.1353]
```

يتحول الإسناد داخل .@ إلى إسناد يعدّل العناصر في أماكنها:

```
gaussian_buffer = zeros(length(grid))
@. gaussian_buffer = exp(-0.5 * grid^2)
```

يحتفظ المتجه المخصص للتخزين المؤقت بهويته، وتتغير عناصره فقط. أبقوا عمليات الاختزال مثل sum والاستدعاءات مثل println خارج الماكرو، إلا إذا كنتم تريدون بالفعل تطبيقها عنصرًا عنصرًا أيضًا.

5.7 صيغ بناء المصفوفات بالاستيعاب

تبني صيغة الاستيعاب مصفوفةً من حلقة في تعبير واحد. نبدأ بالصيغة التي تستخدم حلقة صريحة لتكون مرجعًا لنا:

```
squares_loop = zeros(Int, 4)
for i in 1:4
    squares_loop[i] = i^2
end
squares_comp = [i^2 for i in 1:4]
println(squares_comp)             # [1, 4, 9, 16]
println(squares_loop == squares_comp)   # true
```

تؤدي الفاصلة بين المولّدات إلى بناء مصفوفة ثنائية الأبعاد:

```
table_loop = zeros(Int, 3, 4) # 3 rows, 4 columns
for i in 1:3
    for j in 1:4
        table_loop[i, j] = i * j
    end
end
table_comp = [i * j for i in 1:3, j in 1:4]
println(table_comp)               # [1 2 3 4; 2 4 6 8; 3 6 9 12]
println(table_loop == table_comp) # true
```

تعيد كل صيغة استيعاب مصفوفةً جديدة. استخدموا الحلقة الصريحة عندما ينبغي لتكرار واحد أن ينفذ عدة أعمال مختلفة؛ واستخدموا صيغة الاستيعاب عندما ينتج كل تكرار عنصرًا واحدًا بالضبط.

5.8 القواسم والأعداد الأولية

القاسم الموجب للعدد n يقسمه من دون باقٍ. وعندما نكتب القواسم أولًا، نستطيع تعريف العدد الأولي من دون اللجوء إلى اختصار:

```
function divisors(n)
    result = Int[]
    for d in 1:n
        if n % d == 0
            push!(result, d)
        end
    end
    return result
```

```
end

println(divisors(1))    # [1]
println(divisors(2))    # [1, 2]
println(divisors(12))   # [1, 2, 3, 4, 6, 12]
println(divisors(0))    # Int64[]
```

تستمر الحلقة حتى n لأننا طلبنا جميع القواسم. وتفحص طريقة أسرع الأعداد حتى `isqrt(n)` فقط، وتقرن كل قاسم d بالقاسم n/d؛ أما النسخة البسيطة فيسهل التحقق منها بالنظر.

تبني صيغة الاستيعاب التي تعلمناها في القسم السابق المتجه نفسه:

```
list_divisors = [d for d in 1:12 if 12 % d == 0]
println(list_divisors == divisors(12))   # true
```

نصل الآن إلى التعريف. للعدد الأولي قاسمان موجبان بالضبط، هما 1 والعدد نفسه:

```
function is_prime(n)
    return length(divisors(n)) == 2
end

println(is_prime(1))    # false: one divisor
println(is_prime(2))    # true: 1 and 2
println(is_prime(12))   # false
println(is_prime(97))   # true
println([n for n in 1:30 if is_prime(n)])   # [2, 3, 5, 7, 11, 13, 17,
19, 23, 29]
```

يجعل هذا التعريف 2 عددًا أوليًا و1 غير أولي، وهاتان حالتان يكثر فيهما الخطأ. أما اختبار الأولية الذي يبحث عن قاسم بين 2 و$\sqrt{n}$ فهو خوارزمية مختلفة، وهو مفيد، لكنه ليس التعريف.

5.9 تكامل حركة بندول بطريقة أويلر

تتدلى كتلة نقطية من قضيب صلب طوله L مثبت عند محور عديم الاحتكاك. تُقاس الزاوية θ من الاتجاه الرأسي نحو الأسفل، وتكون موجبةً عكس اتجاه عقارب الساعة، ونعرّف $\omega = \dot{\theta}$. عزم الإرجاع هو $-mgL\sin\theta$، وبقسمته على عزم القصور الذاتي mL^2 نحصل على معادلة الحركة

$$\ddot{\theta} = -\frac{g}{L}\sin\theta.$$

نحتفظ بدالة الجيب؛ فتقريب الزاوية الصغيرة $\sin\theta \approx \theta$ هو بالضبط ما نريد تجنبه.

```
function pendulum_rhs(state; length = 1.0, gravity = 9.81)
    θ, ω = state
    return ω[, -(gravity / length) * sinθ()]
end
```

تتكون الحالة من الزوج (θ, ω)، وتعيد الدالة مشتقتيهما الزمنيتين. ثم تقدّم طريقة أويلر الأمامية الحالةَ باستخدام المشتقة عند الزمن الحالي:

$$\text{state}_{\text{next}} = \text{state}_{\text{now}} + \Delta t \cdot \text{derivative}_{\text{now}}.$$

بسبب التناظر، يحدث أول عبور للصفر في الاتجاه الهابط بعد ربع اهتزازة كاملة؛ ولذلك فإن أربعة أمثال زمن هذا العبور تساوي الزمن الدوري:

```
function pendulum_periodθ(0; length = 1.0, gravity = 9.81, dt = 0.001)
    state = θ[0, 0.0]
    t = 0.0
    while true
        derivative = pendulum_rhs(state; length = length, gravity =
gravity)
        θ = state[1]
        θ_next = θ + dt * derivative[1]
        state[1] = θ_next
        state[2] = state[2] + dt * derivative[2]
        if θ_next <= 0
            crossing = t + dt * θ / θ( - θ_next)
            return 4 * crossing
        end
        t += dt
    end
end
```

نحسب زمن العبور بالاستيفاء بين آخر عينة موجبة وأول عينة سالبة، فلا تقفز الإجابة بمقدار الخطوة الزمنية. تنتهي الحلقة عند أول عبور هابط؛ وعند الإطلاق من السكون بزاوية θ_0 تظل الزاوية بين $-\theta_0$ وθ_0.

صندوق الفيزياء: البندول ذو القضيب الصلب

نفترض كتلة نقطية على قضيب صلب مهمل الكتلة طوله L بالمتر، ومحورًا ثابتًا عديم الاحتكاك، وجاذبية منتظمة g بالمتر لكل ثانية مربعة، من دون تخميد أو قوة دفع خارجية. يسمح القضيب الصلب بالإطلاق من زوايا أكبر من $90°$؛ أما الخيط المرن فيرتخي إذا أُطلق من السكون عند تلك الزوايا. تُقاس الزوايا بالراديان انطلاقًا من الاتجاه الرأسي نحو الأسفل، وتكون موجبةً عكس اتجاه عقارب الساعة؛ وتقاس ω بالراديان لكل ثانية.

$$\ddot{\theta} = -\frac{g}{L}\sin\theta, \qquad \frac{\mathrm{d}}{\mathrm{d}t}\begin{pmatrix}\theta\\ \omega\end{pmatrix} = \begin{pmatrix}\omega\\ -(g/L)\sin\theta\end{pmatrix}.$$

يعطي حد الزاوية الصغيرة $\sin\theta \approx \theta$ زمنًا دوريًا $T_0 = 2\pi\sqrt{L/g}$. ويزداد الزمن الدوري غير الخطي مع السعة، ويتباعد عندما تقترب $\theta_0 \to \pi$. يغيّر تكامل أويلر طاقة المهتز ببطء، ولذلك فإن الزمن الدوري المحسوب به تقريب تعليمي وليس قياسًا دقيقًا. لا يتضمن النموذج ملامسة الأرض أو مرونة القضيب أو حركة المحور.

5.10 الزمن الدوري بدلالة زاوية الإطلاق

بعد أن أصبح الزمن الدوري متاحًا بدالة، تكفينا صيغة استيعاب واحدة لحساب اعتماده على زاوية الإطلاق:

```
using Plots
let
    angles_deg = 5:5:175
    periods = [pendulum_period(deg2radθ()) for θ in angles_deg]
    p = plot(angles_deg, periods; xlabel = "Release angle (degrees)",
        ylabel = "Period (s)", title = "Pendulum period", legend = false,
        linewidth = 2, marker = :circle, markersize = 3)
    savefig(p, joinpath(@__DIR__, "week05-pendulum-period.png"))
    display(p)
end
```

يكون المنحنى شبه أفقي قرب الصفر، ويرتفع بشدة كلما اقتربت زاوية الإطلاق من $180°$. عندما يكون $L = 1$ متر، يبلغ الزمن الدوري للزوايا الصغيرة نحو 2.006 ثانية، ويصبح أطول بنحو 18% عند $90°$. وهذا الرسم نتيجة للنموذج، وليس قياسًا لقضيب بعينه.

5.11 نظرة إلى الخطوة التالية

نتناول في حصة التمرين المرافقة دالةً صغيرةً واحدة قبل أن تبدأوا الواجب المنزلي. ويطلب الواجب الأساسي المنفصل كتابة الطرف الأيمن لمعادلات حركة مقذوف يتعرض لمقاومة تربيعية، وتنفيذ مكامل بطريقة أويلر، والبحث بين زوايا الإطلاق عن أفضل رمية. أما النشاط الإضافي الاختياري فيكامل حركة البندول المزدوج، إذ تظهر معادلتاه المقترنتان في صورة معادلة متجهية واحدة. ولا تُعد مواد النشاط الإضافي أو التوسع الاختياري متطلبًا للعمل اللاحق.

لنحمل معنا هذه العادة: عندما يتكرر حساب، نمنحه اسمًا، ونوضح مدخلاته وما يعيده، ثم نستخدمه حيثما نحتاج إليه.